UNIP

UNIVERSIDADE PAULISTA

Programação Orientada a Objetos

U524.17

UNIP

UNIVERSIDADE PAULISTA

Programação Orientada a Objetos

Autor: Prof. Ricardo da Costa Veras
Colaboradoras: Profa. Vanessa Santos Lessa
Profa. Larissa Rodrigues Damiani

Professor conteudista: Ricardo da Costa Veras

Mestre em Engenharia da Informação pela Universidade Federal do ABC (UFABC). Especialista em Orientação a Objetos pela Faculdade de Informática e Administração Paulista (Fiap). Graduado em Engenharia Elétrica com ênfase em Eletrônica pela Escola de Engenharia Mauá. Desde 2009 é professor da Universidade Paulista (UNIP) e vem ministrando aulas para os cursos de Ciência da Computação, Sistemas de Informação e Engenharia (básico). Tem experiência na área de Tecnologia da Informação desde 1995, tendo trabalhado com consultoria e prestação de serviços de análise e desenvolvimento de sistemas em diversas empresas.

Dados Internacionais de Catalogação na Publicação (CIP)

V476l Veras, Ricardo da Costa.

Programação Orientada a Objetos / Ricardo da Costa Veras. – São Paulo: Editora Sol, 2026.

164 p., il.

Nota: este volume está publicado nos Cadernos de Estudos e Pesquisas da UNIP, Série Didática, ISSN 1517-9230.

1. Variáveis. 2. Polimorfismo. 3. Modificador. I. Título.

CDU 681.3.062

U524.17 – 26

© Todos os direitos reservados. Nenhuma parte desta obra pode ser reproduzida ou transmitida por qualquer forma e/ou quaisquer meios (eletrônico, incluindo fotocópia e gravação) ou arquivada em qualquer sistema ou banco de dados sem permissão escrita da Universidade Paulista.

Prof. João Carlos Di Genio
Fundador

Profa. Sandra Rejane Gomes Miessa
Reitora

Profa. Dra. Marilia Ancona Lopez
Vice-Reitora de Graduação

Profa. Dra. Marina Ancona Lopez Soligo
Vice-Reitora de Pós-Graduação e Pesquisa

Profa. Dra. Claudia Meucci Andreatini
Vice-Reitora de Administração e Finanças

Profa. M. Marisa Regina Paixão
Vice-Reitora de Extensão

Prof. Fábio Romeu de Carvalho
Vice-Reitor de Planejamento

Prof. Marcus Vinícius Mathias
Vice-Reitor das Unidades Universitárias

Profa. Silvia Renata Gomes Miessa
Vice-Reitora de Recursos Humanos e de Pessoal

Profa. Laura Ancona Lee
Vice-Reitora de Relações Internacionais

Profa. Melânia Dalla Torre
Vice-Reitora de Assuntos da Comunidade Universitária

UNIP EaD

Profa. Elisabete Brihy
Profa. M. Isabel Cristina Satie Yoshida Tonetto

Material Didático

Comissão editorial:
Profa. Dra. Christiane Mazur Doi
Profa. Dra. Ronilda Ribeiro

Apoio:
Profa. Cláudia Regina Baptista
Profa. M. Deise Alcantara Carreiro
Profa. Ana Paula Tôrres de Novaes Menezes

Projeto gráfico:
Prof. Alexandre Ponzetto

Revisão:
Lucas Ricardi
Vitor Andrade

Sumário

Programação Orientada a Objetos

APRESENTAÇÃO 7
INTRODUÇÃO 9

Unidade I

1 INTRODUÇÃO À ORIENTAÇÃO A OBJETOS 11
- 1.1 A IDE do Eclipse 12
- 1.2 O método main 16
- 1.3 Classes, objetos, atributos e métodos 16
- 1.4 Algumas convenções de programação em Java 17
- 1.5 Meu primeiro projeto Java 19
- 1.6 Os elementos e suas sintaxes 21

2 VARIÁVEIS, ESTRUTURAS DE CONTROLE E ORGANIZAÇÃO DO PROJETO 27
- 2.1 Tipos de variáveis 27
- 2.2 Strings 29
- 2.3 Estruturas condicionais 32
  - 2.3.1 if – else if – else 32
  - 2.3.2 switch – case 35
- 2.4 Estruturas de repetição 37
  - 2.4.1 for 38
  - 2.4.2 while 40
  - 2.4.3 do – while 41
- 2.5 Packages (pacotes) 43

Unidade II

3 LIDANDO COM AS VARIÁVEIS E SEUS ACESSOS 58
- 3.1 Promotion e casting 58
- 3.2 Modificadores de acesso 60

4 ELEMENTOS CARACTERÍSTICOS DA ORIENTAÇÃO A OBJETO 63
- 4.1 Encapsulamento 63
- 4.2 Sobrecarga de métodos 64
- 4.3 Método construtor 65
- 4.4 Herança 67
- 4.5 Sobrescrita de métodos 70
- 4.6 ArrayList e Vectors 73
- 4.7 Trabalhando com strings 75

Unidade III

5 POLIMORFISMO....101
5.1 Polimorfismo de variáveis....101
5.2 Polimorfismo de métodos....102
5.3 Polimorfismo de classes....102
6 MODIFICADORES DE COMPORTAMENTO E ALGUMAS CLASSES ÚTEIS....106
6.1 Modificadores de comportamento....106
6.2 Modificador final....107
6.3 Modificador static....109
6.4 Modificador abstract....110
6.5 Tabela-resumo sobre os modificadores de comportamento....112
6.6 Interface....114
6.6.1 Por que utilizamos interfaces?....115
6.7 Classe JOptionPane....116
6.8 Wrapper classes....119
6.9 Transformando textos em números: utilizando as wrapper classes....119

Unidade IV

7 EXCEÇÕES....130
7.1 Hierarquia das exceções....132
7.2 Tratamento de exceções....134
7.2.1 Tratando várias exceções....136
7.3 Lançamento de exceções....139
7.4 Criação de exceções....143
8 THREADS....147
8.1 Criando threads....148
8.1.1 Pela herança da classe thread....148
8.1.2 Usando a interface runnable....149
8.1.3 Comparando....149
8.2 Rodando threads....149

APRESENTAÇÃO

O paradigma orientado a objetos (OO) vem sendo vastamente divulgado e utilizado desde a década de 1990 (por volta de 1993) a partir do surgimento da internet. No entanto, seu conceito remete a épocas anteriores, desde os anos 1960.

Tendo sido idealizado por matemáticos e estudiosos da tecnologia da informação, sendo um de seus criadores o estadunidense Alan Kay, o paradigma OO surgiu da evolução dos processos de programação, acompanhando o progresso tecnológico, a necessidade de organização da codificação e a evolução visionária dos procedimentos e entidades de programação.

Na época, o caos enfrentado pelos programadores na análise de centenas de milhares de linhas de código em sequência pedia um novo paradigma, ou seja, uma nova concepção de estruturação que o processo de design (codificação de sistemas) deveria respeitar, a fim de acompanhar de forma ágil o progresso tecnológico, além de proporcionar um melhor entendimento do sistema criado, agilizando suas futuras implementações ou manutenções.

A linguagem Java foi desenvolvida a partir da ideia de ser multiplataforma. Para que isso fosse possível, seus desenvolvedores criaram uma máquina virtual (um software JVM – Java Virtual Machine) para cada plataforma, sendo ela um software que roda programas criados na linguagem Java, de modo que qualquer programa feito naquela linguagem (em qualquer uma das plataformas) pudesse rodar em outra plataforma sem ocorrer problemas em sua funcionalidade, bastando que ali estivesse instalada a máquina virtual Java.

No entanto, essa JVM não pode existir sozinha. Ela faz parte de um Java Runtime Environment (JRE), que é todo o ambiente de execução da linguagem Java, o qual contém a JVM, toda a biblioteca de classes Java e o coletor de lixo (garbage collector), que permite ao programador gerar códigos sem se preocupar em ficar eliminando elementos da memória ao longo do funcionamento do programa à medida que eles não são mais utilizados.

Assim, para que um programa Java funcione em um dispositivo, basta que ele tenha instalado o JRE adequado, de acordo com a plataforma, pois sem ele não é possível acionar e rodar qualquer programa criado nessa linguagem.

Se o leitor for um curioso da programação, vale a pena fazer o download do JDK (Java Development Kit), um kit de desenvolvimento que, além do JRE e, consequentemente, da JVM, possui todo o código fonte do Java (os arquivos com extensão .java) e o JavaDoc (documentação básica de todas as classes existentes na biblioteca, podendo esta ser acessada de modo off-line), além de algumas outras ferramentas de programação.

Em 1991, uma equipe liderada por James Gosling (com a parceria de Patrick Naughton e Mike Sheridan) participou de um projeto pela empresa Sun Microsystems com a intenção de criar uma tecnologia de dispositivos digitais que seria capaz de unir equipamentos eletrônicos diversos (como a televisão, entre outros aparelhos) com o computador. Dessa nova tecnologia, nasceu uma linguagem

de programação cujo nome era **Oak**, que inicialmente seria embutida em dispositivos de controles remotos de televisão, além da própria televisão, e que permitiria gerar solicitações de controle da programação de TV, de controle de vídeo e de controle da própria televisão. Essa tecnologia, apesar de visionária, veio cedo demais e não conseguiu atingir o mercado tecnológico da época, pois exigiria a construção de uma infraestrutura (redes de servidores e de cabos de comunicação) economicamente inviável para sua comercialização.

Porém, com o surgimento da internet, a mesma equipe enxergou uma oportunidade de lançar aquela tecnologia, já que os meios físicos e toda a infraestrutura de comunicação estavam se expandindo naturalmente com a própria internet. Foi então que, com a liberação da internet para o uso das empresas e a expansão do uso por particulares, os criadores da linguagem Oak, cujo nome não poderia ser reutilizado para esta nova linguagem, criaram outra linguagem, à qual deram o nome de **Java**, totalmente adaptada àquela nova tecnologia, a internet.

Assim, em 1995 lançaram a primeira versão do Java, com características inovadoras para a época: uma linguagem inteiramente orientada a objetos, simples, dinâmica, segura, multitarefa, multiplataforma e que não dependeria de outras linguagens.

Antes daquela época, a internet era um meio extremamente versátil de comunicação, porém restrita apenas a informações, não possibilitando a interação direta dos usuários (por exemplo, para fazer compras ou solicitar serviços). Com a linguagem Java, construiu-se uma série de programas (applets) que eram transportados pela internet e que rodavam de forma local nos computadores, dando às páginas uma funcionalidade dinâmica e inovadora e que poderia ser utilizada por qualquer um dos usuários da internet.

A facilidade de criação de sistemas diversos com aquela linguagem, sua versatilidade de acompanhar as novas tecnologias, além do fato de funcionar em diversas plataformas, fizeram crescer o interesse pela sua utilização entre os desenvolvedores e as empresas.

A partir de então, e ao longo dos anos, a Sun Microsystems lançou diversas versões da linguagem, acompanhando as necessidades e as tecnologias crescentes. Em 2007, a Sun acabou por tornar a linguagem Java totalmente open-source (de código aberto), ou seja, permitiu que qualquer um pudesse ter acesso a todo código fonte (original) da linguagem, podendo inclusive tornar-se "parceiro" na sua construção e evolução.

Entre 2009 e 2010, a Sun Microsystems foi adquirida pela empresa Oracle Corporation, a qual ficou responsável por divulgar, acompanhar e controlar as necessidades de crescimento da linguagem Java.

Ao final de 2014, mais de 9 milhões de desenvolvedores por todo o mundo já programavam em Java, transformando essa linguagem numa das mais utilizadas.

INTRODUÇÃO

A linguagem de programação apresentada neste livro-texto segue o paradigma de orientação a objetos (OO), o qual se baseia na criação de entidades de programação que representam elementos do mundo real (uma pessoa, um cliente, uma empresa, uma área de uma empresa, um produto de uma empresa etc.).

Alguns exemplos de linguagens de programação que seguem o paradigma de OO são: Java, C#, C++, PHP, Ruby, Python, entre outras.

Neste livro-texto, focaremos o estudo da OO com base na linguagem de programação **Java**, que é totalmente voltada a esse paradigma. Essa linguagem vem mantendo desde 2001 sua posição entre as quatro primeiras linguagens mais utilizadas por programadores e pelas empresas, segundo o Índice Tiobe (Tiobe [...], [s.d.]).

Aqui, pretende-se organizar o aprendizado do paradigma OO e da linguagem Java de forma progressiva e sequencial a fim de proporcionar um processo didático eficiente e prazeroso. A intenção deste curso não é apresentar todos os conceitos existentes sobre a linguagem Java, mas, sim, os seus principais conceitos básicos, juntamente aos conceitos básicos do paradigma OO.

Sem dúvida nenhuma, o constante crescimento da tecnologia e de seus recursos reflete na linguagem Java. Sempre, ao longo de nossa vida profissional, precisamos ficar atentos à sua evolução, sendo constante o seu estudo e aprendizado.

Este livro-texto está dividido em quatro unidades, cujo conteúdo segue a seguinte estruturação:

- **Unidade I**: serão apresentados os elementos básicos de orientação a objetos (**classe**, **objeto**, **atributo** e **método**) e a estrutura lógica oferecida pela linguagem Java (**tipagem**, **estrutura condicional** e de **repetição**).

- **Unidade II**: serão apresentados recursos específicos de orientação a objetos, como o **encapsulamento**, a **herança** e o **polimorfismo**, assim como sua base de entendimento – os modificadores **public**, **private**, **protected** e **(default)** e os **métodos construtores** –, recursos esses que compõem os pilares desse paradigma.

- **Unidade III**: será explicado o **polimorfismo de classes**, além de apresentadas algumas ferramentas da linguagem relacionadas ao controle da estruturação do sistema, que alteram o comportamento dos elementos trabalhados na sua codificação (os modificadores **static**, **final** e **abstract** e as **interfaces**).

- **Unidade IV**: serão estudados recursos de tratamento de exceções, como os erros em tempo de execução de programas (**exception**), assim como será estudado o conceito de multitarefa ou de processamento paralelo (**thread**).

O empenho no estudo desta disciplina possibilitará ao estudante adquirir uma base sólida de conhecimentos que o permitirá evoluir para um estudo continuado. Além do embasamento teórico apresentado, é importante que o estudante dedique boa parte de seus estudos aos exercícios práticos sugeridos ao longo deste material, pois só assim garantirá seu real aprendizado.

Persista em seus estudos. Empenhe-se em adquirir conhecimentos além deste livro-texto, como os que são sugeridos ao longo da leitura. Essa será a base de seu crescimento profissional. Aproveite bem este curso e garanta o seu sucesso.

Bons estudos!

Unidade I

1 INTRODUÇÃO À ORIENTAÇÃO A OBJETOS

O paradigma orientado a objetos foi criado e pensado com base em três conceitos fundamentais, considerados os pilares da orientação a objetos:

- herança;
- encapsulamento;
- polimorfismo.

Esses três conceitos serão vistos com mais detalhes ao longo deste livro-texto, após vermos conceitos mais básicos da linguagem. Basicamente, a herança é um recurso que permite o reaproveitamento de código, o que remete ao conceito de generalização e especialização; o encapsulamento protege a utilização inapropriada dos atributos de uma classe, de modo a possibilitar o controle de seus valores por meio dos seus métodos; e o polimorfismo é um recurso em que os elementos podem se comportar de formas diferentes dependendo de como são estruturados (e dependendo da necessidade do programa).

A orientação a objetos é um paradigma de linguagem de programação que tem como princípio a utilização de elementos que representam entidades do "mundo real", ou seja, elementos existentes numa situação real (uma questão, um problema, uma solução etc.).

Como exemplo disso, poderíamos citar a construção de um sistema para auxiliar nas atividades de controle de RH ou de finanças de uma empresa, no qual são criadas classes que representam as entidades existentes naquele controle, como o funcionário, a empresa, os usuários, a gerência, a diretoria etc. Outro exemplo poderia ser um sistema de compra e venda de produtos em que são criadas classes que representam os produtos, o vendedor, o cliente, o pagamento, o banco etc. Um último exemplo: se o sistema criado for um jogo de corrida de automóveis, as classes poderiam representar os carros, a pista, o usuário/jogador etc.

Percebam que as entidades representadas são elementos ou algo que encontramos e com que convivemos no dia a dia, na vida profissional, nas necessidades diárias ou no entretenimento, e no programa nós criamos uma representação dessas entidades em forma de classes. Essa é a capacidade de abstração da linguagem orientada a objetos, ou seja, trazer para o programa uma representação dos elementos reais com os quais estamos trabalhando.

Um objeto é um elemento criado **em memória** (na memória RAM) a partir do modelo da classe, de forma a possuir todas as suas características. Já a classe é o **molde** para criarmos os objetos. Como

exemplo, poderíamos pensar que uma classe Pessoa poderia ser a ideia de pessoa em que se definem as possíveis características que ela poderá possuir e as possíveis ações que poderão ser acionadas. Assim, esse modelo deverá permitir a construção de um objeto que poderá ter, por exemplo, um nome, uma idade, um CPF, uma altura, assim como ações que poderão ser acionadas, como cadastrar, contatar, pagar, contratar etc. Dessa forma, o objeto que efetivamente será gerado a partir daquela classe possuirá todas as suas características, mas com valores definidos, tais como o nome **João Batista**, a idade de **30 anos**, o CPF **123.456.789-10**, a altura de **1,60 m** etc., além de permitir acionar qualquer das ações predeterminadas para a classe que o gerou.

Quando dizemos que o programa é orientado a objetos, estamos especificando que apesar de programarmos pensando nos modelos (nas classes) daqueles elementos que queremos trabalhar, devemos sempre pensar que quando o sistema estiver sendo utilizado, esses elementos existirão na memória RAM na forma de **objetos** que representam as entidades reais, ou seja, é um programa criado com base em classes, mas orientado a funcionar com base em objetos na memória.

Neste curso, para que possamos construir nossos exemplos, iremos utilizar um software específico, a IDE do Eclipse, que nos possibilitará colocar em prática todos os exemplos e aprendizados contidos neste livro-texto.

1.1 A IDE do Eclipse

Uma IDE (**i**ntegrated **d**evelopment **e**nvironment – ambiente integrado de desenvolvimento) é um software que fornece um ambiente integrado para se trabalhar com o desenvolvimento de outros softwares, de acordo com a linguagem de programação com que se trabalha. Geralmente esses ambientes integram as áreas de texto, onde escrevemos as linhas de código do nosso programa; as facilidades de identificação dos termos necessários ao código descrito; a geração automática dos códigos compilados (quando necessários); a possível execução rápida do programa criado; a fácil verificação dos erros que cometemos ao codificar um programa; além de facilitadores de programação que nos ajudam a gerar elementos específicos em nossos programas.

Para executar todos os exemplos e exercícios aqui apresentados, precisamos instalar o Java Runtime Environment (JRE) e a IDE do Eclipse, que é um software livre cujo download pode ser feito no endereço https://www.eclipse.org/downloads/. Os processos de instalação são detalhados a seguir.

Instalando o Java

Para este curso, seu micro deve ter instalada a JRE (J2SE) do Java.

Para baixar o arquivo de instalação mais recente, digite em alguma página de busca da internet o seguinte texto:

java jre j2se

Procure o link da Oracle com o texto:

Java 2 Platform, Standard Edition (J2SE) <número_versão> - Oracle

Procure pelo link equivalente a:

Java 2 Platform, Standard Edition version <número_versão>

Após clicar nesse link, na página que abrir, clique no link equivalente à última versão do Java SE <release>, o qual abrirá uma página com diversos links para diversas plataformas (Linux, MacOS e Windows).

Selecione a plataforma de seu micro (existe um link para Windows com o texto Windows x64 Installer, que baixará um arquivo executável de instalação do Java). Esse é um arquivo que instala a JDK que contém a JRE necessária.

Execute o arquivo baixado, instalando o J2SE.

Instalando o Eclipse

Agora vamos instalar a IDE, que nos possibilitará gerar projetos de sistemas em Java. No caso, vamos instalar o Eclipse, que é uma IDE criada inicialmente pela IBM e hoje pertence a uma comunidade de empresas e programadores, a qual, por sua vez, pertence a um projeto continuado pela Eclipse Foundation, uma associação europeia que presta serviços e dá o suporte para auxiliar ao projeto de divulgação, desenvolvimento e elaboração das novas versões dessa linguagem.

Para instalar a versão 64 bits do Eclipse, basta entrar no link a seguir:

https://www.eclipse.org/downloads/

A partir desse link, baixa-se um arquivo executável *.exe.

Na página desse link, clique no botão Download x86_64.

Após abrir a nova página, clique no botão Download.

Após o término do download, deve-se executar o arquivo de instalação.

Para instalar a versão 32 bits do Eclipse (para micros mais antigos), basta entrar na página a seguir referente ao Eclipse Luna, uma versão estável para micro 32 bits dessa IDE:

https://www.eclipse.org/downloads/packages/release/luna/sr2

É necessário clicar no link 32-bit do Eclipse IDE for Java Developers (figura 1). Com esse link, baixa-se um arquivo compactado *.zip.

Figura 1 – Trecho da página de download de arquivos relacionado ao Eclipse na sua versão Luna

Nesse último caso, faz-se o download de um arquivo compactado, que deve ser descompactado no diretório raiz de seu micro (c:\).

Em qualquer uma das instalações, será gerado um diretório eclipse na raiz do micro (c:\eclipse), dentro do qual existe o arquivo executável do Eclipse (eclipse.exe).

O arquivo Eclipse.exe é o arquivo de inicialização do aplicativo Eclipse. Se você instalou uma versão atual, ao ser acionado, já é aberta a área de programação:

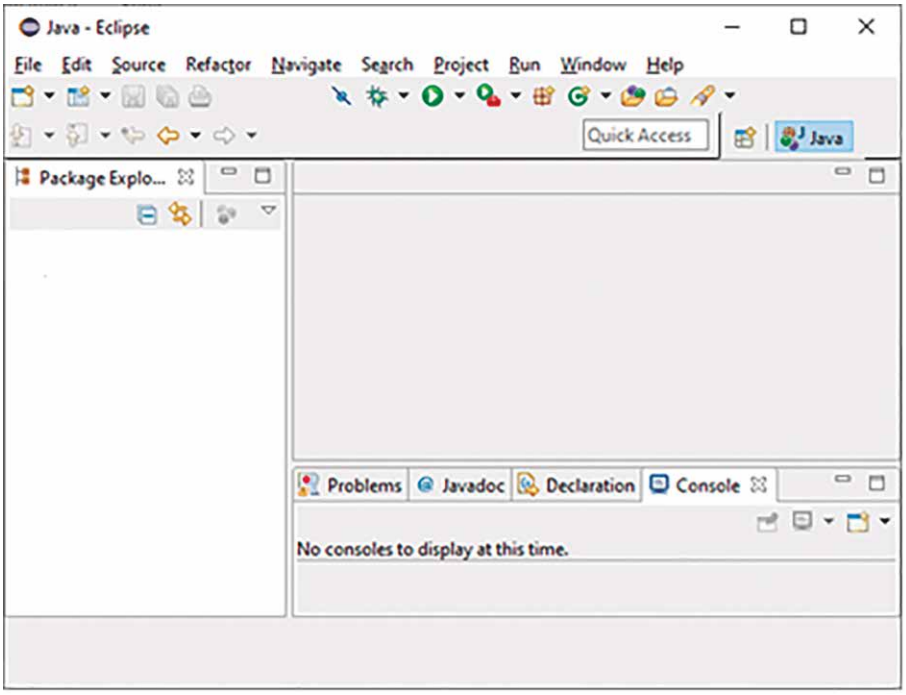


Figura 2 – Tela com a visão de programação (perspectiva) Java do Eclipse – no caso, a tela do Eclipse Luna, cuja disposição dos quadros é semelhante à de outras versões

Caso você inicie o Eclipse na sua versão Luna, ele abre em sua tela de Welcome (figura 3).

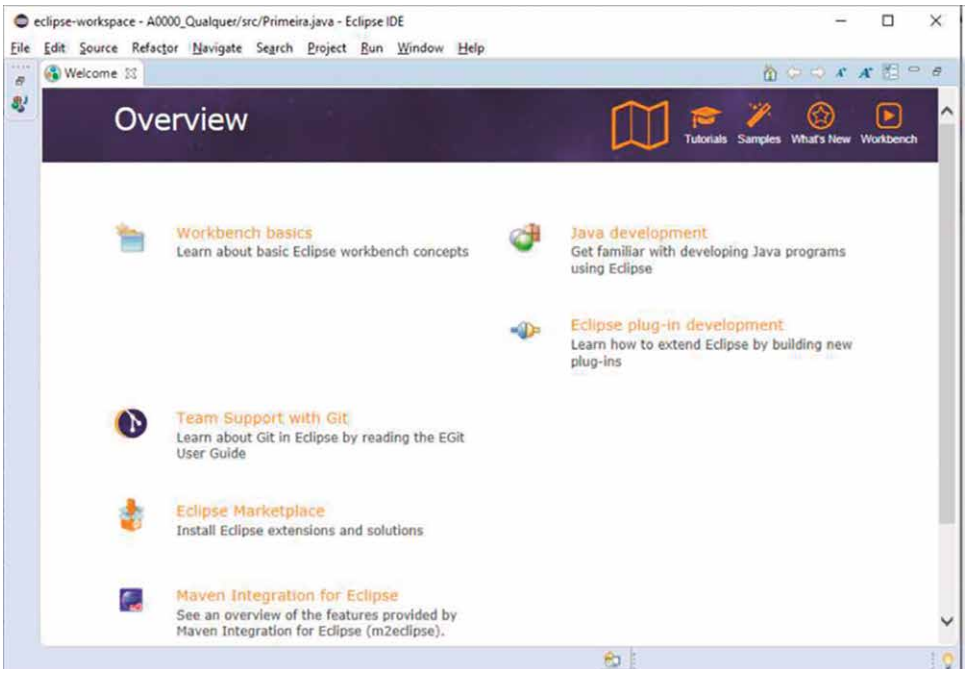


Figura 3 – Tela de Welcome do Eclipse Luna

Neste caso, para abrir o ambiente de programação, deve-se clicar no ícone Workbench, que fica no canto superior direito da tela:

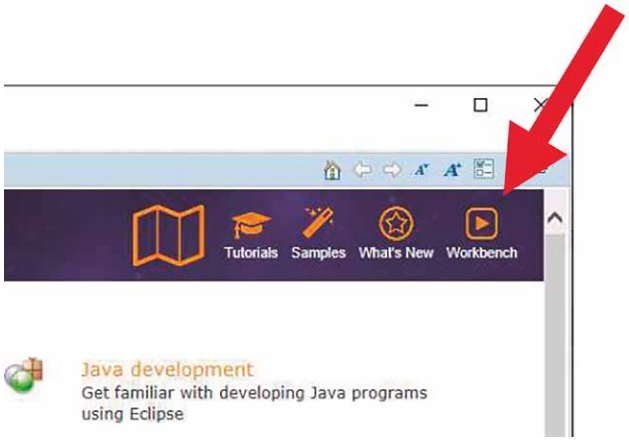


Figura 4 – Canto superior direito da tela de Welcome do Eclipse Luna
com o acesso à área de programação (link Workbench)

Sempre que neste livro-texto for dado um exemplo de código, devemos criá-lo em uma classe gerada no Eclipse, a partir do pacote src de um projeto Java (pacote raiz das classes de um projeto Java). Algumas classes deverão ser criadas com o método main; outras, sem esse método. Essa necessidade estará especificada no exemplo ou no exercício proposto.

1.2 O método main

O método main é o método responsável por acionar um programa Java. Na maioria dos casos, esse método precisa existir em apenas uma das classes do sistema construído (apenas naquela que inicialmente acionará o programa).

Uma vez acionado o método main, a partir dele podemos acionar diversos outros métodos pertencentes a uma infinidade de outras classes.

Sua sintaxe, dentro de uma classe, será:

```
public class ClasseA {
    public static void main(String[] args) {
            // ...linhas de código
    }
}

Ou, ainda:

public class Calculo {
    public static void main(String[] args) {
      double x = 7.45 + 2.91;
      int y = -47 + 82;
      System.out.println(x + " :: " + y);
    }
}
```

Para executar o método main de uma classe no Eclipse, basta abrir o código da classe na tela central do Eclipse e clicar no menu:

Run – Run As – Java Application

1.3 Classes, objetos, atributos e métodos

Classes são modelos a partir dos quais serão construídos os objetos em memória, quando efetivamente o sistema desenvolvido for utilizado.

Uma classe possui:

- **atributos**: elementos que definem a classe, em que podem ser inseridos os valores que a caracterizam;

- **métodos**: elementos que podem ser acionados ou não e que definem as possíveis ações que podem ser executadas a partir dos objetos gerados com aquela classe.

Na UML (Unified Modeling Language – Linguagem de Modelagem Unificada), uma classe é representada como indicado na figura 5.

Figura 5 – Representação de uma classe pela UML

Perceba que a representação mostra também a ordem na qual sempre devemos dispor os elementos em uma classe. Internamente, primeiro devemos sempre elencar todos os seus atributos, e só depois descrevermos os métodos que a classe possuirá.

A classe a seguir (ClasseQualquer) é apenas um exemplo de como deve ser a disponibilidade dos elementos em uma classe:

```
public class ClasseQualquer {
    // ...Primeiro os atributos
    public int atributo01;
    public int atributo02;
    public int atributoN;
    // ...Depois os métodos
    public void metodo01() {
           // ...linhas de código
    }
    public void metodo02() {
           // ...linhas de código
    }
    public void metodoN() {
           // ...linhas de código
    }
}
```

1.4 Algumas convenções de programação em Java

Toda linguagem de programação possui algumas regras que são geradas não pelo criador da linguagem (o que estaria embutido no interpretador, ou compilador, daquela linguagem), mas pelos desenvolvedores que trabalham com aquela linguagem (os profissionais que desenvolvem sistemas).

A linguagem Java possui algumas convenções que **devem** ser seguidas em sua codificação. A seguir estão algumas regras iniciais:

- toda **classe** deve ter um nome que **inicia** com **Letra Maiúscula**;
- todo **método** deve ter um nome que **inicia** com **letra minúscula**;

- todo **atributo** deve ter um nome que **inicia** com **letra minúscula**;

- toda **constante** deve ter um nome **totalmente descrito em CAIXA ALTA** (todas as letras em **MAIÚSCULO**).

Além disto, todo **nome** que identifica uma entidade do Java, seja uma classe, um método, um atributo, uma variável local ou uma constante, deve seguir, de modo geral, as seguintes regras:

- **não** deve iniciar com número;

- **não** deve conter acentuação;

- **não** deve conter espaço;

- **não** deve conter símbolo, com exceção do underline (_);

- **não** deve ser uma palavra reservada da linguagem.

Observação

Essas últimas cinco regras estão embutidas na compilação da linguagem Java, de modo que, se não forem seguidas, é gerado um erro de compilação, e o programa fica impedido de ser executado.

Exemplo de **nomes de variáveis** que não podem ser utilizados:

- **1oAno**, pois contém número no início;

- **Nome do Aluno**, pois contém espaços internos ao nome;

- **MédiaFinal**, pois contém acentuação;

- **Codigo#Principal!**, pois contém símbolos;

- **Double**, pois é uma palavra reservada (que define um elemento na linguagem Java).

1.5 Meu primeiro projeto Java

Sempre que se cria um programa (uma classe) Java, deve-se fazê-lo em um projeto Java, seja ele já existente ou novo. Em nossos exemplos, criaremos classes a partir da raiz do projeto (o pacote ou diretório src dos projetos do Eclipse).

Vamos criar o nosso primeiro programa Java: a classe OlaMundo.

Nosso objetivo será criar uma classe (um programa Java) que, ao ser acionada, imprime na Tela da Console o texto Olá Mundo!.

Para isso, vamos **abrir o Eclipse** e gerar um novo projeto: o Projeto LPOO, nos seguintes passos:

1) Clicar em: **File – New – Project**.

2) Em seguida, na tela que aparece, selecionar a pasta Java, clicando no símbolo (>) naquela opção, e selecionar **Java Project**. Em seguida, deve-se clicar em **Next**>.

3) Inserir um nome para o projeto (por exemplo, LPOO) no campo **Project name**: e clicar em **Finish** (veja na figura a seguir o Package Explorer com o projeto gerado).

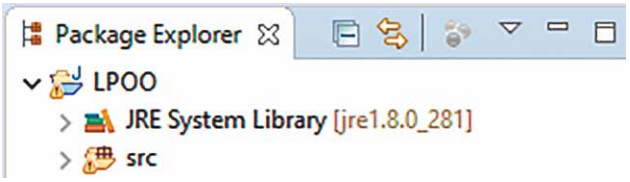


Figura 6 – Projeto LPOO no Eclipse

Agora, vamos criar a classe OlaMundo nos seguintes passos:

1) Clicar no **botão direito do mouse** sobre a package (pacote) src, mostrada na figura a seguir:

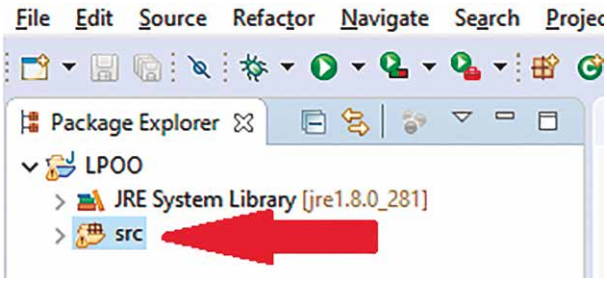


Figura 7 – Pacote src do projeto LPOO

2) Selecionar **New – Class**.

3) Na tela que se abre, no campo **Name:**, inserir o nome da classe (por exemplo, OlaMundo – note que o nome deve seguir as regras de nome de variáveis, assim como as regras de nomes de classes, como visto anteriormente). Perceba que o arquivo criado após se gerar uma classe sempre terá exatamente o mesmo nome da classe, seguido da extensão .java (no exemplo, OlaMundo.java).

Figura 8 – Arquivo e classe OlaMundo

4) Na área de edição de código, completar o programa para que fique com os seguintes comandos:

```
public class OlaMundo {
    public static void main(String[] args) {
       System.out.println("Olá Mundo !!");
    }
}
```

Observe o ponto-e-vírgula ao final da linha de comando de impressão.

5) Para rodar o código criado, clicar com o botão direito do mouse sobre o código (na própria área de edição de código) e selecionar a opção **Run As – Java Application**, de forma que, após o programa ser rodado, aparecerá na área da console a frase desejada (Olá Mundo !!):

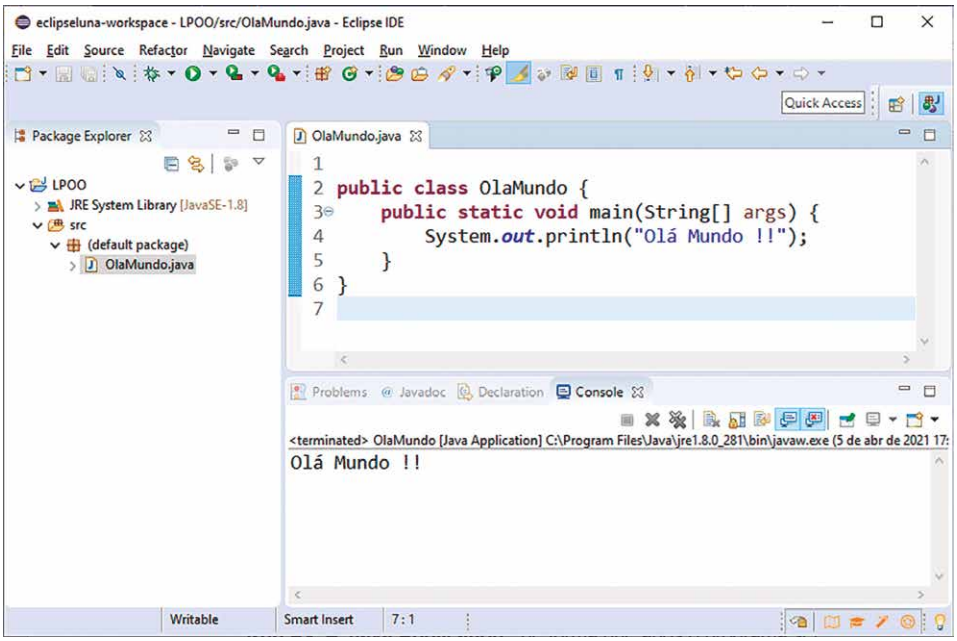


Figura 9 – Classe OlaMundo e o resultado de seu acionamento

1.6 Os elementos e suas sintaxes

Classe

Uma classe é um modelo para gerarmos os objetos na memória quando utilizamos nosso sistema. A declaração de uma classe define a sua estrutura básica, e sua sintaxe deve ser:

```
public class NomeDaClasse {
    //...
}
```

Mais à frente explicaremos o significado da palavra reservada **public** para a linguagem Java. A palavra reservada **class** indica que o elemento será uma classe. O nome da classe deve seguir as regras de convenção.

Objeto e sua instanciação

Os objetos são as classes em memória (na memória RAM), gerados quando o sistema está sendo utilizado pelo usuário. Para que um objeto seja criado em memória, utilizamos a seguinte sintaxe:

```
NomeDaClasse nomeDoObjeto = new NomeDaClasse();
```

Gerar um objeto em memória é **instanciar uma classe**. Neste caso, o nome do objeto é a varável que identificará aquele objeto representando a classe em memória.

A partir de uma mesma classe, podemos instanciar diversos objetos em memória.

Atributo

Um atributo é uma característica que define uma classe. Todo atributo deve ser declarado dentro da classe (entre as "chaves" da classe) seguindo a sintaxe a seguir:

```
modificadores tipoDoAtributo nomeDoAtributo;
```

Ou, ainda:

```
modificadores tipoDoAtributo nomeDoAtributo = valorInicial;
```

A diferença entre essas duas declarações está apenas no fato de se ter ou não um valor inicial para o atributo.

Exemplo:

```
public int quantidadeProduto;
protected String nomeEmpresa;
// ... ou ainda ...
public int quantidadeEstoque = 0;
private String nomeSistema = "Sistema de Compras";
```

Observação

Os modificadores deste exemplo (public, protected ou private) serão explicados mais à frente.

Método

Um método é uma ação que pode ser acionada após se instanciar a classe. Em outros paradigmas, um método pode ser comparado à uma função ou uma sub. A sintaxe geral de declaração de um método é a seguinte:

```
"...
modificadores tipoDeRetorno nomeDoMetodo(parâmetros) {...}
..."
```

Os parâmetros de um método são os dados (ou valores) que o método deverá receber ao ser acionado e que serão processados pelo método. A sintaxe dos parâmetros deve ser a seguinte:

```
(tipo1 parametro1, tipo2 parametro2, ..., tipoN parametroN)
```

Percebam que um método pode receber N parâmetros (separados por vírgula), e para cada um deles devemos definir seu tipo, mesmo que sejam o mesmo tipo.

Para o método, as chaves **{...}** definem seu bloco e englobarão o código a ser executado quando o método for acionado.

O termo tipoDeRetorno define o tipo do dado que será retornado (devolvido como resposta) pelo método. Em geral, quando um método retorna um valor, ao chamarmos o método, ele deve ser guardado, ou ainda recebido, por uma variável. Quando um método não retornar valor algum, então esse termo deve ser void. Da mesma forma, se um método retornar algum valor, então ele deverá ter definido expressamente o valor de retorno.

O termo modificadores, tanto dos atributos quanto dos métodos, indica qual modificador de acesso e/ou modificador de comportamento o elemento terá na sua declaração (eles serão mais bem explicados mais à frente).

Os modificadores podem ser:

- Modificadores de acesso:
  - public;
  - private;
  - protected;
  - (default).
- Modificadores de comportamento:
  - static (este não é utilizado para classes);
  - final;
  - abstract (este não é utilizado para atributos).

O termo tipoDoAtributo deve sempre ser definido para qualquer variável utilizada em um programa Java. Ele indica o tipo de valores que a variável poderá receber. A linguagem Java é **fortemente tipada**, o que significa que para podermos utilizar uma variável em um código, primeiro precisamos declarar seu tipo, que será imutável ao longo daquele bloco de código.

O tipo String na linguagem Java não é um tipo primitivo, mas sim uma classe. Devemos declarar variáveis do tipo String da seguinte forma:

```
String nomeDaVariavel = "valor texto inicial da variável";
```

Perceba que o tipo String é escrito com a primeira letra maiúscula (pois é uma classe) e que o valor literal de sua variável deve ser sempre colocado entre aspas duplas ("txt").

Vejamos o exemplo a seguir, com a classe SomaValores, cujos métodos proporcionam duas formas de adição de valores para tipos diferentes:

```
public class SomaValores {
    // - atributos da Classe:
    public int resultInt;
    public double resultDouble;
    // - métodos da Classe:
    public int somarInteiros(int a, int b) {
      // Este método recebe dois valores inteiros
      // ...e retorna o resultado inteiro da soma dos dois valores
      // ...além de guardar a soma no atributo "resultInt".
      int result = a + b;
```

```
        resultInt = result;
        return result;
    }
    public double somarReais(double a, double b) {
        // Este método recebe dois valores reais
        // ...e retorna o resultado real da soma dos dois valores
        // ...além de guardar a soma no atributo "resultDouble".
        double result = a + b;
        resultDouble = result;
        return result;
    }
    public void mostrarResultados() {
        // Este método inicialmente monta um texto (uma String)
        String txt = "------------------------\n";
        txt += "Resultado da soma de Inteiros: " + resultInt + "\n";
        txt += "Resultado da soma de Reais: " + resultDouble + "\n";
        // ...e ao final mostra este texto montado na Console.
        System.out.println(txt);
    }
    // Abaixo, o método "main" (que será o 1o a ser acionado)
    // ..e que acionará os outros métodos desta Classe
    public static void main(String[] args) {
        // ...instanciando a Classe SomaValores
        SomaValores sv = new SomaValores();
        //..- somando os valores inteiros: 7 + 15
        //..- trazendo o resultado para a variável "vi"
        int vi = sv.somarInteiros(7, 15);
        //..- e mostrando este resultado na Console
        System.out.println(vi);
        //..- somando os valores reais: 3.6 + 9.2
        //..- trazendo o resultado para a variável "vd"
        double vd = sv.somarReais(3.6, 9.2);
        //..- e mostrando este resultado na Console
        System.out.println(vd);
        //------------------------
        //Abaixo vamos acionar o método "mostraResultados()"
        //...do objeto "sv" (que representa a Classe SomaValores)
        //...e que mostra organizadamente os valores dos atributos
        sv.mostrarResultados();
    }
}
```

Os elementos em verde do código que acabamos de apresentar (antecedidos por //) são **comentários de programa** e servem para explicar as linhas de código. O texto nele descrito não será executado, de forma que não existirá no arquivo compilado (arquivo.class de execução).

Ao acionarmos essa classe a partir de seu método main, a tela da console apresentará o resultado indicado na figura 10.

```
22

12.799999999999999

-----------------------

Resultado da soma de Inteiros: 22

 Resultado da soma de Reais: 12.799999999999999
```

Figura 10

Saiba mais

Saiba mais sobre métodos sem retorno e métodos com retorno de valores no capítulo 6, itens 6.1 e 6.2, de:

FURGERI, S. *Java 7*: ensino didático. 2. ed. São Paulo: Érica, 2012.

Exemplo de aplicação

Um professor de programação criou um texto representando uma classe e distribuiu o arquivo entre os alunos para que eles criassem a classe em uma IDE. Quando os alunos copiaram o texto com o código da classe, e depois foram compilá-lo, a IDE acusou nele um erro de compilação, ou seja, existe nesse programa um erro conceitual que impede que ele possa ser executado. O programa distribuído pelo professor foi:

```
public class ClasseA {
    public String x = "teste";
    public double y;
    public void metodo01(int a, int b) {
      System.out.print(a + " : " + b);
    }
    public int metodo02(String s) {
      x = s;
      int a = 4;
      return a;
    }
    public String metodo03(String z) {
      String s = x + z;
      System.out.print(s);
    }
}
```

Qual foi o erro encontrado pelo compilador que o impediu de executar o programa?

A) No método metodo01(...) não se pode mostrar simultaneamente os valores de a e b, pois ambos são de tipos diferentes.

B) No método metodo02(...) o programador deveria ter declarado a variável x.

C) Na declaração de atributos da classe, faltou atribuir um valor à variável y.

D) No corpo do método metodo03(...) há um comando que soma duas strings, e não se pode somar strings a não ser que tenham valores numéricos.

E) No método metodo03(...) faltou o retorno de um valor.

Resposta correta: alternativa E.

Resolução

A) Alternativa incorreta.

Justificativa: os valores de a e de b, apesar de serem de tipos diferentes, estão sendo utilizados por um método de impressão na console que permite a mistura de vários tipos de informações, e, portanto, esse fato não caracteriza um erro.

B) Alternativa incorreta.

Justificativa: se observarmos bem, a variável x utilizada no método metodo02(...) representa na verdade o atributo x que já foi declarado no início da classe; portanto, esse comando está correto.

C) Alternativa incorreta.

Justificativa: vimos que os atributos podem ser declarados com ou sem um valor inicial, podendo ser dado a ele um valor ao longo da utilização do sistema, seja por um método da própria classe ou por métodos de outras classes. Sua declaração pode ter ou não uma atribuição, de forma que não caracteriza um erro.

D) Alternativa incorreta.

Justificativa: no corpo do método método03(...) a variável z recebe um valor que é resultado de uma concatenação de strings, ou seja, esse símbolo de adição significa que o valor de uma variável do tipo string está sendo concatenada (unida) ao valor da outra variável do tipo string e, portanto, não caracteriza um erro.

E) Alternativa correta.

Justificativa: se observarmos a declaração do método metodo03(...), ele possui um tipo de retorno que deve ser um valor do tipo string, e em seu corpo está faltando a declaração de retorno de valor, o qual é representado pela palavra reservada return, o que no caso deveria retornar um valor do tipo string.

2 VARIÁVEIS, ESTRUTURAS DE CONTROLE E ORGANIZAÇÃO DO PROJETO

Estruturas de controle são as estruturas vinculadas à lógica de programação. Neste item, vamos ver como devemos trabalhar com variáveis em Java (lembrando que a linguagem Java é uma linguagem fortemente tipada) e como devem ser descritas as estruturas condicionais e as estruturas de repetição.

2.1 Tipos de variáveis

Variáveis são espaços em memória reservados para que possamos guardar valores, a fim de podermos processá-los. Vimos que toda variável possui um nome (determinando a sua identificação), um tipo (que depende do valor a se trabalhar) e um valor.

Em Java, toda variável deve ser declarada antes de ser efetivamente utilizada, para que o compilador saiba como e o quanto ele deve reservar de memória a fim de poder trabalhar com a informação que será processada.

Como a linguagem Java é uma linguagem totalmente voltada ao paradigma de orientação a objetos, as variáveis podem ser utilizadas como atributos (mantendo um comportamento de variável global na classe), ou como variáveis locais, existindo apenas no bloco onde foi definida.

A declaração de um atributo já foi explicada anteriormente neste material. Já as variáveis locais devem ser declaradas com a seguinte sintaxe:

```
tipoDaVariavel nomeVariavel;
```

Ou, ainda:

```
tipoDaVariavel nomeVariavel = valor;
```

Observação

Essa segunda forma de declaração faz com que tenhamos um valor inicial para a variável.

Perceba que as variáveis locais não possuem o termo de modificadores na sua declaração.

O tipo da variável definirá o tipo da informação que ela conterá, assim como o tamanho do espaço em memória a ser reservado para isso.

A linguagem Java trabalha com diversos tipos de dados. São eles:

- Tipos numéricos inteiros:
  - **byte**: números de -128 a 127;
  - **short**: números de -32.768 a 32.767;
  - **int**: números de -2.147.483.648 a 2.147.483.647;
  - **long**: números de -9.223.372.036.854.775.808 a 9.223.372.036.854.775.807.
- Tipos numéricos reais (de ponto flutuante):
  - **float**: ótima precisão com números de até 7 casas decimais (0.1234567...);
  - **double**: ótima precisão com números de até 15 casas decimais (0.123456789012345...).

Observação

Um valor literal de número real deve ser colocado na forma inglesa, ou seja, usando ponto para separar o valor inteiro do valor decimal.

Exemplo:

```
double variavel1 = 3.25;
double variavel2 = 1.47e2; // representando 1,47 * 10^2
```

- Tipo lógico:
  - **boolean**: para guardar um valor lógico (true ou false).
- Tipo alfanumérico:
  - **char**: para guardar um valor de caractere. Por exemplo: 't', ou 'x', ou '%'...

Percebam que o valor literal de uma variável do tipo char deve ser sempre colocado entre aspas simples, como em 'c'.

Observação

Uma vez declarada uma variável, ela não poderá sofrer alteração de seu tipo ao longo do bloco em que se encontra.

2.2 Strings

Na linguagem Java, as strings são classes (não fazem parte do grupo dos tipos primitivos) e possuem uma forma de declaração específica que permite que uma variável seja declarada de forma equivalente a uma variável primitiva.

Declaração na forma de classe:

```
String str1 = new String();
```

Declaração na forma equivalente ao de uma variável primitiva:

```
String str1 = "Esta é uma String...";
```

Percebam que, na declaração em que damos um valor literal à variável do tipo string, o valor deve ser descrito sempre entre aspas duplas.

Para que possamos concatenar (unir) duas strings, utilizamos o símbolo de adição + (mais).

Exemplo:

```
String str01 = "abc";
String str02 = "def";
String str03 = str01 + str02;
// Uma String em Java aceita concatenação recursiva
str03 += "ghi";
System.out.println(str03); // imprime: abcdefghi
```

Veremos adiante alguns métodos utilizados no tratamento de strings. Por enquanto devemos entender que uma string é um conjunto de caracteres caracterizando uma palavra ou uma frase.

Observação

Diferentemente de outras linguagens (como Python ou C#), na linguagem Java uma string não é um array (uma matriz) de caracteres.

Exemplo de aplicação

Uma empresa recebe, de tempos em tempos, um consultor de informática que pertence a uma empresa terceirizada. Como esse consultor normalmente tem que acessar determinados programas do servidor, a empresa decidiu fazer uma implementação de controle em um sistema já em funcionamento, para que seja inserida a figura de um usuário extra com acessos limitados. Como o sistema da empresa foi feito na linguagem Java, o consultor pretende criar uma classe chamada UsuarioExtra contendo atributos para as seguintes características:

- login do usuário;
- senha do usuário;
- valor cobrado por hora;
- quantidade de horas contratada (horas exatas, sem levar em consideração os minutos, segundos e demais submedidas);
- área cliente (cada área da empresa é considerada um cliente de TI e é representada por uma letra apenas);
- usuário administrador (valor lógico que indica se esse usuário deve ter ou não acesso a áreas restritas do servidor).

Qual dos itens a seguir mostra, **respectivamente**, a tipagem mais adequada para cada um dos atributos descritos anteriormente?

A) string, boolean, int, int, string, string.

B) char, char, int, double, string, boolean.

C) string, string, double, int, char, boolean.

D) string, string, double, double, double, char.

E) string, char, double, int, string, int.

Resposta correta: alternativa C.

Resolução

De acordo com o enunciado, a classe contém atributos que devem caracterizar a classe, e cada um desses atributos deve ser declarado com um tipo, que definirá a informação que nele pode ser guardada.

Segue então uma análise dos atributos um a um:

- O login do usuário é a sua identificação, geralmente representada por uma palavra que identifica o usuário e que, portanto, é uma informação alfanumérica, e deve ser representada por uma variável do tipo string.

- A senha do usuário pode ser composta por letras, números e símbolos; portanto, a variável que a representa deve ser do tipo string.

- O valor cobrado por hora costuma ser um valor monetário; portanto, deve ser representado por uma variável do tipo double (valor numérico real).

- A quantidade de horas contratada, por ser uma "hora exata" (de acordo com o enunciado), não deve levar em consideração as subdivisões de horários; portanto, deve ser representada por uma variável do tipo numérico inteiro.

- A área cliente (por exemplo, as áreas internas de uma empresa, como RH, financeiro, jurídico etc.), segundo o enunciado, devem ser representadas por uma única letra. Neste caso, o sistema aceitaria uma representação por uma variável do tipo char ou string.

- A informação usuário administrador, segundo o enunciado, possui um valor lógico true (verdadeiro) ou false (falso), indicando se o usuário pode ou não acessar áreas restritas do servidor; portanto, deve ser representada por uma variável do tipo boolean.

Assim, teremos:

A) Alternativa incorreta.

Justificativa: a senha do usuário não pode ser representada por uma variável do tipo boolean. Além disso, não é adequado representar um valor monetário por um tipo inteiro nem uma variável lógica como string.

B) Alternativa incorreta.

Justificativa: nem o login, nem a senha do usuário podem ser representados por uma variável do tipo char, já que este tipo aceita apenas um caractere de informação. Além disso, não é adequado representar um valor monetário por um tipo inteiro.

C) Alternativa correta.

Justificativa: essa sequência de tipagens está de acordo com a explicação dada anteriormente. Quanto à última variável (que representa a área cliente), poderia até ter um valor do tipo string, mas o mais adequado, já que ela só receberá uma letra (segundo o enunciado), seria uma variável do tipo char.

D) Alternativa incorreta.

Justificativa: a quantidade de horas contratadas está especificando horas exatas sem os minutos e segundos; portanto, deve ser um valor inteiro. Não há como representar uma área da empresa com um valor do tipo double, e a última variável, por ter um valor lógico, deve ser representada por uma variável do tipo boolean.

E) Alternativa incorreta.

Justificativa: não é possível representar uma senha por uma variável do tipo char, e os dois últimos tipos não são adequados para as variáveis que representam.

2.3 Estruturas condicionais

As estruturas condicionais são consideradas estruturas de decisão.

No Java existem basicamente duas estruturas condicionais:

- a estrutura if – else if – else;
- a estrutura switch – case.

2.3.1 if – else if – else

Na linguagem Java, essa estrutura segue a seguinte sintaxe:

Na sua forma simples

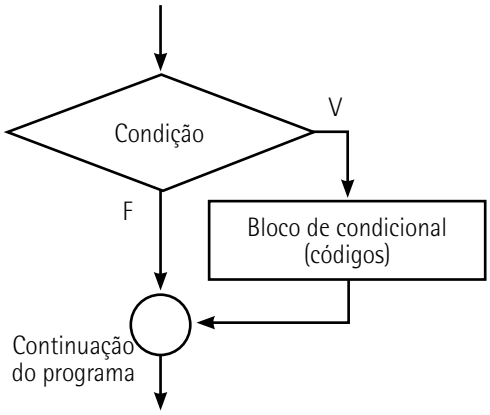


Figura 11 – Fluxograma da estrutura condicional simples

```
if (condicao) {
    <bloco da condicional>
}
```

A condição que define essa estrutura é uma expressão de valor lógico (uma comparação que pode ser verdadeira ou falsa), e o bloco da condicional representa o conjunto de código que somente será executado se a condição (da declaração da estrutura if) for verdadeira.

Percebam que o <bloco da condicional> está entre as chaves {...} que definem a estrutura, de forma que todo código que estiver dentro dessas chaves fará parte da estrutura.

Exemplo:

```
"...
if (nota < 6) {
    System.out.println("Nota considerada baixa");
}
..."
```

Neste exemplo, a frase **Nota considerada baixa** somente será impressa na tela da console se a variável **nota** contiver um valor menor que 6. Do contrário, nada será impresso.

Na sua forma composta

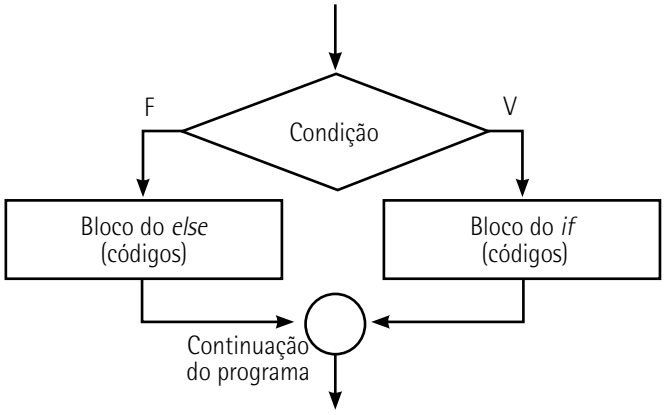


Figura 12 – Fluxograma da estrutura condicional composta

```
if (condicao) {
    <bloco do if>
} else {
    <bloco do else>
}
```

O bloco do if representa o conjunto de código que somente será executado se a condição (da declaração da estrutura if) for verdadeira e o bloco do else representar o conjunto de código que somente será executado se a condição (da declaração da estrutura if) for falsa.

Nesta estrutura, apenas um dos blocos será executado: ou o bloco do if, ou o bloco do else. Assim, após a execução de qualquer um dos blocos, o sistema irá executar os códigos que estiverem além do final da estrutura condicional (fora dela).

Exemplo:

```
"...
if (nota < 6) {
    System.out.println("Nota considerada baixa.");
} else {
    System.out.println("Boa nota.");
}
..."
```

Neste exemplo, a frase Nota considerada baixa. somente será impressa na tela da console se a variável nota contiver um valor menor que 6. De modo contrário (neste exemplo), o sistema imprimirá a frase Nota boa.

Na sua forma encadeada

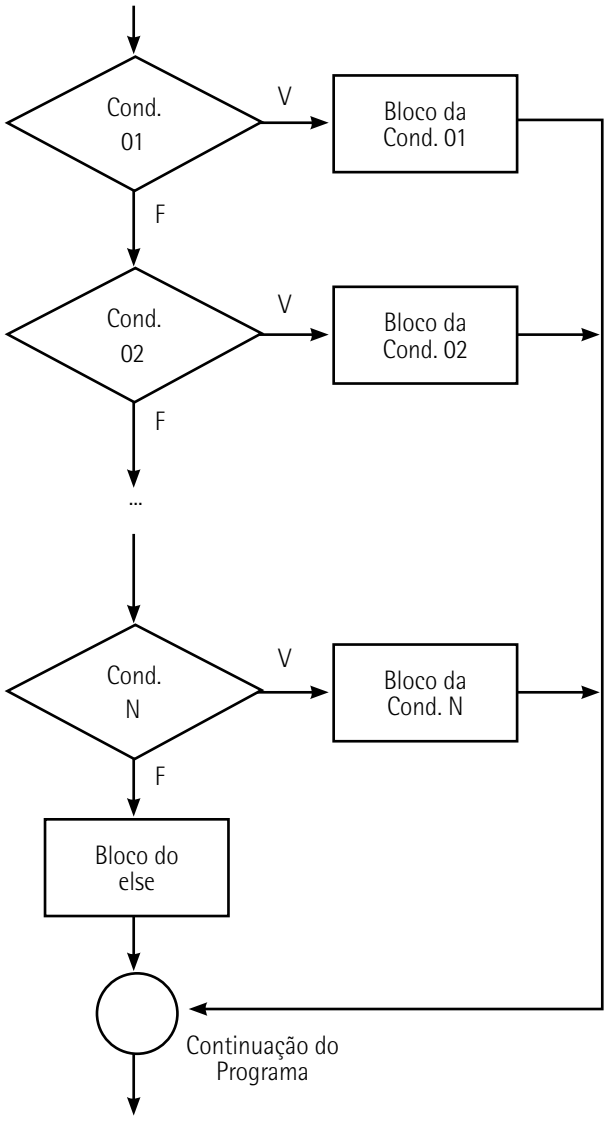


Figura 13 – Fluxograma da estrutura condicional encadeada

```
if (condicao_01) {
    <bloco da condição 01>
} else if (condicao_02) {
    <bloco da condição 02>
...
} else if (condicao_N) {
    <bloco da condição N>
} else {
    <bloco do else>
}
```

O bloco da condição 01 somente será executado se a condição_01 for verdadeira, o bloco da condição 02 somente será executado se a condição_01 for falsa e a condição_02 for verdadeira, e assim por diante, sendo que o bloco do else somente será executado se todas as condições anteriores da mesma estrutura forem falsas.

Nesta estrutura, apenas um dos blocos será executado, de forma que após a sua execução o sistema sairá da estrutura condicional, continuando a executar os códigos que estiverem além do seu final (fora dela).

Exemplo:

```
"...
if (nota < 3) {
    System.out.println("Nota de reprovação.");
} else if (nota < 5) {
    System.out.println("Nota baixa. Deve melhorar.");
} else if (nota < 7) {
    System.out.println("Tente uma nota mais alta.");
} else {
    System.out.println("Excelente nota.");
}
..."
```

Neste exemplo, apenas uma das frases será impressa na tela da console. Percebam a sequência dos valores de comparação, que seguem uma ordem crescente, possibilitando que o programa possa dar respostas adequadas, dependendo do valor da variável nota.

2.3.2 switch – case

A estrutura switch é uma estrutura condicional que, a partir do valor de uma variável, executa um dos seus blocos case definidos, aquele no qual se apresenta um valor exatamente igual ao da variável de controle.

Essa estrutura equivale a uma estrutura if – else if – else, em que a comparação feita nas suas condições seja a de igualdade.

A sintaxe desta estrutura é a seguinte:

```
switch (variavel) {
    case valor_01:
      <bloco do valor 01>
      break;
    case valor_02:
      <bloco do valor 02>
      break;
    ...
    case valor_N:
      <bloco do valor N>
      break;
    default:
      <bloco do default>
      break;
}
```

O bloco do valor 01 somente será executado se o valor da variável, definida na declaração (switch) da estrutura, for exatamente igual ao valor 01 definido na declaração do seu case. O bloco do valor 02 somente será executado se o valor da variável, definida na declaração (switch) da estrutura, for exatamente igual ao valor 02 definido na declaração do seu case, e assim por diante. O bloco default somente será executado se o valor da variável definida na declaração da estrutura for diferente a qualquer um dos valores declarados nos cases daquela estrutura.

A variável de comparação declarada no termo switch desta estrutura pode ter apenas um dos seguintes tipos: byte, short, char, int, string, além de algumas wrapper classes (algumas classes que representam tipos primitivos, como as classes character, byte, short e integer).

Um bom exemplo de utilização de uma estrutura switch – case é a geração de menus de opção, em que o usuário poderia selecionar, dentre uma lista de possíveis opções (na forma de números inteiros), a opção de ação que ele pretende realizar.

Além deste exemplo, pode-se utilizar a estrutura switch – case em processos automáticos que envolvem a leitura de arquivos externos (geralmente arquivos textos), em que há alguns identificadores (caracteres ou conjunto de caracteres) nesse texto que remetem o sistema a determinadas ações específicas dependendo do caractere lido naquela sequência.

Observação

Uma estrutura case pode ser, em qualquer ocasião, substituída por uma estrutura if.

No entanto, nem toda estrutura if pode ser substituída por uma estrutura case, como aquelas em que a lógica de comparação não é a lógica de igualdade.

Exemplo de aplicação da estrutura switch – case:

```
import javax.swing.JOptionPane;
public class TesteSwitch {
    public static void main(String[] args) {
      int x = Integer.parseInt(
        JOptionPane.showInputDialog("Digite o valor de x:"));
      switch(x) {
      case 1:
        System.out.println("O valor de x é igual a 1");
        break;
      case 2:
        System.out.println("O valor de x é igual a 2");
        break;
      case 3:
        System.out.println("O valor de x é igual a 3");
        break;
      default:
        System.out.println("O valor de x é maior que 3");
        System.out.println("...ou menor que 1");
        break;
      }
    }
}
```

2.4 Estruturas de repetição

As estruturas de repetição são estruturas que repetem a execução de determinado bloco (conjunto) de códigos, de acordo com uma regra pré-definida, dependendo da estrutura utilizada.

A cada uma das execuções do bloco definido pela estrutura de repetição, chamamos de **iteração**.

Lembrete

Atenção aos conceitos: **iteração** é uma única execução completa do bloco definido pela estrutura de repetição; **interação** é a ação mútua ou compartilhada entre duas ou mais entidades (por exemplo, entre duas pessoas, ou entre uma pessoa e o computador etc.).

2.4.1 for

A estrutura for, em Java, é uma estrutura de repetição utilizada quando se sabe exatamente a quantidade de repetições a serem executadas.

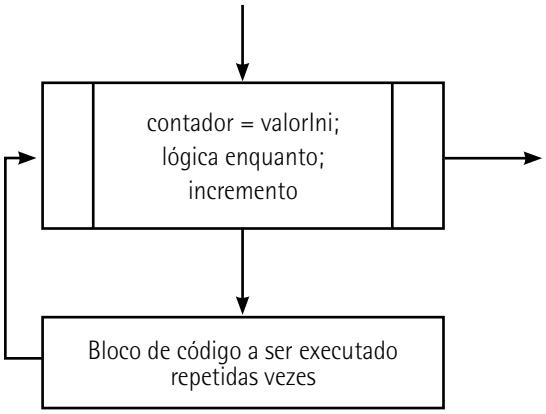


Figura 14 – Fluxograma da estrutura for

Essa estrutura é controlada por uma variável de valor inteiro, a qual chamamos de variável contadora (ou simplesmente contador). Essa variável terá um valor inicial, de forma que a lógica de repetição (enquanto) define o valor final da variável contadora, além do incremento que acontecerá ao final de cada iteração.

A sintaxe dessa estrutura é a seguinte:

```
for (<início>; <condição>; <variação>) {
       <bloco de código a ser executado repetidas vezas>
}
```

Onde:

<início> = atribuição de um valor inicial à variável contadora.

<condição> = condição (valor lógico) da repetição em relação à variável contadora.

<variação> = variação do valor da variável contadora.

Exemplo:

```
for (int x = 1; x <= 10; x++) {
     System.out.println(x);
}
// o exemplo acima imprime na tela da console
// ...os números inteiros de 1 a 10
```

Arrays

A estrutura for é também muito útil quando precisamos trabalhar com matrizes (arrays) de dados.

Em Java, um array é uma variável que pode conter mais de um valor, desde que todos os valores daquele mesmo array tenham o mesmo tipo de informação.

Para gerarmos um array de informações, utilizamos a seguinte sintaxe:

```
tipoArray[] var = new tipoArray[tamanho];
```

A variável var será um array de dados que terão (todos) o tipo definido em tipoArray, e que terá a quantidade de valores definida em tamanho.

Os valores de um array são organizados em posições definidas por índices (números inteiros sequenciais que começam sempre no valor 0). Assim, para ler ou alterar um valor de uma determinada posição do array, basta indicar o índice daquele valor no array com colchetes, na seguinte sintaxe: array[índice].

Exemplo:

```
int[] a = new int[3];
a[0] = 4;
a[1] = -15;
a[2] = 72;
System.out.println(a[1]);
// imprimirá o valor -15
```

No exemplo, a variável a será um array de dados do tipo int com três valores, sendo que na primeira posição (no índice 0) estará o valor 4, na segunda posição (no índice 1) estará o valor -15 e na terceira posição (no índice 2) estará o valor 72.

Observação

Em informática, os índices que indicam a posição do valor em um conjunto de valores são sempre números inteiros, começam sempre no valor 0 e são sempre sequenciais.

É possível se gerar um array de dados, já diretamente (ou explicitamente) com seus valores, utilizando a seguinte sintaxe:

```
double[] b = {1.2, 7.3, -0.8, 4};
```

No exemplo, a variável b foi gerada diretamente como um array de dados do tipo double, com quatro valores numéricos, ou seja, contendo o tamanho 4. Assim, para que possamos acessar cada um dos valores do array, podemos nos utilizar da estrutura for, já que ela é controlada por uma variável contadora, que pode representar cada um dos índices, indicando a posição de cada um dos valores de um array.

Vejamos o exemplo a seguir, que acessa os valores de dois arrays utilizando a estrutura for de repetição:

```
int[] a = new int[3];
a[0] = 4;
a[1] = -15;
a[2] = 72;
double[] b = {1.2, 7.3, -0.8, 4};
// Acessando cada um dos arrays acima (a e b)
// utilizando a estrutura for
for(int x = 0; x < a.length; x++) {
    System.out.println(a[x]);
}
for(int x = 0; x < b.length; x++) {
    System.out.println(b[x]);
}
```

Lembrete

Um array é indexado por números inteiros que sempre iniciarão no valor 0 e sempre serão sequenciais, de modo que nunca existirão lacunas na sua sequência numérica.

2.4.2 while

A estrutura while é uma estrutura de repetição controlada por uma condição, de forma que o bloco definido pela estrutura será executado repetidas vezes enquanto a condição verificada na declaração for verdadeira.

A sintaxe dessa estrutura é a seguinte:

```
while (<condição de repetição>) {
    <bloco de código a ser executado repetidas vezes>
}
```

Exemplo:

```
int x = 1;
while (x <= 10) {
    System.out.println(x);
    x++;
}
// assim como o exemplo dado para a estrutura "for",
// o exemplo acima também imprimirá na tela da console
// ...os números inteiros de 1 a 10
```

Percebe-se no exemplo que o valor da variável de controle (x) é acrescido de uma unidade ao final de cada iteração. No entanto, o comando que definiu esse acréscimo (`x++;`) teve que ser adicionado dentro do bloco da estrutura.

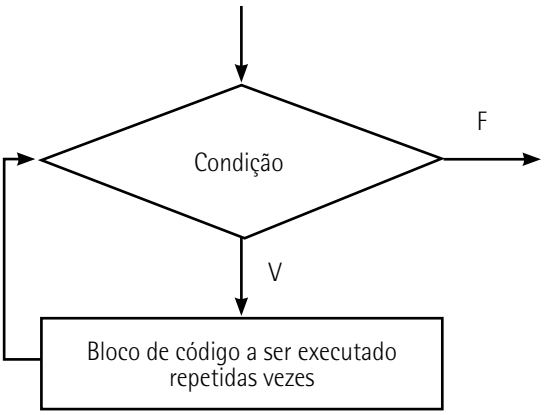


Figura 15 – Fluxograma da estrutura while

Nessa estrutura, devemos sempre tomar cuidado para que a condição definida na sua declaração se torne falsa em algum momento, a fim de que cessem as iterações, saindo da execução repetida do bloco da estrutura. Se isso não ocorrer (se a condição nunca se tornar falsa), ocorre o que chamamos de looping infinito (ou laço infinito), caso em que o sistema nunca finalizará a execução das repetições (as iterações).

2.4.3 do – while

A estrutura do – while é uma estrutura de repetição controlada por uma condição posterior à execução dos códigos de dentro de seu bloco, de forma que, assim como a estrutura while, o bloco definido pela estrutura será executado repetidas vezes enquanto a condição verificada na declaração for verdadeira.

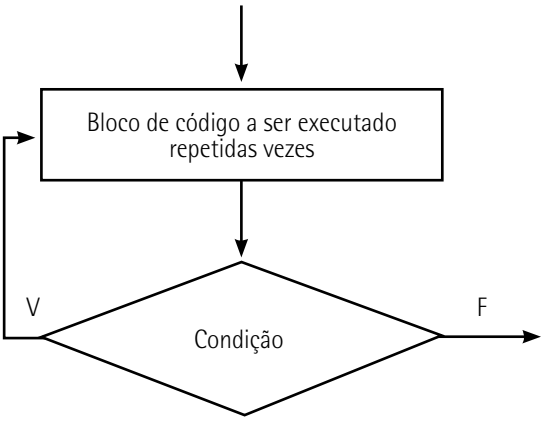


Figura 16 – Fluxograma da estrutura do – while

A diferença dessa estrutura em comparação com a estrutura while é que neste caso a condição é verificada no final da estrutura (ao final da iteração). Efetivamente, a única diferença funcional entre elas pode ser percebida sobre a situação em que a condição seja inicialmente falsa. Neste caso, com a estrutura while, o bloco de códigos interno à estrutura não será executado, mas com a estrutura do – while, esse mesmo bloco interno à estrutura, nessas mesmas condições, será executado ao menos uma vez.

A sintaxe dessa estrutura é a seguinte:

```
do {
    <bloco de código a ser executado repetidas vezas>
} while (<condição de repetição>);
```

Observe que a condição está sendo verificada ao final do bloco e que sua linha de definição termina com ponto-e-vírgula (;).

Também nessa estrutura, devemos sempre tomar cuidado para que a condição definida ao final de sua declaração se torne falsa em algum momento, a fim de que cessem as iterações, saindo da execução repetida do bloco da estrutura, sem que ocorra o looping infinito.

Exemplo:

```
int x = 1;
do {
    System.out.println(x);
    x++;
} while (x <= 10);
// assim como no exemplo dado para a estrutura "while",
// o exemplo acima imprime na tela da console
// ...os números inteiros de 1 a 10
```

A seguir, um exemplo das estruturas de repetição while e do – while, com resultados diferentes:

```
int x = 400;
while (x <= 10) {
    System.out.println(x);
    x++;
}
// neste caso nada será impresso na console,
// já que a condição inicial é falsa
do {
    System.out.println(x);
    x++;
} while (x <= 10);
// neste caso será impresso pelo menos o valor 400,
// de forma que como a condição é falsa, não haverá
// mais iterações, saindo da estrutura de repetição.
```

2.5 Packages (pacotes)

Num projeto Java, criamos packages (ou pacotes) para separarmos as classes em categorias (o equivalente a pastas, quando queremos organizar nossos arquivos no computador), e com isso podemos organizar as classes de nosso projeto.

Fisicamente (por exemplo, se procurarmos no Explorer do Windows), um pacote é identificado como um ou mais diretórios que contêm internamente uma ou mais classes Java.

Por exemplo:

```
package model;
```

Esse é o diretório model que fica na raiz do projeto Java.

Em outro exemplo:

```
package com.mysql.jdbc;
```

Esse é o diretório com/mysql/jdbc (três diretórios, um dentro do outro) que fica na raiz do projeto Java.

Quando uma classe está localizada em um pacote, internamente, na classe, essa situação precisa estar definida (em código), identificando o pacote em que ela se localiza, por meio da palavra reservada package.

```
package nome.do.pacote;
//esta Classe pertence ao pacote "nome.do.pacote"
```

Quando vamos utilizar (ou referenciar/importar) ao longo da classe uma outra classe que está em outro pacote ou em outra biblioteca de classes, somos obrigados a importar aquela classe, utilizando a palavra reservada import, e indicando o caminho até ela:

```
import nome.do.pacote.Classe;
```

Observação

Uma biblioteca de classes é um conjunto de classes que ficam agrupadas geralmente em um arquivo com extensão .jar, o qual foi criado com o intuito de conter diversas classes que podem ser utilizadas por vários projetos ao mesmo tempo. Nesse arquivo, todas as classes estão dispostas em packages diversos. Esse é um arquivo compactado específico que pode ser preparado pelo Eclipse, exportando-se um projeto como arquivo jar (não executável).

Saiba mais

O link a seguir contém um texto que faz parte de um tutorial oficial do Java, do site da Oracle, e que explica como e para que geramos arquivos .jar.

THE JAVA™ tutorials: using JAR files: the basics. *Oracle*, [s.d.]c. Disponível em: https://cutt.ly/YTfIwV2. Acesso em: 3 dez. 2025.

Caso se queira importar **todas** as classes do pacote, no lugar do nome da classe, colocamos o caractere * (asterisco).

```
import nome.do.pacote.*;
```

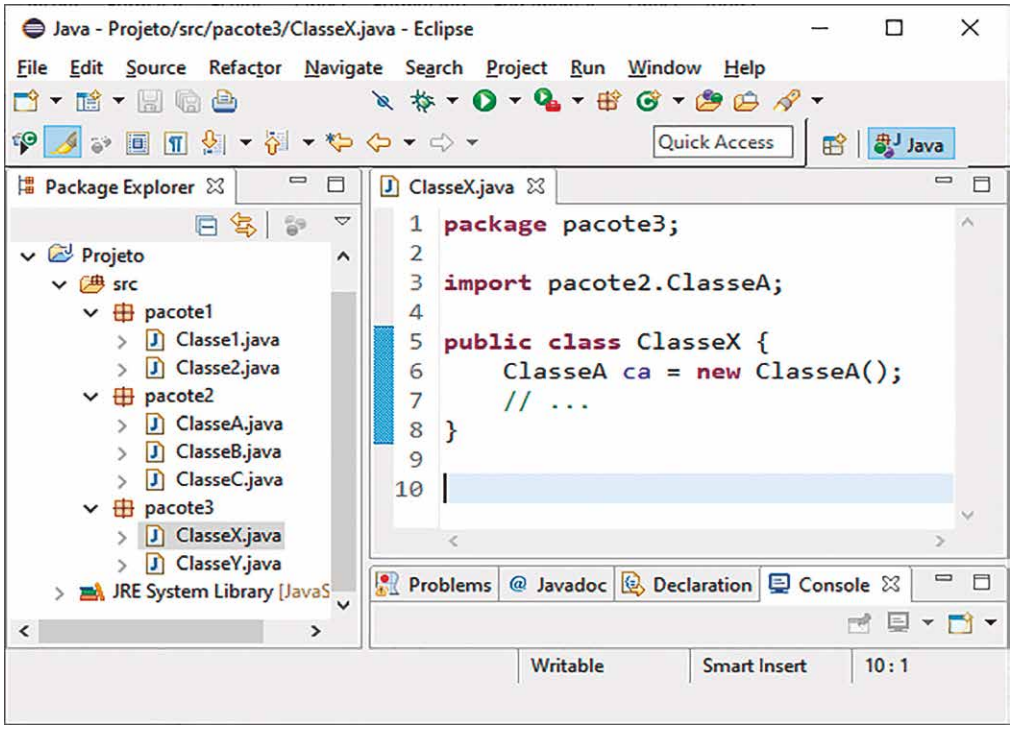


Figura 17 – Projeto organizado em packages

Na imagem, podemos observar um projeto Java organizado em três pacotes, sendo que o pacote pacote1 contém duas classes, o pacote pacote2 contém três classes e o pacote pacote3 contém duas classes. Percebe-se que, na área central, vê-se o código da classe ClasseX, que pertence ao pacote pacote3 e por isso contém definida a seguinte linha de código:

```
package pacote3;
```

Também se observa que um dos atributos da ClasseX é um objeto da ClasseA que está em outro pacote, e por isso precisou ser importado com o seguinte comando:

```
import pacote2.ClasseA;
```

Para finalizar esta unidade, observe os exemplos de aplicação a seguir. As resoluções são apresentadas na sequência.

Exemplos de aplicação

1) Criar uma classe (ClasseA) com o método main que deverá ter duas variáveis do tipo inteiro (v1 e v2) contendo quaisquer valores numéricos. Criar uma terceira variável do tipo inteiro (v3) que deverá receber o cálculo da soma das duas variáveis iniciais, e no final o programa deverá mostrar na tela da console o valor dessa soma com o texto:

"Soma = <valor>"

2) Criar uma classe (ClasseB) com dois atributos (a1 e a2) do tipo double e o método main. Criar um método chamado mostrarValores que exibe na tela da console em linhas diferentes os valores dos atributos com o texto ("Valor do atributo a1: <valor>" e na linha seguinte o texto equivalente ao outro aributo). No método main deve-se inserir valores nos atributos, sendo que para isso deve-se antes instanciar a classe; a partir dela, inserir os valores nos atributos; e, ao final, mostrar os valores dos atributos chamando o método mostrarValores.

3) Criar uma classe com o nome Calculadora (sem o método main). Essa classe deverá possuir três atributos públicos do tipo double (n1, n2 e res) e quatro métodos públicos sem retorno (somar, subtrair, multiplicar e dividir), de modo que cada método deverá realizar a respectiva conta com os dois atributos (n1 e n2) e, sem retornar valor algum, inserir o valor do resultado no atributo res.

Criar também a classe TesteCalc com o método main, que deverá realizar (a partir da classe Calculadora) e mostrar o resultado das seguintes contas:

a) 5 + 3,7

b) 18,4 – 23,2

c) 7 * 1,5 + 4,2

d) 13,8 / 4,2 + 1,8

4) Criar uma classe (de nome Verificadora), sem o método main, que possui dois métodos que recebem um valor numérico inteiro como parâmetro, de modo que o método verificaParImpar**( )** deverá retornar a string "PAR", "IMPAR" ou "NULO" (este último, para o caso de receber o valor zero) de acordo com o valor do módulo (valor absoluto ou valor equivalente positivo) do número recebido

por parâmetro, e o método verificaNegPos() deverá retornar a string "POSITIVO", "NEGATIVO" ou "ZERO", dependendo do valor recebido por parâmetro.

Essa classe Verificadora deverá possuir também um método de nome verificarValor(), que recebe um número como parâmetro e imprime o resultado segundo os dois outros métodos da classe, de modo a mostrar na tela da console um texto equivalente ao seguinte (imaginando que este último método tenha recebido o valor -2):

"O número -2 é NEGATIVO e tem módulo PAR."

Criar uma classe TesteVerificadora que deverá instanciar a classe Verificadora e em seguida verificar (a partir do método verificarValor()) os seguintes números: 101, 24, -17 e -8.

5) Criar uma classe (ClasseC) com o método intervaloComFor(...) e o método intervaloComWhile(...) de forma que ambos recebam como parâmetro dois números inteiros, um representando o início de um intervalo e o outro representando o final do intervalo. Eles deverão mostrar na tela da console os números inteiros do intervalo (inclusive eles mesmos) de acordo com as especificações a seguir (observação: o primeiro método deverá utilizar laço de repetição for e o outro, laço de repetição while):

- Criar um terceiro método com o nome chamarMetodos(...) que recebe dois números inteiros como parâmetro e simplesmente aciona os dois métodos anteriores na sequência com os mesmos números.

- O programa deverá dar como resultado na tela da console um texto que deverá estar no seguinte formato de amostragem (imaginando que o método chamarMetodos(...) tenha recebido os valores 3 e 9):

"...
Resultado com for:
3, 4, 5, 6, 7, 8, 9
Resultado com while:
3, 4,
5, 6,
7, 8,
9
..."

Criar uma classe TesteC que instancia a ClasseC deste enunciado e roda o método chamarMetodos(...) com os valores:

a) 3 e 9 (o mesmo do exemplo mostrado no enunciado).

b) -7 e 10.

Resolução

1)

```
public class ClasseA {
    public static void main(String[] args) {
      int v1 = 7;
      int v2 = 20;
      int v3 = v1 + v2;
      System.out.println("Soma = " + v3);
    }
}
```

2)

```
public class ClasseB {
    public double a1;
    public double a2;
    public static void main(String[] args) {
      ClasseB cb = new ClasseB();
      cb.a1 = 73.42;
      cb.a2 = -105.77;
      cb.mostrarValores();
    }
    public void mostrarValores() {
      System.out.println("Valor do atributo a1: " + a1);
      System.out.println("Valor do atributo a2: " + a2);
    }
}
```

3)

```
public class Calculadora {
    public double n1;
    public double n2;
    public double res;
    public void somar() {
      this.res = n1 + n2;
    }
    public void subtrair() {
      this.res = n1 - n2;
    }
    public void multiplicar() {
      this.res = n1 * n2;
    }
    public void dividir() {
      this.res = n1 / n2;
    }
}
public class TesteCalc {
    public static void main(String[] args) {
      // Instanciando a Classe Calculadora
      Calculadora calc = new Calculadora();
      // Realizando a conta: 5 + 3,7
```

```
        // ..lembrando que os números decimais devem ser
        // ..descritos com ponto ao invés de vírgula
        calc.n1 = 5;
        calc.n2 = 3.7;
        calc.somar();
        System.out.println("Resultado: 5 + 3,7 = " + calc.res);
        // Realizando a conta: 18,4 - 23,2
        calc.n1 = 18.4;
        calc.n2 = 23.2;
        calc.subtrair();
        System.out.println("Resultado: 18,4 - 23,2 = "
                                + calc.res);
        // Realizando a conta: 7 * 1,5 + 4,2
        //..obs.: para realizar 2 contas devemos nos utilizar
        //..de um recurso que é inserir o resultado num dos
        //..atributos.
        calc.n1 = 7;
        calc.n2 = 1.5;
        calc.multiplicar();
        calc.n1 = calc.res;
        calc.n2 = 4.2;
        calc.somar();
        System.out.println("Resultado: 7 * 1,5 + 4,2 = "
                                + calc.res);
        // Realizando a conta: 13,8 / 4,2 + 1,8
        calc.n1 = 13.8;
        calc.n2 = 4.2;
        calc.dividir();
        calc.n1 = calc.res;
        calc.n2 = 1.8;
        calc.somar();
        System.out.println("Resultado: 13,8 / 4,2 + 1,8 = "
                                + calc.res);
    }
}
```

4)

```
public class Verificadora {
    public String verificaParImpar(int valor) {
        // tornando o valor recebido em seu valor absoluto
        if (valor < 0) valor *= -1;
        String txt = "NULO";
        if (valor != 0) {
            txt = "PAR";
            int res = valor % 2;
            if (res != 0) txt = "IMPAR";
        }
        return txt;
    }
    public String verificaNegPos(int valor) {
        String txt = "ZERO";
        if (valor != 0) {
            txt = "POSITIVO";
            if (valor < 0) txt = "NEGATIVO";
```

```
        }
        return txt;
    }
    public void verificarValor(int valor) {
        String txt = "O número " + valor;
        txt += " é " + verificaNegPos(valor);
        txt += " e tem módulo " + verificaParImpar(valor) + ".";
        System.out.println(txt);
    }
}
public class TesteVerificadora {
    public static void main(String[ ] args) {
        // Instanciando a Classe Verificadora
        Verificadora ver = new Verificadora();
        // Realizando as verificações solicitadas
        ver.verificarValor(101);
        ver.verificarValor(24);
        ver.verificarValor(-17);
        ver.verificarValor(-8);
    }
}
```

5)

```
public class ClasseC {
    public void intervaloComFor(int ini, int fin) {
        String txt = "";
        for (int x = ini; x <= fin; x++) {
            txt += x;
            if (x != fin) txt += ", ";
        }
        System.out.println(txt);
    }
    public void intervaloComWhile(int ini, int fin) {
        String txt = "";
        int x = ini;
        int cont = 0;
        while (x <= fin) {
            txt += x;
            if (x != fin) txt += ", ";
            if (cont == 1) {
                txt += "\n";
                cont = 0;
            } else {
                cont++;
            }
            x++;
        }
        System.out.println(txt);
    }
    public void chamarMetodos(int ini, int fin) {
        System.out.println("Resultado com for:");
        intervaloComFor(ini, fin);
        System.out.println("Resultado com while:");
        intervaloComWhile(ini, fin);
```

```
    }
}
public class TesteC {
    public static void main(String[] args) {
      ClasseC cc = new ClasseC();
      cc.chamarMetodos(3, 9) {
      cc.chamarMetodos(-7, 10) {
  } }
```

Resumo

Nesta unidade, vimos alguns conceitos básicos de programação e como eles são aplicados na linguagem Java.

Quando programamos nessa linguagem, trabalhamos basicamente apenas com três elementos: as **classes**, que são os modelos que utilizamos para construir os objetos (as instâncias daquela classe) em memória; os **métodos**, que são as ações definidas nas classes e que somente serão acionados se forem explicitamente "chamados" no programa; e os **atributos**, que representam as informações que caracterizam as classes.

Utilizamos como IDE (ambiente integrado de desenvolvimento) o Eclipse, que é um programa gratuito criado pela IBM e que facilita a criação de projetos em Java.

A linguagem Java possui algumas convenções de programação que regulamentam desde a forma com que se programa até os nomes dos elementos que trabalhamos. Vimos que, ao programarmos, devemos colocar todos os atributos no início da classe e que os métodos devem estar depois dos atributos. O nome de uma classe deve iniciar com letra maiúscula, enquanto o nome dos métodos e dos atributos devem iniciar com letra minúscula.

Os métodos devem ser descritos com a seguinte sintaxe:

```
modificadores tipoDeRetorno nomeDoMetodo(parâmetros) {…}
```

A sintaxe dos parâmetros do método (quando houver) deve ser:

```
(tipo1 parametro1, tipo2 parametro2, ..., tipoN parametroN)
```

O modificador é um termo que pode definir como o elemento poderá ser acessado (modificadores de acesso), ou como ele se comportará (modificadores de comportamento) no sistema.

Os atributos devem ser descritos com a seguinte sintaxe:

```
modificadores tipo nomeDoAtributo;
```

Ou, ainda:

```
modificadores tipo nomeDoAtributo = valorInicial;
```

Num programa, os comentários (que podem ser inseridos como comentário de linha utilizando `//`, ou comentário multilinha utilizando `/*...*/`), são colocados no sistema para que este possa ser explicado a todo desenvolvedor ou analista que for dar manutenção ou implementar o sistema. Esses comentários não interferem na execução do código, de forma que são ignorados quando o sistema gera o arquivo compilado do programa.

Quando geramos um programa (uma classe), o fazemos em um arquivo com extensão .java, que é um "arquivo fonte" com o código criado pelo desenvolvedor. Quando vamos executar esse programa, o Eclipse gera automaticamente o arquivo compilado desse programa, que é um arquivo com extensão .class utilizado pela JVM para rodar as aplicações.

A linguagem Java é fortemente tipada, o que significa que para que possamos utilizar variáveis locais ou globais, elas devem ser previamente declaradas no programa. A sintaxe dessa declaração é:

```
tipoDaVariavel nomeVariavel;
```

Ou, ainda:

```
tipoDaVariavel nomeVariavel = valor;
```

Seu tipo define o tipo da informação que nós podemos guardar nela. Os tipos mais utilizados são: int (para valores inteiros), double (para valores reais), string (que é considerada uma classe e é utilizada para guardar frases e palavras) e boolean (para valores lógicos).

As estruturas condicionais são aquelas que definem se um bloco de linhas de código, definido pelo espaço entre a abertura e o fechamento de chaves, será ou não executado, dependendo do resultado de certas condições (comparações). Na linguagem Java existem algumas estruturas condicionais, como a estrutura simples (if), a estrutura composta (if – else), a estrutura encadeada (if – else if – else) e a estrutura case (switch – case), sendo que nesta última a única comparação possível é a de igualdade. Nas estruturas condicionais, apenas um bloco de código é executado: aquele cuja primeira condição, ou comparação, tiver resultado verdadeiro. Suas sintaxes são:

Da estrutura if (geral):

```
if (condicao_01) {
     <bloco da condição 01>
} else if (condicao_02) {
     <bloco da condição 02>
...
} else if (condicao_N) {
```

```
        <bloco da condição N>
} else {
        <bloco do else>
}
```

Da estrutura switch – case:

```
switch (variavel) {
        case valor_01:
                <bloco do valor 01>
                break;
        case valor_02:
                <bloco do valor 02>
                break;
        ...
        case valor_N:
                <bloco do valor N>
                break;
        default:
                <bloco do default>
                break;
}
```

As estruturas de repetição são aquelas que permitem que um determinado conjunto de códigos seja executado repetidas vezes. No Java temos três estruturas de repetição: a estrutura for, em que a repetição é controlada por uma variável de controle (em geral uma variável contadora); a estrutura while, em que a repetição se dá enquanto uma condição for verdadeira, sendo que essa condição é verificada antes de cada iteração; e a estrutura do – while, em que a repetição também se dá enquanto uma condição for verdadeira, sendo que neste caso essa condição é verificada ao final de cada iteração. Suas sintaxes são:

Da estrutura for:

```
for (<início>; <condição>; <variação>) {
                <bloco de código a ser executado repetidas vezas>
}
```

Da estrutura while:

```
while (<condição de repetição>) {
        <bloco de código a ser executado repetidas vezas>
}
```

Da estrutura do – while:

```
do {
        <bloco de código a ser executado repetidas vezas>
} while (<condição de repetição>);
```

Exercícios

Questão 1. (Sugep UFRPE 2018, adaptada) Leia o texto a seguir, a respeito do paradigma de orientação a objetos:

O paradigma de orientação a objetos surge como o advento da reutilização de código e a facilidade na manutenção. Seu princípio é a construção de código implementando as entidades do mundo real, por meio do conceito de classes que possuem relação entre si. Pode-se considerar que a programação orientada a objetos (POO) está embasada nos seguintes pilares.

- **Abstração**: como estamos lidando com objetos do mundo real, por exemplo, carro, casa, pessoa etc., precisamos imaginar como esses objetos vão se integrar dentro do nosso sistema e modelar seu comportamento, abstraindo o comportamento e características específicas de cada um.

- **Encapsulamento**: não importa, para um código que invoca um método, saber como outro vai ser executado. O encapsulamento evita a utilização inapropriada dos atributos de uma classe, de modo a possibilitar o controle de seus valores por meio dos seus métodos. Trata-se de uma característica que traz principalmente segurança ao código.

- **Herança**: assim como no mundo real, a herança em programação orientada a objetos seria a capacidade de uma classe herdar de outra métodos e atributos, sendo, portanto, uma característica relacionada ao reúso de código.

- **Polimorfismo**: existem animais capazes de se adaptar a algumas necessidades do mundo real e se comportar de forma diferenciada, em alguns casos. Essa particularidade também é possível no paradigma de orientação a objetos. Mesmo herdando o comportamento de outra classe, a classe herdeira pode modificar o seu comportamento em determinadas situações.

Adaptado de: SILVA, F. M. da. *Paradigmas de programação*. Porto Alegre: Sagah, 2019. p. 21.

Com base na leitura e nos seus conhecimentos sobre programação orientada a objetos (POO), avalie as afirmativas.

I – POO é um paradigma de programação, sugerindo um padrão de desenvolvimento que independe da linguagem utilizada.

II – O conceito de herança está fortemente atrelado ao reúso de código, o que pode aumentar a produtividade da aplicação.

III – Apesar de Java ter sido projetada para orientação a objetos, os códigos nessa linguagem podem ser escritos utilizando-se programação estruturada.

É correto o que se afirma em:

A) I, apenas.

B) II, apenas.

C) I e III, apenas.

D) II e III, apenas.

E) I, II e III.

Resposta correta: alternativa E.

Análise das afirmativas

I – Afirmativa correta.

Justificativa: POO não é uma linguagem de programação, e sim um paradigma de programação, ou seja, é uma modelagem de escrita de código. Para que uma linguagem de programação seja considerada orientada a objetos, deve possibilitar a implementação dos conceitos de herança, polimorfismo e encapsulamento. Alguns exemplos dessas linguagens são C++, C#, Python e, é claro, Java.

II – Afirmativa correta.

Justificativa: a herança diz respeito à capacidade de uma classe herdar métodos e atributos de outra classe. Está, portanto, atrelada ao reúso de código e ao aumento da produtividade da aplicação.

III – Afirmativa correta.

Justificativa: o paradigma de programação estruturada é aquele que, geralmente, aprendemos no nosso primeiro contato com programação. Linguagens como Cobol, C e Pascal implementam esse paradigma. Ele é baseado em três estruturas básicas:

- **sequência** (os passos são executados linearmente, um após o outro);
- **decisão** (utiliza comandos como if-else e switch, que "decidem" se determinado trecho de código será executado ou não);
- **iteração** (execução repetitiva de determinado trecho de código, que pode ser feita por comandos como for e while).

Por mais que seja considerada uma linguagem orientada a objetos, é possível, sim, desenvolvermos códigos em Java utilizando programação estruturada. Esses códigos, muitas vezes, resultam do "vício" do programador em seguir o paradigma estruturado.

Questão 2. (IFMT 2019) Considere o trecho de código, escrito na linguagem Java, apresentado a seguir.

```
 1 - public class MinhaClasse
 2 - {
 3 -    public static void main(String[] args)
 4 -    {
 5-         int numero1 = 13;
 6 -        int numero2 = 31;
 7 -        if (numero1 = numero2)
 8 -            System.out.print("Os numeros sao iguais");
 9 -        else
10 -          System.out.print("Os numeros sao diferentes");
11 -    }
12 - }
```

É correto afirmar que:

A) A sua execução apresentará a mensagem: Os numeros sao iguais.

B) A sua execução apresentará a mensagem: Os numeros sao diferentes.

C) A sua compilação apresentará uma mensagem de erro na linha 1.

D) A sua compilação apresentará uma mensagem de erro na linha 3.

E) A sua compilação apresentará uma mensagem de erro na linha 7.

Resposta correta: alternativa E.

Análise da questão

É fácil fazermos a análise do objetivo do código, que consiste apenas na comparação entre os valores das variáveis numero1 e numero2, seguida da exibição de uma mensagem. Porém, justamente na comparação, na linha 7, ocorre um deslize que fará com que seja apresentada uma mensagem de erro durante a compilação.

Em Java, o operador = é um operador de atribuição. Observe o exemplo a seguir.

```
int x = 23;
```

Nessa linha, foi declarada a variável x, do tipo inteiro, à qual foi atribuído o valor 23. Para compararmos valores entre si, devemos utilizar o operador relacional ==, que verifica se dois operandos são iguais entre si. Portanto, para que o código seja corrigido, a linha 7 deve ser modificada para

```
if (numero1 == numero2)
```

É importante notar que, em Java, blocos de comandos que têm apenas uma instrução não precisam, obrigatoriamente, estar envolvidos por chaves. Esse é o caso do bloco do if e do bloco do else da questão. Logo, a ausência de chaves não geraria uma mensagem de erro durante a compilação. No entanto, é considerada uma boa prática sempre envolver blocos de declaração por um par de chaves, de forma a evitar problemas em futuras edições do código.

Unidade II

3 LIDANDO COM AS VARIÁVEIS E SEUS ACESSOS

3.1 Promotion e casting

Nesta unidade, inicialmente, veremos como a linguagem Java trabalha com seus valores numéricos, quando misturamos tipos primitivos. Em seguida, iremos nos aprofundar na Teoria de Orientação a Objeto.

Veremos que o objetivo de todo esse conteúdo está relacionado ao processo de programação e a como devemos pensar a construção de um sistema para que ele atinja, da melhor forma possível, as necessidades identificadas pelas regras de negócio definidas pelo cliente.

Ao programarmos um sistema, muitas vezes trabalhamos em equipe. Devemos ter em mente que esse sistema que estamos construindo poderá ser alterado, corrigido ou implementado por diversos outros programadores e que futuramente poderão dar continuidade a ele. Assim, utilizamos os vários recursos de programação existentes na orientação a objetos (que veremos neste livro-texto), com o intuito de determinar uma linha de ação e de compreensão do sistema às possíveis futuras equipes que venham a trabalhar nesse (ou com esse) mesmo sistema.

Quando trabalhamos com linguagens fortemente tipadas, devemos tomar alguns cuidados ao misturarmos tipos de dados diferentes em algumas ocasiões. Isso é muito comum quando lidamos com fórmulas matemáticas, em que misturamos tipos inteiros e tipos reais em uma mesma equação. Para algumas situações, o sistema se resolve automaticamente, mas para outras precisamos forçar a alteração da tipagem dos dados, a fim de que os cálculos ocorram corretamente. A ideia é que o compilador trabalha com variáveis de valores numéricos diversos, utilizando quantidades de memória e processamentos diferenciados, de acordo com a tipagem definida para aquela variável.

O tipo numérico que se utiliza de menos memória e menos processamento é o tipo byte e o que se utiliza de mais memória, além de mais processamento, é o tipo double.

A **promotion** é a "elevação" dos tipos numéricos para um tipo que pode representar uma maior grandeza.

As regras definem que um valor ou uma variável de tipo primitivo de menor capacidade numérica pode ser utilizado no lugar em que seria utilizado um valor ou uma variável de tipo com maior capacidade numérica.

A promotion não provoca perda de valores. Ela é feita implicitamente (automaticamente) pelo compilador. Assim, quando reservamos um espaço maior e preparamos o sistema para trabalhar com

um determinado tipo de dado, caso tenhamos um tipo de dado inferior, isso não alterará em nada o processamento previsto. Isso tudo significa que para o compilador trabalhar com um dado que possua um tipo inferior ao que foi reservado, o compilador fará a "promoção" automática daquela informação e a processará no mesmo espaço de memória que já estava reservado para isso, o que significa que a promoção acontece automaticamente.

Portanto, a promotion é implícita (automática) e ocorre ao trabalharmos com tipos "menores" quando poderíamos trabalhar com tipos "maiores". A ordem sequencial de promotion (do tipo de menor capacidade numérica para o de maior capacidade) é como segue:

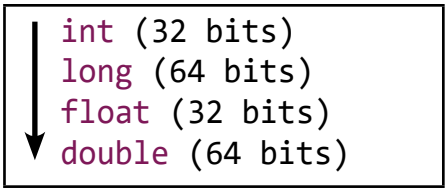


Figura 18

Exemplos de promotions:

```
int f = 4;
long g = 5L + f; //promotion do valor de f, que é int; para long
float h = 3.4f + g; //promotion do valor de g, que é long, para float
double i = 2.5 + h; //promotion do valor de h, que é float, para double
```

Observação

Devemos entender que a promoção ocorre para o valor da variável, e não para a variável (o tipo da variável não se altera).

O **casting** é a mudança de um tipo para outro de forma explícita (forçada) por meio do operador de casting (ou seja, o "nome do tipo" entre parênteses).

Para que possamos trabalhar com tipos que necessitam de memória de processamento maiores do que aquele que foi reservado inicialmente (na declaração do tipo), teremos antes que transformar aquela informação para que ela possa ser trabalhada naquele espaço que foi reservado (que é menor). Essa transformação pode causar a perda de dados (perda de valores ou de precisão), por isso deve ser determinada explicitamente pelo desenvolvedor.

Quando ocorre algum tipo de "erro" nesse sentido, o compilador pode, durante a compilação, detectar a impossibilidade da operação (não realizando-a). Do contrário, não podendo detectá-la no código, poderá ocorrer um erro em tempo de execução, caso a conversão do valor não seja possível de realizar.

Assim, a demotion (que chamamos de **casting**) é uma espécie de "rebaixamento" do tipo numérico para um tipo que representa uma menor grandeza.

A ordem sequencial de demotion (de um tipo de maior capacidade numérica para um de menor capacidade) é como segue:

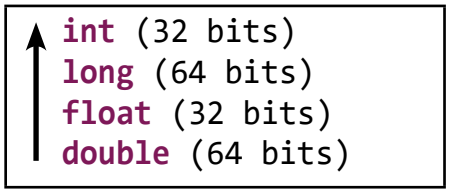


Figura 19

Exemplo de casting:

```
double j = 2.5;
float k = 5.2f + (float)j; //casting para float
long l = 3L + (long)k; //casting para long
int m = 7 + (int)l; //casting para int
short n = (short)m; //casting para short
byte o = (byte)n; //casting para byte
```

Lembrete

Um casting deve ser utilizado sempre que atribuímos a um tipo de menor abrangência um valor de um tipo de maior abrangência.

Assim como a promoção ocorre sobre um valor, o casting também ocorrerá sobre o valor da variável, e não sobre a variável (o tipo da variável não se altera).

3.2 Modificadores de acesso

Os modificadores de acesso são termos que utilizamos junto à declaração das classes, dos atributos e dos métodos e que modificam a forma de acessarmos esses elementos por meio dos códigos de programas.

Vimos que um sistema gerado em linguagem Java é baseado inteiramente na programação de classes e que uma classe possui, basicamente, atributos (dados) e métodos (ações). Vimos também que cada classe compilada é um arquivo com extensão .class.

Quando criamos um sistema, nele é gerada uma infinidade de classes, de forma que diversos arquivos são gerados e utilizados na sua construção. Dessa forma, podemos organizar nosso sistema em pacotes, ou packages (que veremos mais adiante), e em cada pacote inserimos as classes que pertencem a uma determinada categoria. Dessa forma, podemos definir se classes de diversos pacotes podem acessar-se

mutuamente ou se uma determinada classe (ou seus elementos) foi construída para ser acessada apenas por um determinado grupo de classes, e não por outros.

Para controlarmos os acessos das classes, utilizamos quatro modificadores de acesso disponibilizados, que são:

- public;
- private;
- protected;
- (default) – ou sem modificador.

Observação

Esses modificadores de acesso controlam não só o acesso à alteração do valor da variável, mas também o acesso à leitura desse valor.

O modificador public (público)

Uma declaração (de classes, atributos ou métodos) com o modificador public indica que o elemento pode ser acessado de qualquer lugar e por qualquer entidade (objeto) que possa visualizar a classe a que esse elemento pertence.

Exemplo:

```
public int idadePessoa;
public void imprimirDados() {...}
```

O modificador private (privado)

Uma declaração (de atributos ou métodos) com o modificador private indica que o elemento não pode ser acessado (visualizado) ou utilizado por nenhuma outra classe, a não ser apenas por métodos da própria classe a que pertence.

Exemplo:

```
private String nomeAluno;
private double calcularPeso() {...}
```

O modificador protected (protegido)

O modificador protected torna o elemento (atributo ou método) acessível às classes do mesmo pacote ou às classes relacionadas por herança (que veremos mais adiante neste curso). Seus elementos não são acessíveis a outras classes fora do pacote em que foram declarados.

Exemplo:

```
protected JButton b01;
protected String getEnderecoCliente() {...}
```

O (default) (padrão ou sem modificador)

Uma declaração (de classes, atributos ou métodos) em que não há definição de modificador possui o que chamamos de modificador padrão (default), fazendo com que sejam acessíveis somente por classes do mesmo pacote.

Exemplo:

```
class Principal { ... }
```

Ou, ainda:

```
boolean permiteProsseguimento;
double verificarAlturaRelativa() {...}
```

A tabela de níveis de acesso resume os itens explicativos sobre os modificadores de acesso:

Quadro 1 – Tabela de níveis de acesso

Modificador	Classe	Package (pacote)	Subclasse (classe filha)	Qualquer classe
public	Sim	Sim	Sim	Sim
protected	Sim	Sim	Sim	Não
(default)	Sim	Sim	Não	Não
private	Sim	Não	Não	Não

4 ELEMENTOS CARACTERÍSTICOS DA ORIENTAÇÃO A OBJETO

Alguns conceitos formam o que chamamos de bases da orientação a objetos. Veremos neste item alguns de seus elementos.

4.1 Encapsulamento

O encapsulamento, considerado um dos pilares da orientação a objetos, é um recurso em que "protegemos" os atributos de uma classe, fazendo com que todo o controle de acesso à leitura ou alteração de seu valor seja feito por meio dos métodos da classe.

Para termos uma ideia de como esse recurso é comum e de grande importância na programação, vários frameworks se utilizam dele na sua funcionalidade.

Observação

Um framework é um conjunto de códigos, classes e bibliotecas de classes, geralmente utilizado como ferramenta para o desenvolvimento de sistemas e criado para solucionar problemas comuns. Existem muitos frameworks criados para programação em linguagem Java e que podem agilizar o processo de construção de softwares, diminuindo o tempo de desenvolvimento e, consequentemente, o custo do projeto. Alguns frameworks Java são muito conhecidos, como o Hibernate (para persistência de dados), o Spark (para desenvolvimento de serviços web) e o Spring (que dispõe uma série de recursos que simplificam o desenvolvimento de aplicações Java).

Basicamente, para que uma classe seja encapsulada, basta que todos os seus atributos fiquem com o modificador de acesso private e que se gerem os métodos setters e getters para cada um deles.

- Os métodos setters são os métodos que vão alterar o valor dos atributos (sem retorno de valor).
- Os métodos getters são os métodos que vão retornar o valor dos atributos (com retorno de valor).

No encapsulamento, o nome do método deve iniciar com a palavra set ou get (em minúsculo) e mais o nome exato do atributo iniciado com maiúsculo (setAtributo ou getAtributo).

A sintaxe padrão desses métodos é a seguinte:

```
public void setAtributo (tipoAtributo variavel) {
    atributo = variavel;
}
public tipoAtributo getAtributo () {
    return atributo;
  }
```

Exemplo de um encapsulamento: encapsulando a classe Pessoa (que já possui os atributos nome e idade):

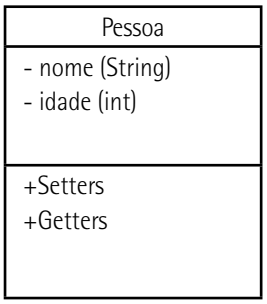


Figura 20 – Representação (UML) da classe Pessoa encapsulada

```
public class Pessoa {
    private String nome;
    private int idade;
      // Métodos Setters e Getters
    public void setNome (String s) {
      nome = s;
    }
    public String getNome () {
      return nome;
    }
    public void setIdade (int i) {
      idade = i;
    }
    public int getIdade () {
      return idade;
    }
}
```

Observação

Os métodos getters e setters são públicos, enquanto os atributos da classe são privados.

Na figura 20, os elementos privados são representados iniciando-se com o sinal - (menos) e os elementos públicos são representados iniciando-se com o sinal + (mais). Essa é uma das características da UML.

4.2 Sobrecarga de métodos

A **assinatura** de um método é o conjunto formado pelo seu nome e pelos seus parâmetros. Assim, na declaração do método temos:

```
modificador(es) tipoDeRetorno nomeDoMetodo(Parâmetros) {
       (assinatura do método)
}
```

Sabe-se que uma classe não permite que haja dois métodos com a mesma assinatura. Sendo assim, quando há, numa mesma classe, dois ou mais métodos com o mesmo nome, ambos devem obrigatoriamente estar diferenciados em seus parâmetros, caracterizando uma sobrecarga do método.

Desse modo, **sobrecarga de métodos** é um tipo de polimorfismo caracterizado pela existência de mais de um método com mesmo nome em uma mesma classe (ou em classes que caracterizam herança – conceito que veremos adiante), diferenciados em seus parâmetros (na **quantidade** ou na **tipagem** dos parâmetros).

Exemplo: classe Calculadora com três métodos de nome "somar", porém diferenciados nos parâmetros:

```
// Um exemplo de Sobrecarga de Métodos (do método "somar")
public class Calculadora {
    // Método "somar" com 2 parâmetros do tipo "int"
    public int somar (int a, int b) {
      return (a + b);
    }
    // Método "somar" com 3 parâmetros do tipo "int"
    public int somar (int a, int b, int c) {
      return (a + b + c);
    }
    // Método "somar" com 2 parâmetros do tipo "double"
    public double somar (double a, double b) {
      return (a + b);
    }
}
```

Percebe-se que não há problema dos três métodos ("somar") coexistirem numa mesma classe, já que há diferenças na definição dos seus parâmetros, seja na quantidade de parâmetros, seja nos tipos nele existentes.

4.3 Método construtor

A instância de uma classe é a geração do objeto que a representa na memória RAM. Sabe-se que podemos gerar várias instâncias a partir de uma mesma classe, de modo que cada objeto gerado conterá as mesmas características (mesmos tipos de informações, podendo realizar as mesmas ações definidas pela classe), porém contendo, na maioria das vezes, informações diferentes.

Um método construtor é o método que é acionado toda vez que uma classe é instanciada, e toda classe possui um método construtor.

Quando **não** criamos um método construtor explicitamente em uma classe, mesmo assim ela possuirá um método construtor padrão sem parâmetros do qual o compilador se utilizará para gerar a classe em memória (instanciar a classe gerando o objeto).

O objetivo do método construtor é dar as características iniciais de um objeto assim que ele é gerado na memória. Podemos, com o método construtor de uma classe, tanto dar valores iniciais aos seus atributos quanto preparar uma situação inicial (rodando outros métodos, inclusive de outras classes) para que ele opere de acordo com o exigido pelo sistema.

As características de um método construtor são:

- ele deve possuir exatamente o mesmo nome da classe;
- ele não deve possuir qualquer definição de tipo de retorno;
- ele deve ser público.

Sua sintaxe é:

```
public NomeDaClasse (parametros) {
    ...
}
```

Observação

Uma vez determinados um ou mais métodos construtores para uma classe, para que ela seja instanciada deve-se utilizar um dos métodos construtores nela definidos.

Uma classe pode possuir mais de um método construtor (caracterizando, nesse caso, sobrecarga de métodos construtores).

Exemplo:

```
public class Pessoa {
    // atributos da Classe Pessoa
    private String nome;
    private int idade;
    private double altura;
    // #####  Métodos Construtores (MC)  #####
    // MC com apenas um parâmetro
    public Pessoa(String s) {
      nome = s;
    }
    // MC com três parâmetros equivalentes a seus atributos
    public Pessoa(String s, int i, double d) {
      nome = s;
      idade = i;
      altura = d;
    }
}
```

A classe Pessoa do exemplo que acabamos de apresentar possui dois métodos construtores, de modo que, para instanciar um objeto a partir dessa classe, o programador deverá se utilizar de um desses dois métodos construtores, assim como visto na sintaxe a seguir:

```
nomeClasse nomeObjeto = new MetodoConstrutor();
```

Ou, ainda:

```
nomeClasse nomeObjeto;
...
nomeObjeto = new MetodoConstrutor();
```

O último termo da declaração da instância é uma chamada a um método construtor, que muitas vezes é apenas o método construtor padrão quando ele não é especificado na classe.

4.4 Herança

Assim como o encapsulamento, a herança é considerada um dos pilares da orientação a objetos, em que se caracteriza a especialização e generalização de classes.

A herança é um recurso de orientação a objetos que permite a criação de novas classes a partir das características de outra classe já criada. Assim, há um reaproveitamento de código, de modo que uma classe herdará todos os métodos e todos os atributos de outra classe.

Desta forma, as classes que herdam as características de outras classes são chamadas de **subclasses** (ou classes filhas) e as classes já existentes mas que deram origem às subclasses são chamadas de **superclasses** (classes mãe). Gera-se então uma hierarquia de classes, criando assim classes mais genéricas (as superclasses) com as características mais gerais e classes mais específicas (as subclasses) com as características mais individuais.

Na herança em Java, uma classe filha pode herdar apenas uma classe mãe, já que em Java não há herança múltipla. Essa classe filha possuirá todos os atributos e todos os métodos da classe mãe, independentemente do modificador de acesso que cada um deles tiver. Imaginando que a classe mãe herde elementos de outra classe, se uma classe a herdar, tornando-se filha, herdará também tudo o que as superclasses da primeira tiver. Para que uma classe herde outra classe, deve-se inserir na declaração da primeira (classe filha) a palavra extends seguida do nome da classe que ela está herdando (classe mãe). É importante se ter em mente que, pelo conceito de herança, quando uma classe herda outra classe (quando uma classe filha herda uma classe mãe), ela passa a "ser" essa outra classe (assim, a classe filha é a classe mãe).

Vejamos um exemplo de herança. Imaginemos que vamos criar as classes para controlar a relação comercial de uma empresa em que existem os funcionários e os clientes (pessoas físicas). Podemos com isso criar as duas classes (classe Funcionario e classe Cliente), de forma que ambas terão atributos e métodos em comum, além dos seus atributos e métodos individuais. Dessa forma, pode-se fazer com que as características comuns (gerais) fiquem em uma classe mais genérica (por exemplo, a classe Pessoa) e as características individuais (específicas) permaneçam nas próprias classes filhas, especificando-as.

Podemos identificar, neste caso, os conceitos de generalização e especificação das características dessas classes, de forma a termos o seguinte cenário:

A classe Pessoa:

```
public class Pessoa {
    private String nome;
    private String cpf;
    //... e demais características comuns
}
```

A classe Funcionario, que herda a classe Pessoa:

```
public class Funcionario extends Pessoa {
    private String funcional;
    private double salario;
    //... e demais características específicas
}
```

A classe Cliente, que herda a classe Pessoa:

```
public class Cliente extends Pessoa {
    private String projeto;
    private String areaNegocio;
    //... e demais características específicas
}
```

Observação

Pelos conceitos que acabamos de apresentar, ambas as classes (Funcionario e Cliente) conterão os atributos nome e cpf existentes na classe Pessoa, assim como podemos dizer que Funcionario é Pessoa e Cliente é Pessoa.

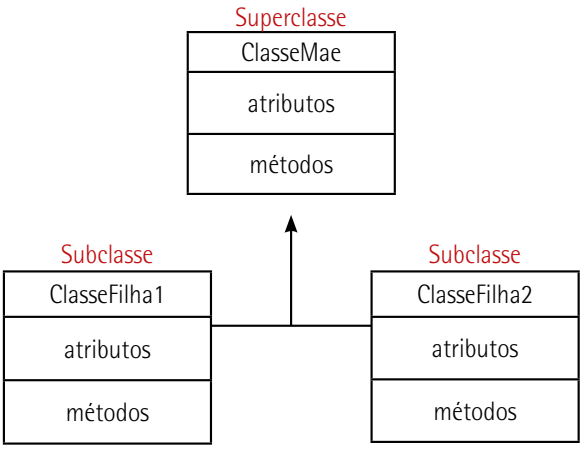


Figura 21 – Representação de herança com uma superclasse e duas subclasses

A linguagem Java não admite **herança múltipla** (uma subclasse herdando várias superclasses ao mesmo tempo), diferentemente de outras linguagens (como C++ e Python).

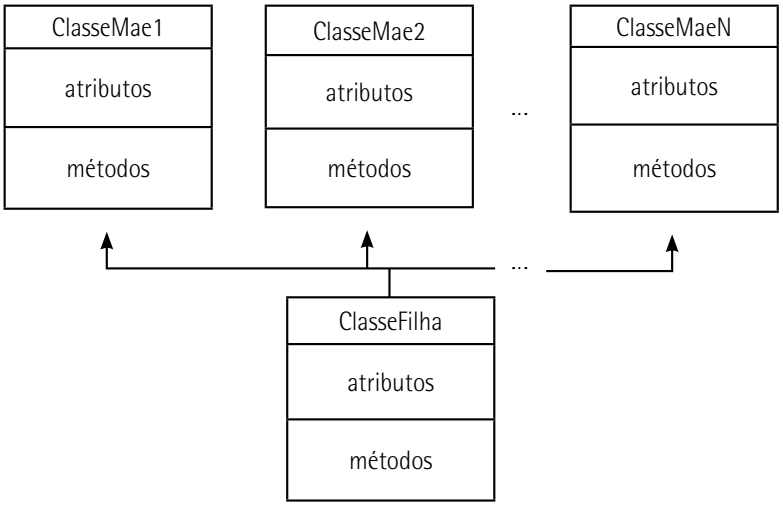


Figura 22 – Representação de herança múltipla para linguagens como C++ e Python

Apesar de não ser possível se fazer uma herança múltipla, o Java permite o que chamamos de **herança encadeada**, em que uma classe herda outra classe, que por sua vez herda outra classe, e assim por diante (veja a imagem a seguir).

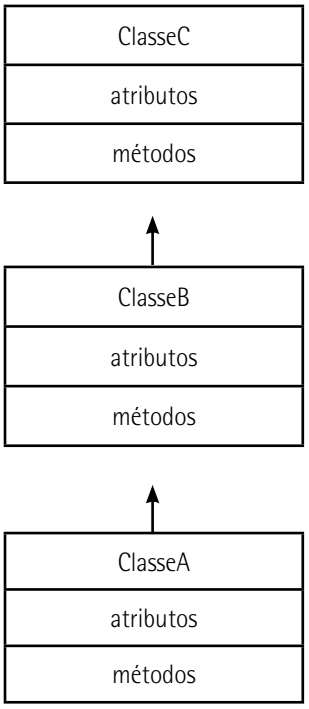


Figura 23 – Representação de herança encadeada permitida pelo Java, em que a ClasseA herda a ClasseB, que herda a ClasseC

Nessa última representação, temos que a ClasseA conterá **todos** os elementos (atributos e métodos) das classes ClasseB e ClasseC. A classe ClasseB foi herdada diretamente pela ClasseA, e a classe ClasseC por encadeamento pela ClasseA.

Observação

Na linguagem Java, a classe "mãe de todas as classes" é a classe Object. Quando criamos uma classe em Java, mesmo que não explicitemos uma herança (com o termo extends), ela automaticamente irá herdar a classe Object.

Saiba mais

Saiba mais sobre herança em Java no capítulo 8, itens 8.2 e 8.4, da obra a seguir:

BARNES, D. J.; KÖLLING, M. *Programação orientada a objetos com Java*: uma introdução prática utilizando o BlueJ. São Paulo: Pearson Prentice Hall, 2004.

4.5 Sobrescrita de métodos

Uma classe filha (ou subclasse) herdará todos os métodos da classe mãe (ou superclasses). Dessa forma, se instanciarmos uma classe filha, podemos acionar os métodos existentes na classe mãe, desde que eles não sejam privados. No entanto, algumas vezes precisamos que aquele método tenha uma ação diferente, para a classe filha, daquela prevista e codificada na classe mãe, a fim de permitir que a subclasse tenha um comportamento mais específico e individual do que aquele esperado e definido pelo método da superclasse.

Para isso, recriamos na subclasse aquele ou aqueles métodos da superclasse com a mesma assinatura, porém implementados com as alterações específicas ao "negócio" da própria subclasse. Assim, ao instanciarmos a classe filha e acionarmos aquele método, acionaremos o método da classe filha, e não o da classe mãe.

Deve-se lembrar que, depois de reescrito, não é mais possível, a partir de uma outra classe, acionarmos diretamente, para um objeto gerado com a classe filha, aquele método que pertence à classe mãe.

Observação

Somente é possível invocar o método sobrescrito da classe mãe invocando-o a partir de um objeto da classe mãe, ou de dentro da classe filha (utilizando a palavra super).

Assim, sobrescrita de métodos é a existência de métodos com exatamente a mesma assinatura, tanto na superclasse quanto na subclasse, de forma que o método na subclasse estará sobrescrevendo (substituindo) o método da superclasse.

Lembrete

A assinatura de um método é a parte da declaração que comporta o nome do método e seus parâmetros.

Exemplo:

A classe Funcionario:

```
public class Funcionario {
    public double salario;
    //... e demais atributos
    public void mostraSalario () {
       System.out.println("Salário: " + salario);
    }
    public double calculaBonificacao() {
       double bonif = salario * 0.0165;
       return bonif;
    }
}
```

A classe Gerente herda a classe Funcionario:

```
public class Gerente extends Funcionario {
    // para o Gerente, a bonificação é
    // maior que a dos Funcionários
    public double calculaBonificacao() {
       double bonif = salario * 0.0205;
       return bonif;
    }
}
```

Assim, a partir de uma instância da classe Gerente podemos rodar os métodos mostraSalario() e calculaBonificacao(). Porém, no caso desse último método, o que será acionado será o da própria classe Gerente, e não aquele da classe Mãe, substituindo-o.

Exemplo de aplicação

De acordo com os conceitos de orientação a objetos, podemos afirmar que:

I – Utilizamos o encapsulamento para criar os métodos construtores que encapsulam os objetos em memória.

II – Métodos construtores são aqueles que geram os objetos em memória e somente são acionados no momento da instância da classe, sendo que uma classe pode conter vários métodos construtores, o que caracterizaria uma sobrecarga de métodos.

III – Herança é um recurso que permite a generalização e especificação das classes, caracterizando também o reaproveitamento de código.

IV – Sabendo que não pode haver mais de um método com mesma assinatura em uma classe, o fato de uma subclasse "ser" uma superclasse impede também que haja na subclasse um método que possua a mesma assinatura de um método da superclasse.

V – Uma classe chamada ClasseA contém um atributo xyz de um determinado tipo, mas sem o modificador na sua declaração. Sabendo que a ClasseA está no pacote pac01, então a ClasseB, que está no pacote pac02, não poderá acessar diretamente o atributo xyz, a não ser por meio de métodos da ClasseA.

Quais das afirmações são corretas?

A) Somente I, III e IV.

B) Somente II, III e V.

C) Somente II, III e IV.

D) Somente I, II e III.

E) Somente I, IV e V.

Resposta correta: alternativa B.

Resolução

I – Afirmativa incorreta.

Justificativa: utilizamos encapsulamento para garantir um controle de acesso aos atributos da classe, fazendo com que eles só possam ser acessados a partir de métodos da própria classe.

II – Afirmativa correta.

Justificativa: devemos sempre lembrar que quando estamos instanciando uma classe, acionamos o seu método construtor. Caso criemos explicitamente métodos construtores no corpo da classe, somos obrigados a utilizar algum deles para gerar a sua instância, de forma que uma classe pode ter sobrecarga de métodos construtores, ou seja, mais de um método construtor, diferenciados entre si por seus parâmetros, seja na quantidade deles, seja no tipo de pelo menos um deles.

III – Afirmativa correta.

Justificativa: na herança de classes temos uma classe que é mais genérica (a superclasse), cujos elementos pertencerão a todas as subclasses que a herdam, e temos também as classes mais específicas (as subclasses), que, além de possuírem todos os elementos da superclasse (atributos e métodos), caracterizando uma forma de reaproveitamento de código (já que não se precisa codificá-los novamente nas subclasses), podem possuir seus próprios atributos e métodos, que são restritos a elas, e por isso são específicos de cada uma delas.

IV – Afirmativa incorreta.

Justificativa: a incoerência desta afirmação está no fato de uma subclasse poder possuir um método exatamente com a mesma assinatura da superclasse, caracterizando com isso a sobrescrita de método. Isso pode acontecer quando o método da superclasse não é satisfatório para aquela subclasse, de forma que, ao se gerar outro método com mesma assinatura na subclasse, se está substituindo aquele da superclasse. No entanto o conceito de uma subclasse "ser" uma superclasse é correto, já que ele possui tudo o que a superclasse possui.

V – Afirmativa correta.

Justificativa: devemos lembrar que um atributo sem a declaração de seu modificador de acesso significa que aquele atributo possui o modificador default, o qual permite acesso direto apenas às classes do mesmo pacote (package). Uma classe que esteja em outro pacote somente poderá acessar aquele atributo a partir de métodos públicos da classe que o contém.

4.6 ArrayList e Vectors

O ArrayList é implementado como um array de objetos. Ele é dimensionado dinamicamente (com relação à memória utilizada, sempre que for necessário, o seu tamanho aumenta proporcionalmente a 50% do tamanho da lista). Além disso o ArrayList permite que seus elementos sejam adicionados por meio do método add(), acessados diretamente pelo método get() e removidos pelo método remove().

Vector e ArrayList são muito semelhantes na sua utilização. Ambos geram vetores de objetos, porém há uma diferença na sua utilização no que diz respeito à utilização de threads (item que vamos ver adiante), de modo que vários threads podem operar em um ArrayList ao mesmo tempo (e por isso é

considerado não sincronizado, ou assíncrono); no entanto apenas um único thread pode operar em um Vector por vez (e por isso é considerado sincronizado, ou síncrono).

No exemplo a seguir, é gerada a classe ClasseTipo, com seus atributos e métodos:

```
public class ClasseTipo {
    public int atrib01;
    public String atrib02;
    public double atrib03;
    public ClasseTipo(int i, String s, double d) {
      atrib01 = i;
      atrib02 = s;
      atrib03 = d;
    }
    public void mostraCT() {
      System.out.println(atrib01 + ": " + atrib02
                    + " (" + atrib03 + ")");
    }
}
```

Assim, a partir da classe ClasseTipo, vamos gerar um ArrayList (um vetor) de objetos gerados a partir daquela classe (ClasseTipo).

Observação

Os objetos, criados utilizando-se seu método construtor, serão gerados a partir de uma mesma variável (ct), porém cada objeto é guardado independente e separadamente tanto no ArrayList quanto no Vector.

Com ArrayList:

```
ArrayList<ClasseTipo> conjunto = new ArrayList();
ClasseTipo ct;
ct = new ClasseTipo(1, "Primeiro", 25.6);
conjunto.add(ct);
ct = new ClasseTipo(2, "Segundo", 25.7);
conjunto.add(ct);
ct = new ClasseTipo(3, "Terceiro", 25.8);
conjunto.add(ct);
ct = null;
int tam = conjunto.size();
// lendo cada um dos objetos do vetor
for (int i = 0; i < tam; i++) {
    conjunto.get(i).mostraCT();
}
```

Observação

A declaração de ArrayLists leva os símbolos < e >, os quais envolvem o nome da classe que representa todos os objetos daquele ArrayList (<Classe>), indicando o tipo do objeto que será inserido no array.

O objeto ct é alterado a cada nova instância da ClasseTipo, porém pelo fato de serem a cada momento adicionados no ArrayList, os endereços de memória desses objetos ficarão guardados no array mantendo esses objetos ativos e acessíveis dentro do ArrayList, podendo o objeto ct ter seu endereço alterado para a nova instância.

Com Vector:

```
Vector<ClasseTipo> conjunto = new Vector();
ClasseTipo ct;
ct = new ClasseTipo(1, "Primeiro", 25.6);
conjunto.add(ct);
ct = new ClasseTipo(2, "Segundo", 25.7);
conjunto.add(ct);
ct = new ClasseTipo(3, "Terceiro", 25.8);
conjunto.add(ct);
ct = null;
int tam = conjunto.size();
for (int i = 0; i < tam; i++) {
    conjunto.get(i).mostraCT();
}
```

Observação

Assim como a declaração de ArrayLists, a declaração de Vectors também leva os símbolos < e >, os quais envolvem o nome da classe que representa todos os objetos daquele Vector (<Classe>).

Também para o Vector, tem-se a mesma observação em relação ao objeto ct descrita nas observações do ArrayList.

Percebe-se que a forma, ou estrutura, de utilização de cada um deles (ArrayList e Vector) é semelhante.

4.7 Trabalhando com strings

Sabemos que a classe String representa os tipos de dados alfanuméricos como palavras, frases e textos em geral. Na linguagem Java, esse tipo de dado não é um tipo primitivo, de forma que sua classe corresponde a um conjunto de caracteres, os quais contêm uma série de métodos que permitem a manipulação dessas strings de diversas formas.

Podemos, a partir dos métodos que pertencem à classe String, verificar seu tamanho (a quantidade de caracteres que possui), dividi-la em partes, alterar uma parte dela substituindo por outra, verificar se existe um texto específico dentro dela e descobrir em que posição está, entre outras ações.

Estes métodos são úteis quando vamos trabalhar com organização de dados, mineração de dados, inteligência artificial, aprendizado de máquina (machine learning), análise de sentimentos e muito mais.

Neste item, vamos ver alguns métodos (os mais comuns) e mostrar como trabalhar com cada um deles a fim de aprendermos a analisar textos.

O método length()

Esse método retorna um valor do tipo int que representa o tamanho de uma string, ou seja, a quantidade de caracteres que a string possui, lembrando que, por caracteres, estão compreendidos números, letras maiúsculas, letras minúsculas, símbolos (@, !, #, $ ...) e o espaço.

Sua sintaxe é:

```
<nome_da_String>.length()
```

Exemplo:

```
"...
String nome = "Fulano de Tal";
int tam = nome.length(); // no caso: tam = 13
..."
```

Neste exemplo, a variável tam receberá o tamanho da variável nome, já que o objeto nome representa uma string (é um objeto do tipo string).

O método toUpperCase() e toLowerCase()

Esses dois métodos retornam o valor da variável alterado de duas formas:

- **toUpperCase()**: retorna o texto inteiramente em maiúsculo (caixa alta);
- **toLowerCase()**: retorna o texto inteiramente em minúsculo (caixa baixa).

É importante saber que esses dois métodos não alteram o valor da variável, mas sim **retornam** o valor equivalente à sua característica.

Sua sintaxe é:

```
<nome_da_String>.toUpperCase()
<nome_da_String>.toLowerCase()
```

Exemplo:

```
"...
String nome = "Fulano de Tal";
String txt1 = nome.toUpperCase();
// no caso, txt1 recebe: FULANO DE TAL

String txt2 = nome.toLowerCase();
// no caso, txt2 recebe: fulano de tal
// mas a variável "nome" continua com o valor inicial:
// Fulano de Tal
..."
```

O método trim()

Esse método retorna o texto original (um valor do tipo string) no qual são removidos todos os espaços em branco que aparecem no início e no final do texto original. É importante observar que esse método não altera o valor da variável original, mas apenas **retorna** o valor alterado. Outra observação importante é a de que os espaços internos, que porventura estejam a mais, continuarão intactos (sem alteração).

Sua sintaxe é:

```
<nome_da_String>.trim()
```

Exemplo:

```
"...
String texto = "   um    texto qualquer    ";
String txt = texto.trim();
System.out.println("[" + txt + "]");
// imprimirá: [um    texto qualquer]
System.out.println("[" + texto + "]");
// imprimirá: [   um    texto qualquer    ]
..."
```

No exemplo que acabamos de apresentar, o texto contém mais de um espaço antes e depois das palavras, e na variável txt, que recebe o valor alterado da variável texto, não contém esses espaços iniciais e finais (foram retirados inteiramente). É importante observar que entre a palavra um e a palavra texto contém mais de um espaço e que ele foi mantido nas duas variáveis (texto e txt), já que o método trim() não mexe nos espaços internos.

Observação

Os colchetes da impressão foram inseridos apenas no ato da impressão das variáveis, e não em seus valores internos, para que fosse possível verificar de forma fácil a alteração ocorrida sobre o texto a partir da utilização do método.

O método charAt(...)

Diferentemente de outras linguagens de programação, na linguagem Java uma string não é um array de caracteres, de modo que para acessarmos um determinado caractere no interior da string, devemos nos utilizar de métodos próprios da classe.

O método charAt(...) permite então que acessemos um caractere numa determinada posição do interior de uma string. A posição desejada é um valor numérico inteiro, com um valor equivalente ao de um índice de arrays (ou seja, começando com o valor 0), e que deve ser indicado no parâmetro do método. Esse método retornará um valor do tipo char que representará o caractere existente na posição indicada no parâmetro.

Sua sintaxe é:

```
<nome_da_String>.charAt(<num_posicao>)
```

Exemplo:

```
"...
String texto = "palavra";
char c1 = texto.charAt(0); // c1 recebe 'p'
char c2 = texto.charAt(6); // c2 recebe 'a' (o último 'a')
char c3 = texto.charAt(8);
// este último resulta em ERRO pois é uma posição
// inexistente, gerando uma exceção do tipo
// "StringIndexOutOfBoundsException"
..."
```

O método indexOf(...)

Utilizamos esse método quando precisamos procurar um trecho de String (uma substring) ou até uma letra dentro da String. O método retornará um número inteiro equivalente ao valor da posição inicial da substring procurada. É importante ressaltar que caso haja repetição da substring, ou da letra, no texto original, o método indexOf() retornará a posição apenas do primeiro caso encontrado do trecho a partir da posição inicial (que pode ser o próprio valor dessa posição, dependendo do caso).

Sua sintaxe é:

```
<nome_da_String>.indexOf(<substring>)
```

ou:

```
<nome_da_String>.indexOf(<substring>, <posic_inicial>)
```

Exemplo:

O exemplo a seguir mostra a posição de todas as letras a que aparecem no texto inicial, utilizando-se de um laço de repetição, sempre renovando a posição inicial de busca da substring (no caso, a posição da letra a):

```
"...
String texto = "aa batata";
int pos = texto.indexOf("a");
while (pos >= 0) {
    System.out.print(pos + "\t");
    pos = texto.indexOf("a", pos + 1);
}
// imprimirá: 0 1 4 6 8
// Porém:
System.out.print(texto.indexOf("w"));
// Imprimirá: -1
// pois esta String "w" não existe no texto original
..."
```

O método replace(...)

Esse método retorna um valor do tipo string, de modo que toda ocorrência encontrada com o trecho indicado no parâmetro (substring procurada) será substituído pela nova substring. Vale ressaltar que a variável original não sofre a alteração, ou seja, o método replace() **retorna** um valor alterado sem alterar o valor da variável original.

Caso o método não encontre a substring procurada, ele não resulta em erro, simplesmente retornando o valor original sem alterações.

Sua sintaxe é:

```
<nome_da_String>.replace(<substr_procurada>, <nova_substr>)
```

Exemplo:

```
"...
String texto = "palavra";
String txt = texto.replace("a", "x");
// txt receberá: pxlxvrx
// onde TODA ocorrência da substring "a" foi alterada,
// de modo que o valor da variável "texto"
// continua sendo "palavra".

txt = texto.replaceAll("k", "x");
// neste caso txt receberá: palavra
// já que a substring "k" não existe na String original.
..."
```

O método substring(...)

Esse método retorna um valor do tipo string equivalente a um trecho da string original. Os parâmetros deste método devem receber um número inteiro equivalente ao índice relativo à posição inicial e à posição final+1 (este último, opcional).

Sua sintaxe é:

```
<nome_da_String>.substring(<posic_inicial>)
```

ou:

```
<nome_da_String>.substring(<posic_inicial>, <posic_final>)
```

No primeiro caso, só com o valor da posição inicial, o texto resultante equivale ao trecho que inicia no índice indicado e termina no último índice (vai até o final da string original).

No segundo caso, o valor da posição inicial equivalerá ao índice exato do caractere que fará parte da substring resultante, e a posição final indica que a substring resultante terá todos os caracteres até um índice anterior ao valor indicado (vide exemplo a seguir).

É importante salientar que caso um dos valores de posição (seja o inicial ou o final) não exista no texto original (lembrando que o valor final pode sempre ser um valor acima do valor do último índice), esse comando resultará em um erro, interrompendo a execução do código e provocando o que chamamos de exceção (do tipo StringIndexOutOfBoundsException).

Exemplo:

```
"...
String texto = "um texto qualquer";
txt = texto.substring(3);
// txt receberá: "texto qualquer"
// já que na posição 3 está a
// primeira letra t da palavra "texto"
txt = texto.substring(3, 10);
// txt receberá: "texto q"
// já que no índice 9 (um antes do valor 10) temos
// a primeira letra "q" da palavra "qualquer".

txt = texto.substring(3, 30);
// Este comando resulta em ERRO,
// já que a posição 30 não existe na String original.
..."
```

O método split(...)

Esse método é muito utilizado em leitura de arquivos de texto externos ao programa, para identificar partes de seu texto que geralmente contêm dados específicos sobre configurações ou de banco de dados.

Este método retorna um array de strings (uma matriz de strings) que contém, em cada uma de suas posições, trechos (substrings) do texto original "recortado em partes". Esse recorte é realizado segundo uma referência, ou, ainda, uma substring, indicada no parâmetro do método.

Observação

O fato de retornar um array facilita sua manipulação, já que podemos ler um array utilizando uma estrutura for (vide exemplo a seguir).

Sua sintaxe é:

```
<nome_da_String>.split(<referencia>)
```

Quanto à substring de referência, podemos ter três situações:

- **caso essa substring seja encontrada**: neste caso ele gerará um array com todas as substring separadas pela referência, na ordem que elas aparecem (observação: a string de referência é retirada ao gerar as substrings – entende-se que esse valor é apenas uma referência para a identificação e separação das substrings);

- **caso essa substring** não seja encontrada: neste caso ele gerará um array com apenas um elemento, que é o próprio texto completo da string original;

- **caso seja uma string vazia ("")**: neste caso, retornará um array de strings em que cada um dos caracteres da string original ocupará uma posição no array resultante, de modo que o tamanho do array será igual ao tamanho da string original.

Exemplo:

```
"...
String texto = "um texto qualquer";
String[] matStr = texto.split("");
for (int x = 0; x < matStr.length; x++) {
    System.out.println(matStr[x]);
}
// Este laço de repetição irá imprimir letra a letra
// do texto original, uma abaixo da outra:
// u
// m
```

```
//
// ... e assim por diante, até
// e
// r
matStr = texto.split(" ");
for (int x = 0; x < matStr.length; x++) {
    System.out.println(matStr[x]);
}
// Este laço de repetição imprimirá:
// um
// texto
// qualquer
// ...já que a referência é um espaço (" "), e com isso,
// tudo que NÃO é "espaço", vira um elemento do array.
matStr = texto.split("w");
for (int x = 0; x < matStr.length; x++) {
    System.out.println(matStr[x]);
}
// Este laço de repetição imprimirá:
// um texto qualquer
// ...já que a referência ("w") é um valor
// inexistente no texto original.
..."
```

O método toCharArray()

Este método retorna um array de caracteres (valores do tipo char) com todos os caracteres da string original, cada um deles ocupando uma das posições do array, na ordem em que aparecem no texto original.

Cada elemento do array será considerado um dado do tipo char, e como tal deverá ser trabalhado.

Sua sintaxe é:

```
<nome_da_String>.split(<referencia>)
```

Exemplo:

```
"...
String texto = "um texto qualquer";
char[] matChar = texto.toCharArray();
for (int x = 0; x < matChar.length; x++) {
    System.out.println(matChar[x]);
}
// Este laço de repetição irá imprimir letra a letra
// do texto original, uma abaixo da outra:
// u
// m
//
// ... e assim por diante, até
// e
// r
..."
```

O método equals(...)

Quando uma string for instanciada como se fosse uma classe, as comparações realizadas com o operador **==** não funcionarão como funcionam com strings criadas na forma literal.

```
"...
// ##### Criando de forma literal:
String txt1 = "qualquer coisa";
String txt2 = "qualquer coisa";
if (txt1 == txt2) {
    System.out.println("São iguais");
} else {
    System.out.println("São diferentes");
}
// Neste caso, como ambas foram criadas de forma literal
// a estrutura condicional aceitará a comparação com o
// operador ==, e imprimirá que "São iguais".
// Obs.: Isto também funcionará para a situação
// de comparação com o método equals(...).
// ##### Criando na forma de Classes:
String txt3 = new String("qualquer coisa");
String txt4 = new String("qualquer coisa");
if (txt1 == txt2) System.out.println("Iguais - comp. 1");
if (txt1.equals(txt2)) System.out.println("Iguais - comp. 2");
if (txt1 == txt3) System.out.println("Iguais - comp. 3");
if (txt3 == txt4) System.out.println("Iguais - comp. 4");
if (txt1.equals(txt3)) System.out.println("Iguais - comp. 5");
if (txt3.equals(txt4)) System.out.println("Iguais - comp. 6");
// Nestes casos acima, as únicas saídas serão:
// Iguais - comp. 1
// Iguais - comp. 2
// Iguais - comp. 5
// Iguais - comp. 6
// Isto ocorre por que as comparações com o operador == só
// funcionarão se as duas variáveis tiverem sido criadas
// na sua forma literal.
// Caso a variável (qualquer uma delas) tenha sido criada na
// forma de instância da Classe String, a única forma de
// comparação possível será utilizando o método equals(...)
..."
```

O método format(...)

Esse método, que retorna um valor do tipo string, permite a formatação de dados como datas ou números, a fim de adequá-los à sua forma de amostragem, dependendo de como devem aparecer em relatórios ou textos em geral.

Existem várias maneiras de utilizarmos o método format(...). Aqui, vamos apresentar um conteúdo básico (mas suficiente) para conseguirmos configurar textos de relatórios.

O método se utiliza de alguns símbolos, interpretando-os à medida que os encontra ao longo da string. Vejamos um primeiro exemplo:

```
String nome = "José";
int dia = 4;
String mes = "maio";
int ano = 1950;
// -------------------------
String txtGeral = "O Sr. %s, nasceu em %s de %s de %s.";
String msg = String.format(txtGeral, nome, dia, mes, ano);
System.out.println(msg);
/*
Este programa imprimirá na tela da Console:
O Sr. José, nasceu em 4 de maio de 1950.
*/
```

Esse exemplo mostra como podemos criar uma mensagem que inicialmente não contém alguns de seus dados, mas que podem ser preenchidos ao longo do programa a fim de completar a frase a ser amostrada.

O símbolo %s que aparece no valor (texto) da variável txtGeral indica ao compilador que ali deverá ser inserido um valor (a ser interpretado como uma string). Perceba que há quatro desses símbolos no texto, de modo que o comando dado com o método format(...) contém como parâmetro cinco variáveis, todas elas separadas por vírgula, sendo a primeira o texto a ser interpretado, e as demais deverão contemplar todos os símbolos que aparecem ao longo do texto inicial.

Dessa forma, cada símbolo será substituído pelo valor da variável localizada após a variável que representa o texto original e relativa à sua ordem de aparição, ou seja: o primeiro símbolo será substituído pelo valor da segunda variável; o segundo símbolo será substituído pelo valor da terceira variável; e assim por diante, lembrando que **todos** os símbolos devem ter um valor correspondente definido após o texto.

Sabemos que a solução poderia também ser realizada utilizando-se de concatenação simples, como:

```
"...
String txtGeral = "O Sr. " + nome + ", nasceu em " + dia + " de " + mes + " de "
+ ano + ".";
..."
```

No entanto, muitas vezes nos deparamos com textos que exigem uma formatação específica de amostragem, como quando queremos que as datas fiquem num formato do tipo dd/mm/aaaa.

Nesse caso, se utilizarmos uma concatenação simples:

```
"...
int dia = 4;
int mes = 5;
int ano = 1950;
// ....
String msg = "No dia " + dia + "/" + mes + "/" + ano + ", nasceu...";
System.out.println(msg);
/*
Este programa imprimirá na tela da Console:
No dia 4/5/1950, nasceu...
*/
..."
```

Percebam que a data não apareceu num formato padrão que possamos utilizar em documentos ou relatórios.

Sendo assim, apresentamos alguns exemplos de utilização do método format(...), seguidos da explicação dos símbolos utilizados.

1º exemplo:

```
"...
String txt = "No dia %02d/%02d/%04d, nasceu...";
String msg = txt.format(txt, dia, mes, ano);
System.out.println(msg);
/*
Este programa imprimirá na tela da Console:
No dia 04/05/1950, nasceu...
*/
..."
```

Neste caso foi utilizado o símbolo %02d, de modo que:

- o d indica que se está trabalhando com números inteiros;
- o 0 indica que o valor textual deverá ser completado com zeros à esquerda;
- o 2 indica a quantidade de dígitos que devem compor a representação do número (assim como o 4 no símbolo %04d).

Outro exemplo da utilidade desse formato é a impressão de números de conta em banco, código (ou número de funcional) de funcionários etc.:

```
int icc = 572; // número da conta
int idv = 4; // dígito verificador
String cc1 = "Conta Corrente: %06d-%02d";
String ccFinal = cc1.format(cc1, icc, idv);
System.out.println(ccFinal);
/*
Este programa imprimirá na tela da Console:
Conta Corrente: 000572-04
*/
```

2º exemplo:

```
"...
int num = 1234567;
String val = "Valor: %,1d";
System.out.println(val.format(val, num));
/*

Este programa imprimirá na tela da Console:
Valor: 1.234.567
*/
..."
```

Neste caso, utilizamos o símbolo %,1d de modo que:

- A vírgula (,) indica que deverá ser impresso o separador de **milhares**. Ele pode aparecer como no sistema inglês, com vírgula, ou como no sistema brasileiro, com ponto, dependendo da configuração geral do micro do usuário.

- O 1 indica que ao menos um dígito, o da unidade, deverá aparecer. Os demais só aparecem se existirem no número.

3º exemplo:

```
"...
double med = 12345.67;
String medS = "Medida: %,1.5f";
System.out.println(medS.format(medS, med));
/*
Este programa imprimirá na tela da Console:
Medida: 12.345,67000
*/
..."
```

Neste caso, utilizamos o símbolo %,1.5f, de modo que:

- O f indica que estamos formatando um dado numérico de ponto flutuante, ou seja, um valor do tipo float ou double.

- O .5 indica que queremos que apareçam de forma fixa apenas cinco casas decimais depois da vírgula, que são completadas com zeros caso o número não tenha essa quantidade de casas decimais.

Saiba mais

Para saber mais sobre as diversas formatações que o método format(...) fornece, acesse o link a seguir, do site oficial do tutorial do Java (The JavaTM Tutorials), que mostra as possíveis formatações:

THE JAVA™ tutorials. *Oracle*, [s.d.]a. Disponível em: https://cutt.ly/YTDYEfo. Acesso em: 3 dez. 2025.

Exemplo de aplicação

Imagine que você foi contratado para dar manutenção em um sistema. O cliente solicitou que você acrescentasse uma nova funcionalidade em um código já existente, a qual consiste em analisar um conjunto de informações vindas de um arquivo texto na forma de array de strings, e que em cada posição desse array existe um texto que reúne diversas informações separadas pelo símbolo # (hashtag, cerquilha ou sustenido).

Exemplo de um trecho daquele array:

```
"...
matriz[25] = "Projeto FGH#Fulano de Tal#Filial de São Paulo#..."
matriz[26] = "Projeto XYZ#Beltrano dos Anzóis#Filial de Vitória#..."
matriz[27] = "Projeto KLM#..."
...
matriz[41] = "Projeto PQR#Ciclano das Couves#Filial de Salvador#..."
..."
```

Esse trecho do array mostra como os dados estão dispostos e dizem respeito a dados específicos como o nome dos projetos em andamento, quem é o responsável, em qual filial da empresa está sendo desenvolvido esse projeto, entre muitas outras informações.

Além disso, as informações deverão ser analisadas de várias formas, pois o sistema que as processa limita o tamanho de cada uma das informações, além de comparar valores numéricos existentes nas informações da string, com intervalos numéricos definidos em regras de negócio específicas de cada área da empresa.

Sendo assim, qual dos grupos de estruturas e métodos a seguir seria necessário para destrinchar cada um dos dados do array de informações que acabamos de mostrar, em relação às problemáticas descritas, e permitiria trabalhar com cada uma das informações individualmente?

A) estrutura for, estrutura if, método split() das strings, método legth() das strings.

B) estrutura if, estrutura switch – case, método replace() das strings e método substring() das strings.

C) estrutura for, estrutura switch – case, método replace() das strings e método length() das strings.

D) estrutura while, estrutura if, método toLowerCase() das strings e método trim() das strings.

E) estrutura for, estrutura switch – case, método trim() das strings e método toUpperCase() das strings.

Resposta correta: alternativa A.

Resolução

A) Alternativa correta.

Justificativa: a estrutura for nos permitiria acessar cada um dos elementos do array. O método split nos permitiria destrinchar cada string (cada elemento do array), a partir do caractere de referência (#). O método length nos permitiria definir o tamanho do array, além de informar o tamanho das informações, já que há restrições nas regras de negócio. A estrutura if nos permitiria realizar comparações sobre os valores numéricos e sobre os intervalos (segundo o enunciado).

B) Alternativa incorreta.

Justificativa: está faltando um laço de repetição para ler a matriz de strings. A estrutura switch – case permite apenas comparações de igualdade, não sendo possível verificar intervalos numéricos, segundo o que diz o enunciado. O método replace a princípio não auxilia nenhuma das necessidades elencadas no enunciado. O método substring só seria possível se soubéssemos a posição exata das strings #, o que seria possível com o método indexOf.

C) Alternativa incorreta.

Justificativa: a estrutura switch – case permite apenas comparações de igualdade, não sendo possível verificar intervalos numéricos, segundo o que diz o enunciado. O método replace a princípio não auxilia nenhuma das necessidades elencadas no enunciado.

D) Alternativa incorreta.

Justificativa: os métodos toUpperCase e trim a princípio não auxiliam nenhuma das necessidades elencadas no enunciado.

E) Alternativa incorreta.

Justificativa: os métodos toLowerCase e trim a princípio não auxiliam nenhuma das necessidades elencadas no enunciado.

Para finalizar esta unidade, observe os exemplos de aplicação a seguir. As resoluções são apresentadas na sequência.

Exemplos de aplicação

1) Criar uma classe encapsulada de nome Produto com os seguintes atributos: codigo (int), nome (String), qtdEstoque (int), este último representando a quantidade de produto em estoque. Nessa classe, criar um método construtor que recebe como parâmetro cada um dos atributos. Criar um método de nome mostrarDados(), que deve imprimir os valores de todos os atributos.

2) Criar uma classe encapsulada de nome Alimento que herda a classe Produto com o seguinte atributo: peso (double). Nessa classe, criar um método construtor que deve receber como parâmetro um valor para cada um dos atributos (lembrar que essa classe possui uma superclasse, que também deve ser acionada). Criar um método de nome mostrarDados(), que deve imprimir os valores de todos os atributos (inclusive os da superclasse).

3) Criar uma classe encapsulada de nome Bebida que herda a classe Produto com o seguinte atributo: volume (double). Nessa classe, criar um método construtor que recebe como parâmetro cada um dos atributos (lembrar que essa classe possui uma superclasse, que também deve ser acionada. Criar um método de nome mostrarDados(), que deve imprimir os valores de todos os atributos (inclusive os da superclasse).

4) Criar uma classe TesteProduto que possui o método main, gerando nele uma instância da classe Alimento e uma instância da classe Bebida (em ambos com informações a sua escolha) e acionando o método mostrarDados().

5) Criar uma classe encapsulada de nome Data com os seguintes atributos: dia (inteiro); mes (inteiro); ano (inteiro); e bissexto (lógico).

Regras específicas:

- Os métodos setters deverão ser privados, e os getters deverão ser públicos (o método set do atributo bissexto não deve ser criado).
- O programa não deve aceitar valores de mês menores do que 1 nem maiores do que 12.
- O programa não deve aceitar valores de ano ou de dia menores do que 1.
- Os métodos setters deverão sempre (além de validarem e inserirem o valor no respectivo atributo) retornar um valor lógico (true ou false) de acordo com a validação, de forma que se o valor for válido retornará true e se for inválido retornará false. Observação: caso se tente incluir um valor não permitido, deve-se imprimir na console o seguinte texto: "Data Inválida!".

- Ao receber o valor de um ano (no método setAno(...)), deve-se preencher também o valor do atributo bissexto (com true ou false), de modo que se o valor do ano for divisível por 4 (não negativo), deve-se imprimir na tela da console "Ano Bissexto", e o atributo bissexto deverá ficar com o valor true; caso contrário, deverá ficar com o valor false. Observação: existem várias outras regras para se determinar se um ano é ou não bissexto. Além disso, pela regra verdadeira nem todo valor de ano divisível por 4 representa um ano bissexto (como os anos que terminam com 00 – por exemplo, o ano 2100 não será bissexto). Mas essas e outras regras não estão sendo levadas em consideração neste exercício.

- Implementar o método que inclui valor ao atributo dia para que, além da validação já existente, valide-o de acordo com os meses do ano, de forma que:

  – **meses 1, 3, 5, 7, 8, 10 e 12**: deverá aceitar até no máximo o dia 31;

  – **meses 4, 6, 9 e 11**: deverá aceitar até no máximo o dia 30;

  – **mês 2**: deverá aceitar até o dia 28 (para qualquer ano) e até o valor 29 para os anos bissextos (para isso, deve-se verificar o valor do atributo bissexto).

- Criar o método alterarData(...) (público e sem retorno), que deve receber como parâmetro os valores de ano, mês e dia (nessa ordem) e deve incluir os valores aos respectivos atributos utilizando os métodos set já existentes na classe (com as devidas validações já programadas). Os valores devem ser incluídos nos atributos de acordo com a seguinte sequência: primeiro o ano, depois o mês e depois o dia, de forma que caso resulte alguma inconsistência de data, ela não deverá ser alterada.

- Criar um método construtor para a classe Data, que possui os parâmetros para receber os valores do ano, mês e dia (nessa ordem), e que deve colocar automaticamente os valores enviados como parâmetros nos respectivos atributos. Para isso, pode-se utilizar do método alterarData(...) já construído. Caso a data inserida seja inválida, o sistema deverá colocar como data 01/01/01.

- Criar um método getData, que retorna o valor da data como uma string no formato dd/mm/aaaa.

Resolução

1)

```
public class Produto {
    private int codigo;
    private String nome;
    private int qtdEstoque;
    public Produto (int cod, String nom, int qtd) {
      codigo = cod;
      nome = nom;
      qtdEstoque = qtd;
    }
    public void mostrarDados() {
```

```
        String txt = "Código: " + codigo + "\n";
        txt += "Nome: " + nome + "\n";
        txt += "Quantidade em Estoque: " + qtdEstoque;
        System.out.println(txt);
    }


// Métodos Setters e Getters do Encapsulamento
    public void setCodigo(int codigo) {
        this.codigo = codigo;
    }
    public int getCodigo() {
        return codigo;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public String getNome() {
        return nome;
    }
    public void setQtdEstoque(int qtdEstoque) {
        this.qtdEstoque = qtdEstoque;
    }
    public int getQtdEstoque() {
        return qtdEstoque;
    }
}
```

2)

```
public class Alimento extends Produto {
    private double peso;  // .. (em gr - gramas)
    public Alimento (int cod, String nom, int qtd, double p) {
        super(int cod, String nom, int qtd);
        peso = p;
    }
    public void mostrarDados() {
        super.mostraDados();
        String txt = "Peso: " + peso + "(gr)";
        System.out.println(txt);
    }
    // Métodos Setters e Getters do Encapsulamento
    public void setPeso(double peso) {
        this.peso = peso;
    }
    public double getPeso() {
        return peso;
    }
}
```

3)

```
public class Bebida extends Produto {
    private double volume;  // .. (em ml - mililitro)
```

```
    public Bebida (int cod, String nom, int qtd, double v) {
      super(int cod, String nom, int qtd);
      volume = v;
    }
    public void mostrarDados() {
      super.mostraDados();
      String txt = "Volume: " + volume + "(ml)";
      System.out.println(txt);
    }
    // Métodos Setters e Getters do Encapsulamento
    public void setVolume(double volume) {
      this.volume = volume;
    }
    public double getVolume() {
      return volume;
    }
}
```

4)

```
public class TesteProduto {
    public static void main (String[] args) {
      Alimento a1 = new Alimento(132, "Biscoito", 200, 180);
      Bebida b1 = new Bebida(5141, "Suco", 500, 90);
      a1.mostrarDados();
      b1.mostrarDados();
    }
}
```

5)

Existem várias soluções para esse exercício. Segue uma possível solução:

```
public class Data {
    private int dia;
    private int mes;
    private int ano;
    private boolean bissexto;
    // Método Construtor
    public Data(int d, int m, int a) {
      dia = 1;
      mes = 1;
      ano = 1;
      alterarData(d, m, a);
    }
    // Método alterarData
    public void alterarData(int d, int m, int a) {
      int d0 = dia;
      int m0 = mes;
      int a0 = ano;
      boolean x = false;
      x = setAno(a);
      if (x) x = setMes(m);
      if (x) x = setDia(d);
```

```
    if (!x) {
      dia = d0;
      mes = m0;
      ano = a0;
    }
  }
  // Métodos Setters e Getters
  public int getDia() {
    return dia;
  }
  private boolean setDia(int dia) {
    boolean res = false;
    if (dia >= 1) {
      if (mes == 2) {
        if (bissexto && dia <= 29) res = true;
        else if (dia <= 28) res = true;
      } else if (mes==4 || mes==6 || mes==9 || mes==11) {
        if (dia <= 30) res = true;
      } else {
        if (dia <= 31) res = true;
      }
    }
    if (res) this.dia = dia;
    eles System.out.println("Data Inválida!");
    return res;
  }
  public int getMes() {
    return mes;
  }
  private boolean setMes(int mes) {
    boolean res = false;
    if (mes >= 1 && mes <= 12) {
      res = true;
      this.mes = mes;
    } else {
      System.out.println("Data Inválida!");
    }
    return res;
  }
  public int getAno() {
    return ano;
  }
  private boolean setAno(int ano) {
    boolean res = false;
    bissexto = false;
    if (ano >= 1) {
      res = true;
      this.ano = ano;
      if (ano % 4 == 0) {
        bissexto = true;
        System.out.println("Ano Bissexto");
      }
    } else {
      System.out.println("Data Inválida!");
    }
    return res;
```

```
    }
    public String getData() {
      String res = "";
      String sData = "%02d/%02d/%04d";
      res += sData.format(sData, dia, mes, ano);
      return res;
    }
}
```

Resumo

Nesta unidade, vimos inicialmente que o sistema permite que misturemos tipos numéricos em fórmulas matemáticas, segundo suas equações. No entanto, dependendo do tipo da variável que receberá o valor, o sistema resolverá ou não automaticamente o tipo do valor numérico utilizado. Quando o tipo da variável receptora tem uma abrangência maior, o sistema (compilador) se encarrega automaticamente de promover ao tipo maior os valores de abrangência menor, fazendo o que chamamos de promotion. Quando o tipo da variável receptora tem uma abrangência menor, o sistema (compilador) exige que o programador indique para qual tipo os valores das variáveis de abrangência maior devem ser alterados, e esse programador deve fazer o que chamamos de casting.

```
int f = 4;
long g = 5 + f; //promotion do valor de f (que é int) para long
int h = 7 + (int)g; //casting de g (que é long) para int
```

Podemos controlar o acesso (em relação à programação) a determinados elementos (classe, atributo ou método). Em Java, trabalhamos com quatro modificadores de acesso: public, private, protected ou (default). Assim, um elemento public pode ser acessado por qualquer classe. Por sua vez, um elemento private só pode ser acessado por métodos da própria classe. Já um elemento protected pode ser acessado por classes herdeiras (subclasses) ou pelas classes do mesmo pacote. Por fim, um elemento (default) pode ser acessado apenas pelas classes do mesmo pacote.

O encapsulamento é um recurso em que restringimos o acesso (via código) aos atributos de uma classe, colocando para eles o modificador de acesso private, de modo a terem o acesso controlado por métodos da própria classe: os métodos setters e getters, cuja sintaxe geral é:

```
public void setAtributo (tipoAtributo variavel) {
     atributo = variavel;
}
public tipoAtributo getAtributo () {
     return atributo;
}
```

Uma classe não pode conter mais de um método que tenha a mesma assinatura (nome_do_método + parâmetros). No entanto, podemos gerar numa mesma classe diversos métodos que possuam o mesmo nome, desde que eles estejam diferenciados nos seus parâmetros (seja no tipo dos parâmetros, seja na quantidade deles). Quando uma classe possui mais de

um método com o mesmo nome, mas diferentes parâmetros, dizemos que nela há uma sobrecarga de métodos (overloading).

Toda instância é gerada a partir de um método construtor. Quando geramos um objeto em memória, o compilador se utiliza do método construtor daquela classe para criar a instância, de modo que se aquele não existir explicitamente no código, o compilador se utilizará de um método construtor padrão para isso. Desse modo, o método construtor é o método que gera na memória a instância de uma classe. Criamos um ou mais métodos construtores numa classe para que, ao instanciarmos aquela classe, o sistema possa automaticamente iniciar processos ou inicializar atributos com informações necessárias para o funcionamento do sistema. Se uma classe possui explicitamente um ou mais métodos construtores, ao gerarmos suas instâncias somos obrigados a utilizá-los na codificação. São características de métodos construtores:

- possuir exatamente o mesmo nome da classe;
- não possuir a definição de tipo de retorno;
- ser público.

Sua sintaxe é:

```
public NomeDaClasse (parâmetros) {
     ...
}
```

A herança é uma característica da orientação a objetos que permite que uma classe adquira as características (atributos e métodos) de outra classe, de forma que a classe que **herda** é chamada de subclasse (ou classe filha) e a classe que **é herdada** é chamada de superclasse (ou classe mãe). O termo que indica que uma classe está herdando outra é o termo extends, e sua declaração deve ser da seguinte forma:

```
public class Subclasse extends Superclasse { ... }
```

Uma classe que herda uma outra passa a "ser" essa classe, e ela possuirá todos os atributos e métodos da superclasse, independentemente do modificador de cada um deles. Na linguagem Java não há herança múltipla, mas sim encadeamento de heranças, de forma que ao herdar uma classe, a subclasse herdará todas as superclasses de sua superclasse. Esse recurso (herança) remete ao conceito de generalização e especificação, de modo que as subclasses serão classes mais específicas e a superclasse terá características mais genéricas e abrangentes.

Por conta da especificidade das subclasses, é comum termos uma situação em que alguns métodos herdados da superclasse precisam estar mais adequados às suas características. Assim, podemos recriar esses métodos na subclasse (com a mesma assinatura), de forma que, caso sejam executados, se executará aquele da subclasse, e não o da superclasse. Essa característica, que é a existência de métodos com a mesma assinatura na superclasse e na subclasse, é chamada de sobrescrita de métodos (ou overriding).

Quando precisamos trabalhar com arrays de objetos, é comum trabalharmos com as classes ArrayList ou Vector, que são classes próprias para essas situações. A forma de se trabalhar com essas classes é basicamente a mesma, diferenciando-se apenas nos casos em que temos a utilização de threads (em que se convém trabalhar com ArrayList). Sua declaração é como segue:

```
ArrayList<UmaClasse> objArrayList = new ArrayList();
```

Ou:

```
Vector<UmaClasse> objVector = new Vector();
```

Em ambos os casos, podemos adicionar nesse objeto, a partir do método add, outros objetos criados a partir da classe neles definida (no caso, a classe UmaClasse):

```
objArrayList.add(objUmaClasse);
```

Ou:

```
objVector.add(objUmaClasse);
```

A vantagem de utilizar este tipo de array de objetos é não precisarmos guardar o objeto de origem, cujo endereço de memória foi adicionado ao array.

Também vimos nesta unidade alguns métodos da classe String, que são úteis quando vamos trabalhar com mineração de dados, classificação de informações, busca de textos, entre outras utilidades. Dessa classe, vimos como utilizar cada um dos seguintes métodos: length(), toUpperCase(), toLowerCase(), charAt(...), trim(), indexOf(...), replace(...), replaceAll(...), substring(...), equals(...), format(...), split(...) e toCharArray().

Exercícios

Questão 1. (FCC 2019, adaptada) Um analista de TI está programando em Java e deseja relacionar a classe AcompanhaManancial de tal maneira que ela herde tudo que a classe Manancial tem, criando uma relação de superclasse e subclasse. Isso é conseguido em Java usando, inicialmente:

A) @Override public class Manancial { public class AcompanhaManancial {

B) public class AcompanhaManancial extends Manancial {

C) public class Manancial { @Override public class AcompanhaManancial {

D) protected interface AcompanhaManancial extends Manancial {

E) protected class Manancial includes AcompanhaManancial {

Resposta correta: alternativa B.

Análise da questão

De maneira geral, uma classe pode ser entendida como um "modelo" destinado à geração de objetos. A sintaxe básica de declaração de uma classe em Java é demonstrada a seguir.

```
public class NomeDaClasse {
  //...
}
```

A herança, considerada um dos pilares da orientação a objetos, é um recurso que permite a criação de novas classes a partir das características de outra classe já criada. Há, portanto, um reaproveitamento de código, de modo que uma classe filha (ou subclasse) herdará os métodos e os atributos da classe mãe (ou superclasse). Para isso, deve-se inserir, na declaração da classe filha, a palavra extends, seguida do nome da classe mãe.

Pelo enunciado, sabemos que AcompanhaManancial é a classe filha, que herda as características da Manancial, que é a classe mãe. Portanto, para declarar a classe filha realizando a herança proposta, devemos usar, inicialmente, o trecho:

```
public class AcompanhaManancial extends Manancial {
```

Questão 2. (Ibade 2019, adaptada) Os modificadores de acesso são padrões de visibilidade de acessos às classes, atributos e métodos. Esses modificadores são palavras-chave reservadas pelo Java, ou seja, não podem ser usadas como nome de métodos, classes ou atributos. Os modificadores de acesso são classificados conforme as descrições a seguir:

I – Indica que método ou variável só pode ser acessado de dentro da classe que os criou. Uma classe que herde de uma superclasse com atributos declarados de acordo com esse modificador só poderá ter acesso a eles por meio dos métodos públicos da própria superclasse, caso contrário, não haverá acesso a esses atributos.

II – Indica que o método ou a variável assim declarada pode ser acessado somente dentro do pacote em que está contido, ou por meio de uma subclasse.

III – Indica que a classe, o método ou a variável assim declarada pode ser acessado em qualquer lugar e a qualquer momento da execução do programa.

Os modificadores de acesso descritos como I, II e III são denominados, respectivamente:

A) Protected, public e static.

B) Dynamic, private e protected.

C) Public, static e dynamic.

D) Private, protected e public.

E) Static, dynamic e private.

Resposta correta: alternativa D.

Análise da questão

Conforme estudado, um sistema gerado em linguagem Java é baseado na programação de classes. Uma classe possui, basicamente, atributos (dados) e métodos (ações). Pode-se organizar o sistema em pacotes e, em cada pacote, inserimos as classes pertencentes a determinada categoria.

Além disso, podemos tornar componentes da nossa aplicação mais ou menos acessíveis por outras partes do programa. Para controlar os acessos, estão disponíveis quatro modificadores básicos de acesso. Eles estão elencados a seguir, do menos restritivo ao mais restritivo.

- **public**: é o modificador de acesso menos restritivo dos quatro. Uma declaração (classe, atributo ou método) feita com esse modificador indica que o elemento pode ser acessado de qualquer lugar e por qualquer objeto que possa visualizar a classe a que esse elemento pertence. Dessa forma, a descrição 3 do enunciado da questão diz respeito a esse modificador.

- **protected**: torna um elemento (atributo ou método) acessível apenas às classes do mesmo pacote, ou a classes relacionadas por herança. A descrição 2 corresponde e esse modificador.

- **default** (padrão, ou sem modificador): uma declaração (classe, atributo ou método) feita sem um modificador de acesso explícito assume o modificador padrão. Nesse caso, o elemento só pode ser acessado por outras classes definidas do mesmo pacote. O modificador default é mais restritivo do que o public e o protected, mas é menos restritivo do que o private. O modificador default nem mesmo aparece no texto das alternativas da questão; logo, nenhuma das descrições do enunciado corresponde a ele.

- **private**: é o modificador de acesso mais restritivo. Indica que o elemento (atributo ou método) não pode ser acessado ou utilizado por nenhuma outra classe, a não ser apenas por métodos da própria classe a que pertence. A descrição 1 corresponde a esse modificador.

Unidade III

5 POLIMORFISMO

Considerado outro pilar da orientação a objetos, o polimorfismo é uma característica que define que alguns elementos da linguagem orientada a objetos (como suas variáveis, os atributos, os métodos e as classes) podem ter formas diferentes de comportamentos.

Para o Java, o polimorfismo é a capacidade de um determinado elemento se comportar de formas diferentes, dependendo de como ele é declarado, ou de como está sendo utilizado, a partir da sua codificação.

5.1 Polimorfismo de variáveis

Em Java, algumas variáveis (como os atributos) têm "alcance global" na classe (variáveis globais, ou seja, aquelas que podem ser utilizadas por qualquer método da classe) e outras são geradas e declaradas em blocos, podendo ser utilizadas apenas naquele bloco.

No entanto, é possível termos duas variáveis com mesmo nome num mesmo bloco, mas com comportamentos diferentes (global e local). Como exemplo deste caso, é muito comum encontrarmos essa situação nos encapsulamentos, quando nos métodos set... temos o parâmetro com o mesmo nome do atributo. Neste caso, temos duas variáveis com o mesmo nome, mas com comportamentos diferentes (uma sendo o atributo da classe e a outra sendo a variável local, definida no parâmetro).

Para diferenciarmos as duas, colocamos juntamente à variável que representa o atributo a palavra this., como no exemplo a seguir:

```
public class Carro {
    public double velocidade;
    //... e demais atributos
    public void setVelocidade (double velocidade) {
      this.velocidade = velocidade;
    }
    //... e demais métodos
}
```

Observação

Na IDE do Eclipse é possível gerar automaticamente as classes setters e getters. Seus métodos setters são definidos como o exemplo que acabamos de apresentar.

Percebe-se nesse exemplo que o método setVelocidade(...) recebe como parâmetro um valor numérico do tipo double, e que o parâmetro será recebido pela **variável local** velocidade. Esse método insere o valor, recebido pelo seu parâmetro, no **atributo** velocidade. Assim, para diferenciar as varáveis (atributo velocidade da variável local velocidade), utilizamos a palavra reservada this, de forma que:

- **this.velocidade**: equivale ao atributo velocidade da classe;
- **velocidade**: equivale à variável local do método.

Ou seja, são duas formas diferentes de variáveis que possuem o mesmo nome existentes num mesmo bloco (o bloco do método setVelocidade(...)).

5.2 Polimorfismo de métodos

Vimos anteriormente que uma mesma classe não aceita métodos que possuam a mesma assinatura entre si. No entanto, vimos também (em itens anteriores) que existem duas situações em que há semelhanças de assinaturas entre métodos, no que diz respeito ao seu nome, representando as duas formas de polimorfismo de métodos. São elas:

- **Sobrecarga de métodos (ou overloading)**: a presença, numa mesma classe ou em classes com herança, de dois ou mais métodos com o mesmo nome e diferenciados nos parâmetros.
- **Sobrescrita de métodos (ou overiding)**: a presença, neste caso apenas em classes com herança, de dois ou mais métodos com a mesma assinatura (mesmo nome e mesmos parâmetros), um na superclasse e outro na subclasse.

5.3 Polimorfismo de classes

O polimorfismo de classes é um tipo de polimorfismo em que uma classe se comporta como se fosse outra classe. Isso só é possível quando há herança de classes, em que uma superclasse se comporta como uma subclasse.

Sabemos que uma classe (classe mãe, ou superclasse) pode ser herdada por diversas outras classes (classes filhas, ou subclasses). Dessa forma, podemos fazer com que um objeto representando uma classe mãe receba (ou seja instanciado com) a instância de uma de suas classes filhas, caracterizando assim o polimorfismo de classe.

Exemplo:

```
public class Carro {
    private String tipoClasse = "Carro";
    //... e demais atributos
    public String getTipoClasse () {
      return tipoClasse;
    }
    public void mostraDados(Carro crr) {
      System.out.println(crr.getTipoClasse());
    }
}
public class Fusca extends Carro {
    private String tipoClasse = "Fusca";
    //... e demais atributos
    public String getTipoClasse () {
      return tipoClasse;
    }
}
public class Ferrari extends Carro {
    private String tipoClasse = "Ferrari";
    //... e demais atributos
    public String getTipoClasse () {
      return tipoClasse;
    }
}
```

Vejamos então a ocorrência do polimorfismo de classe:

```
public class Teste {
    public static void main(String[] args) {
      Carro c1 = new Carro();
      Fusca fu = new Fusca();
      Ferrari fe = new Ferrari();
      c1.mostraDados(c1);
      c1.mostraDados(fu);
      c1.mostraDados(fe);
      /*
       * imprimirá:
       *   Carro
       *   Fusca
       *   Ferrari
       */
    }
}
```

Neste exemplo, o método mostraDados(...) da classe Carro recebe como parâmetro um objeto do tipo Carro. Porém na classe Teste geramos três instâncias, uma instância para cada uma das classes (Carro, Fusca e Ferrari). Com essas instâncias, chamamos o método mostraDados(...) da classe Carro (que recebe a classe Carro como parâmetro), e enviamos cada uma daquelas instâncias por vez, observando que a saída é diferente para cada uma delas. Isso mostra que o objeto crr do parâmetro daquele método se comporta de formas diferentes para cada um dos objetos enviados.

Exemplo de aplicação

Um dos pilares da orientação a objetos é a possível existência de polimorfismo entre os seus elementos, de forma que a linguagem permite o polimorfismo de métodos e de classes.

De acordo com o conceito de polimorfismo, qual dos itens a seguir está **incorreto**?

A) A sobrecarga é um tipo de polimorfismo de métodos em que, numa mesma classe ou entre superclasses e suas subclasses, temos a presença de métodos com o mesmo nome, mas com parâmetros diferentes na mesma classe ou entre superclasses e subclasses.

B) A sobrescrita é um tipo de polimorfismo de métodos em que há ocorrência de métodos com mesma assinatura e, portanto, só pode ocorrer entre superclasses e subclasses.

C) Quando um método tem como parâmetro um ou mais objetos que representam classes consideradas superclasses, esses parâmetros podem ser "preenchidos" (na chamada ao método) com objetos que representam outras classes, desde que elas sejam subclasses daquela representada pelo parâmetro.

D) O polimorfismo é uma característica de OO em que um mesmo elemento pode ser apresentado, ou utilizado, de formas diferentes, dependendo de condições específicas.

E) O polimorfismo é um recurso que permite que uma subclasse se utilize de qualquer método privado de uma superclasse, sem que seja preciso instanciá-la.

Resposta correta: alternativa E.

Resolução

A) Alternativa correta.

Justificativa: a sobrecarga, também conhecida como overloading, é observada quando temos ou numa mesma classe, ou em classe que caracteriza herança, mais de um método com o mesmo nome, mas com parâmetros diferenciados. Criamos esses métodos para que o conceito envolvido pela ação do método possa ser utilizado com tipos ou quantidades de informações diferentes.

B) Alternativa correta.

Justificativa: a sobrescrita, também conhecida como overiding, pode ser observada quando um método de uma classe filha (ou subclasse) possui a mesma assinatura da classe mãe (ou superclasse). Assim, quando instanciamos a classe filha, sabemos que ela contém todos os métodos da classe mãe, os quais podem ser acionados com aquele objeto. No entanto, se acionarmos o método que sofreu sobrescrita, será o método da classe filha que estaremos acionando, pois ele substitui o da classe mãe.

C) Alternativa correta.

Justificativa: o polimorfismo de classe é bastante observado em parâmetros de métodos quando esses parâmetros representam classes que possuem subclasses, de modo que esses objetos (os parâmetros) podem se comportar como um objeto que foi instanciado com qualquer uma de suas subclasses.

D) Alternativa correta.

Justificativa: a ideia do polimorfismo é a de apresentar formas diversas de se trabalhar com um mesmo elemento. Sua observância é mais simples quando pensamos em polimorfismo de métodos, em que temos métodos que possuem o mesmo nome (e, portanto, uma mesma chamada), mas que atuam de formas diferentes dependendo da necessidade do programa, dependendo dos valores enviados nos parâmetros.

E) Alternativa incorreta.

Justificativa: um método privado não pode ser utilizado (chamado) diretamente por nenhum método que não seja da própria classe. Isso não quer dizer que uma subclasse não o contenha, já que ela herda todos os métodos da superclasse. A subclasse apenas não poderá acionar diretamente esse método privado, mas poderá indiretamente, ao acionar outro método na superclasse que se utiliza daquele método privado. O polimorfismo, como se pode observar nas explicações que acabamos de oferecer, contém um conceito que não está relacionado a esse tipo de recurso.

Saiba mais

Saiba mais sobre polimorfismo no capítulo 7, item 7.13, de:

FURGERI, S. *Java 7*: ensino didático. 2. ed. São Paulo: Érica, 2012.

6 MODIFICADORES DE COMPORTAMENTO E ALGUMAS CLASSES ÚTEIS

Os modificadores de comportamento são aqueles que alteram a forma de se trabalhar com os elementos e, consequentemente, o comportamento do elemento na funcionalidade do sistema.

Veremos que esses modificadores são utilizados com o intuito de controlar a forma de se programar o sistema, ou de como outros sistemas podem se utilizar dele. Esses modificadores são recursos que podem definir como a equipe de programadores deverá lidar com cada elemento do sistema.

6.1 Modificadores de comportamento

Esses modificadores são "somados" aos modificadores de acesso e alteram o comportamento dos elementos (classes, métodos e/ou atributos).

Lembrete

Modificadores de acesso (public, private, protected e (default)) são os termos iniciais das declarações de atributos e métodos, que controlam quais classes podem ou não acessá-los.

Tem-se os seguintes modificadores de comportamento:

- static;
- final;
- abstract.

Sua sintaxe, na declaração dos elementos, deve seguir a seguinte sequência:

modifAcesso modifComport declaracaoDoElemento

Exemplo:

```
public abstract class ClasseA {...}
public final void metodo01() {...}
```

Observação

Perceba que primeiro se declara o modificador de acesso e em seguida o modificador de comportamento.

6.2 Modificador final

O modificador final pode ser utilizado com classes, atributos e/ou métodos, alterando o comportamento desses elementos segundo as seguintes regras:

Uma classe final é uma classe que não pode ser herdada

Isso quer dizer que numa cadeia de heranças, uma classe final será a última classe da cadeia, já que não poderá ser herdada por outras classes.

Exemplo:

```
public final class ClasseA {
    //.. seus atributos e métodos
}
```

Isso quer dizer que se tentarmos criar outra classe que herda a classe ClasseA, o compilador acusará um erro de programação:

Exemplo:

```
// ### ERRO !!
public class ClasseB extends ClasseA {...}
//.. já que uma Classe final NÃO pode ser herdada ####
```

Observação

Esse tipo de erro (a ClasseB herdando a ClasseA) impede que o programa seja compilado (gera um erro de compilação), impossibilitando seu acionamento.

Um método final é um método que não pode ser sobrescrito

Quando uma classe herda outra, a primeira passa a possuir todos os métodos da classe herdada. O programador pode querer sobrescrever algum método da superclasse (recriá-lo na subclasse com a mesma assinatura), por conta de alguma característica específica daquela subclasse. No entanto, se o método da superclasse for um método final, essa substituição não pode ser realizada.

Exemplo:

```
public class ClasseA {
    public final void metodo01() {
       // ...
    }
    //.. seus atributos e demais métodos
}
```

```
public class ClasseB extends ClasseA {
    //.. seus atributos e métodos
    // ### ERRO !!
    public void metodo01() {...}
    //.. já que um método final NÃO pode ser sobrescrito ###
}
```

Observação

Esse tipo de erro (o metodo01(...) sendo sobrescrito) acentuado no exemplo impede que o programa seja compilado (gera um erro de compilação), impossibilitando seu acionamento.

Um atributo final é um atributo que não pode ser alterado (ou seja, é uma constante)

Normalmente, o atributo de uma classe é uma variável que adquire algum valor durante a sua existência em memória (na instância da classe) e que também pode ter seu valor alterado. No entanto, algumas vezes necessitamos estabelecer alguns valores que não podem ser alterados ao longo do sistema (valores como taxas, índices indicativos de uma empresa, elementos repetitivos de programação etc.). Nesse caso, trabalha-se com constantes identificadas como atributos finais. Pela convenção de programação, um atributo final (uma constante) deve ser descrito em caixa alta, ou seja, com todas as letras em maiúscula.

Exemplo:

```
public class CalculoMatematico {
    public final double PI = 3.1415926;
    //.. seus atributos e demais métodos
}
```

Ou, ainda:

```
public class Financeiro {
    public final double TAXA_IMPOSTO = 0.00175;
    //.. seus atributos e demais métodos
}
```

Neste caso, qualquer tentativa de alteração do valor dessa constante PI (pertencente à primeira classe) ou à constante TAXA_IMPOSTO (pertencente à segunda classe) que se faça ao longo do sistema (de uma forma geral) gerará um erro de compilação, impedindo que a classe seja compilada.

6.3 Modificador static

O modificador static pode ser utilizado com atributos e/ou métodos, de modo que todo elemento estático (static) possa ser acessado sem a necessidade de se instanciar a classe a que ele pertence.

Veja o exemplo a seguir:

```
public class Calculo {
    public double PI = 3.1415926;
    //.. e demais atributos
    public double calcularValor(double a, double b) {
      double resultado = 0;
      // processamento do valor - cálculo com "a" e com "b"
      return resultado;
    }
    //.. e demais métodos
}
```

No exemplo, se os métodos não forem estáticos, para se utilizar o atributo e o método da classe Calculo, deve-se antes instanciar a classe:

```
public class Teste {
    public static void main (String[] args) {
      // instanciando a Classe Calculo
      // ...para poder utilizar seus elementos:
      Calculo c = new Calculo();
      double x = 72.31 * c.PI;
      double y = c.calcularValor(1.75, 5.48);
      System.out.println(x + " :: " + y);
      //.. imprimirá: 227.168560906 :: 0.0
    }
}
```

Mas se os elementos da classe Calculo forem estáticos, para utilizarmos esses elementos na classe Teste não se faz necessária a instância daquela classe, e a chamada é feita diretamente a partir da classe, e não por uma instância da classe:

```
public class Calculo {
    public static double PI = 3.1415926;
    //.. e demais atributos
    public static double calcularValor(double a, double b) {
      double resultado = 0;
      // processamento do valor - cálculo com "a" e com "b"
      return resultado;
    }
    //.. e demais métodos
}
public class Teste {
    public static void main (String[] args) {
      // Utilizando os elementos sem instanciar a Classe
      double x = 72.31 * Calculo.PI;
```

```
        double y = Calculo.calcularValor(1.75, 5.48);
        System.out.println(x + " :: " + y);
        //.. imprimirá: 227.168560906 :: 0.0
    }
}
```

6.4 Modificador abstract

O modificador abstract pode ser utilizado com classes e/ou métodos, alterando o comportamento desses elementos segundo as seguintes regras:

Uma classe abstract é uma classe que não pode ser instanciada

Existem algumas razões para que possamos querer que uma classe não seja instanciada. Duas delas são as seguintes:

- quando criamos uma classe que conterá uma série de métodos estáticos e/ou atributos finais e estáticos, de forma que seus elementos (atributos e métodos) deverão ser acessados diretamente sem a instância da classe;
- quando criamos uma classe com elementos genéricos (uma classe mãe, ou superclasse), da qual geramos várias subclasses (herdando aquela classe mãe), e desejamos que a superclasse não seja instanciada, para ser utilizada somente a partir de suas classes filhas (instanciando-as).

Exemplo:

```
public abstract class ClasseA {
    // seus atributos
    public void metodo01() {
        // processamento do método
    }
    //.. e demais métodos
}
public class ClasseB extends ClasseA {
    // seus atributos e métodos
}

public class Teste {
    public static void main (String[] args) {
        // Instanciando a ClasseB,
        // ..já que a ClasseA não pode ser instanciada.
        ClasseB cb = new ClasseB ();
        cb.metodo01(); // método herdado da ClasseA
    }
}
```

Um método abstract é um método que não possui implementação, mas que exige a sua implementação na herança

Características importantes dos métodos abstratos:

- métodos abstratos só podem existir em classes abstratas;
- métodos abstratos não possuem implementação (percebam que métodos abstratos terminam com ponto-e-vírgula):

```
public abstract tipoRetorno nomeMetodo(parâmetros);
```

- os métodos abstratos necessitam obrigatoriamente ser implementados nas subclasses;
- caso tenhamos uma classe abstrata que herdou outra classe abstrata, os métodos abstratos desta última não precisam ser implementados na subclasse que a herdou, já que ela deverá ser herdada por outra classe.

Observação

Sabemos que um método abstrato só pode existir em uma classe abstrata, mas uma classe abstrata não precisa ter métodos abstratos, podendo ter tanto métodos comuns como também métodos abstratos – lembrando que uma classe abstrata é simplesmente uma classe que não pode ser instanciada.

Exemplo: imagine que estamos criando um jogo de corrida de carros no qual geramos as classes que representarão cada um dos carros que competirão juntamente na pista de corrida. Neste caso, geramos uma classe mais genérica (a classe Carro), que conterá as características que todos os tipos de carros de corrida terão no jogo. Essa classe, porém, não será uma classe instanciada diretamente, já que o jogador deverá poder escolher uma dentre as diversas opções de tipos de carro.

Na classe Carro (que será a superclasse com as características mais gerais que todos os carros terão no jogo), inserimos dois métodos abstratos (frear e acelerar), para que os programadores se lembrem de implementá-los em cada um dos tipos de carro que criarem para o jogo.

```
public abstract class Carro {
    // :: métodos abstratos
    public abstract void frear();
    public abstract void acelerar();
    // :: métodos implementados
    public void virar(String tipoVirada) {
       // :: construção da lógica
       if (tipoVirada.equals("direita")) {
          // .. lógica de curva para a direita
       }
       // ...
    }
    // .. e demais métodos
}
```

No exemplo, como a classe Carro é abstrata, não poderá ser instanciada. Isso quer dizer que outras classes irão herdá-la, especificando sua atuação no sistema. Nesse caso, os desenvolvedores que criarem as suas subclasses, ou seja, as classes que herdarem a classe Carro, deverão implementar os seus métodos abstratos. A preocupação em se fazer isso está em especificar cada uma dessas ações em cada uma das classes Filhas (como nas classes Fusca e Ferrari), de modo a obrigar que a cada uma dessas classes seja dado um comportamento específico dentro desses métodos.

```
public class Fusca extends Carro {
    // :: Implementação obrigatória dos métodos abstratos
    // .. existentes na Classe Carro herdada
    public void frear() {
       // .. lógica do método frear para o Fusca
    }
    public void acelerar() {
       // .. lógica do método acelerar para o Fusca
    }
    // .. e demais métodos
}
```

Observação

A IDE do Eclipse possui uma facilidade (um wizard) que faz com que de dentro da classe Fusca o desenvolvedor consiga puxar (gerar na classe) todos os métodos abstratos das classes herdadas. Clicando com o botão esquerdo do mouse sobre a classe Fusca e abrindo o menu Source – Override/Implement Methods..., abrirá uma tela na qual os métodos abstratos das classes herdadas já estarão selecionados, bastando clicar sobre o botão OK desta tela, o qual irá gerar de forma padrão (com retornos padrões, quando necessário) os métodos necessários (com exatamente a mesma declaração definida no método abstrato, ou seja, mesmo tipo de retorno e mesmos parâmetros).

6.5 Tabela-resumo sobre os modificadores de comportamento

O quadro a seguir resume todos os conceitos que acabamos de definir referentes aos modificadores de comportamento:

Quadro 2 – Resumo sobre os modificadores de comportamento

	static	final	abstract
Classe	(Não existe)	Não pode ser herdada por outra classe	Não pode ser instanciada (deverá ser utilizada a partir de suas classes filhas)
Método	Pode ser **acionado** sem precisar instanciar a classe a que pertence	Não pode ser sobrescrito (não pode ser substituído por outro método em uma subclasse)	É um método sem implementação (só com a sua declaração) Ele deve ser implementado em uma classe filha Somente pode existir em uma classe abstrata
Atributo	Pode ser **lido** sem precisar instanciar a classe a que pertence (será um valor constante)	É constante (seu nome deve estar em caixa alta)	(Não existe)

Exemplo de aplicação

Quanto ao conceito de modificadores de comportamento, analise as afirmativas a seguir:

I – Uma classe estática é aquela destinada a conter somente constantes, a fim de não sofrer alterações nos valores de seus atributos.

II – O modificador final, de certa forma, é um modificador que proíbe ações com o elemento por ele caracterizado, de modo que uma classe final não pode ser herdada, um método final não pode ser sobrescrito e um atributo final não pode ter seu valor alterado.

III – Criam-se métodos abstratos para que, quando um desenvolvedor gerar uma subclasse daquela classe a que o método pertence, esses métodos sejam criados e tenham sua lógica devidamente desenvolvida, geralmente para suprir uma necessidade imposta por uma regra de negócio do sistema.

IV – Uma classe abstrata é uma classe que poderá ser utilizada de forma indireta, ou para servir de repositório de métodos estáticos, ou para servir de base e complementar a subclasse que a herda.

De acordo com as afirmativas, podemos dizer que estão corretas:

A) Apenas as afirmativas II, III e IV.

B) Apenas as afirmativas I, II e III.

C) Apenas as afirmativas I e IV.

D) Apenas as afirmativas II e IV.

E) Apenas as afirmativas I, III e IV.

Resposta correta: alternativa A.

Resolução

I – Afirmativa incorreta.

Justificativa: não se pode criar uma classe estática. Esse modificador somente é utilizado para atributos e métodos, de modo a fazer com que eles possam ser acessados sem a necessidade de se instanciar a classe a que pertencem.

II – Afirmativa correta.

Justificativa: o modificador final pode ser utilizado com classes, métodos e atributos, gerando respectivamente classes que não aceitam herança, métodos que não podem ser substituídos por outros em subclasses, impedindo o que conhecemos por overiding, e atributos considerados constantes e que, portanto, não podem ter seu valor alterado.

III – Afirmativa correta.

Justificativa: um método abstrato, obrigatoriamente construído em uma classe abstrata (que conceitualmente não pode ser instanciada, mas pode ser herdada), obriga a sua implementação nas subclasses que herdam a classe a que pertencem. Dessa forma, cria-se um método abstrato numa classe quando um desenvolvedor pretende definir regras que devem ser seguidas nas classes que a herdarem.

IV – Afirmativa correta.

Justificativa: uma classe abstrata, que não pode ser instanciada, pode servir de repositório de métodos estáticos ou como base para a construção de subclasses que conterão todos os seus métodos e atributos.

6.6 Interface

Uma interface é uma entidade da orientação a objetos **equivalente a** uma classe totalmente abstrata. Ela não é considerada uma classe, mas, assim como uma classe, também pode conter atributos e métodos. No entanto, em uma interface, todo atributo será um atributo estático e final, e todo método será um método abstrato.

Lembrete

Atributos estáticos e finais são considerados constantes, e um método abstrato é aquele que não possui implementação e que deve ser implementado na herança.

Como todo atributo e todo método de uma interface tem características como as que acabamos de definir, seus modificadores de comportamento não são descritos na declaração desses elementos.

Exemplo:

```
public interface Inter01 {
    public int ATRB01 = 11;
    public double ATRB02 = 22.33;
    public void metodo01(int x);
    public int metodo02();
}
```

Observação

Como pertencem a uma interface, os atributos ATBR01 e ATRB02 são estáticos e finais, e os métodos metodo01 e metodo02 são abstratos, apesar de essas características não estarem explícitas na declaração desses elementos.

Uma interface é implementada por uma classe por meio da palavra-chave implements, ao final da declaração da classe.

```
public class ClasseA implements InterfaceA {
    ...
}
```

O principal objetivo de uma interface é obrigar as classes que a implementaram a implementarem os seus métodos.

Algumas características da interface:

- uma interface somente possuirá métodos abstratos e/ou atributos finais (constantes);
- o nome de uma interface segue o mesmo padrão de nomes de classes (começa com letra maiúscula);
- uma classe pode implementar múltiplas interfaces:

```
public class ClasA implements InterfA, InterfB, InterfC {
    ...
}
```

No caso de implementações múltiplas de interfaces, a classe deverá implementar todos os métodos de todas as interfaces que ela está implementando.

Observação

Os métodos de uma interface não levam o modificador abstract, apesar de serem abstract. Já os atributos de uma interface não levam o modificador final, apesar de serem final.

6.6.1 Por que utilizamos interfaces?

Utilizamos interfaces para servirem de guias na implementação das regras de negócio das empresas.

Quando uma empresa possui uma ou mais regras de negócio que devem ser propagadas às diversas representações de seus produtos na forma de sistemas de informática, ao construirmos um sistema

orientado a objeto, sempre que for criada uma classe que representa aqueles produtos, essas regras (métodos definidos na interface) deverão ser implementadas naquela classe. Isso, no mínimo, irá obrigar ao desenvolvedor conhecer (ou procurar conhecer) o produto no qual ele está trabalhando e suas particularidades, para depois então gerar suas representações no sistema.

Saiba mais

Saiba mais sobre interfaces no capítulo 8 de:

SCHILDT, H. *Java para iniciantes*: crie, compile e execute programas Java rapidamente. 6. ed. Porto Alegre: Bookman, 2015.

6.7 Classe JOptionPane

JOptionPane é uma classe da biblioteca do Java que possibilita a criação de uma caixa de diálogo padrão que ou solicita um valor para o usuário, ou retorna uma informação padrão, dependendo do método utilizado e da necessidade do desenvolvedor.

Quadro 3 – Métodos da classe JOptionPane

Métodos	Descrição
showConfirmDialog	Solicita uma confirmação como YES, NO, CANCEL
showInputDialog	Solicita algum valor (campo texto para preenchimento)
showMessageDialog	Mostra ao usuário uma informação
showOptionDialog	Unificação dos três tipos acima

Quadro 4 – Estilos de mensagens

Estilos da mensagem	Descrição
ERROR_MESSAGE	X num octógono
INFORMATION_MESSAGE	i num círculo
WARNING_MESSAGE	! num triângulo
QUESTION_MESSAGE	? num quadrado
PLAIN_MESSAGE	Sem ícone

Quadro 5 – Estilos de botões

Estilos do botão	Descrição
DEFAULT_OPTION	Botão OK
YES_NO_OPTION	Botões Sim e Não
YES_NO_CANCEL_OPTION	Botões Sim, Não e Cancelar
OK_CANCEL_OPTION	Botões OK e Cancelar

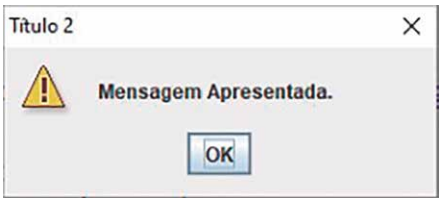
Quadro 6 – Tipos de respostas (ou retornos)

Tipos de respostas	Valor de retorno
`YES_OPTION` - (clicando em Sim)	0
`NO_OPTION` - (clicando em Não)	1
`CANCEL_OPTION` - (clicando em Cancelar)	2
`OK_OPTION` - (clicando em OK)	0
`CLOSED_OPTION` - (fechando a janela de mensagem, ou teclando em ESC, ou clicando no X que fecha a janela)	–1

Exemplos de utilização com sua respectiva visualização de tela de mensagem (observação: suponha JFrame frame = null;):

```
JOptionPane.showMessageDialog(frame,
     "Mensagem Apresentada.",
     "Título 1",
     JOptionPane.INFORMATION_MESSAGE);
```

Figura 24 – Tela de mensagem




```
JOptionPane.showMessageDialog(frame,
     "Mensagem Apresentada.",
     "Título 2",
     JOptionPane.WARNING_MESSAGE);
```

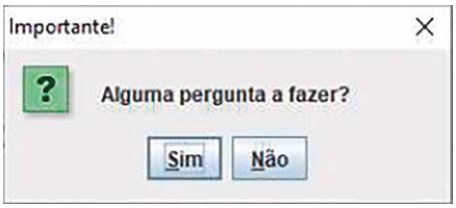
Figura 25 – Tela de mensagem

```
JOptionPane.showMessageDialog(frame,
     "Mensagem Apresentada.",
     "Título 3",
     JOptionPane.ERROR_MESSAGE);
```

Figura 26 – Tela de mensagem

```
JOptionPane.showMessageDialog(frame,
     "Mensagem Apresentada.",
     "Título 4",
     JOptionPane.PLAIN_MESSAGE);
```

Figura 27 – Tela de mensagem

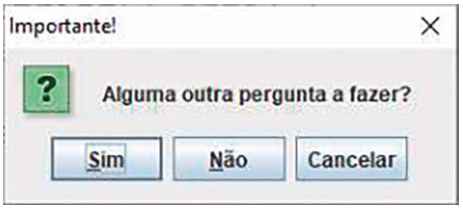



```
int n = JOptionPane.showConfirmDialog(
     frame,
     "Alguma pergunta a fazer?",
     "Importante!",
     JOptionPane.YES_NO_OPTION);
```

Figura 28 – Tela de mensagem

Observação

Clicando em Sim retorna 0, clicando em Não retorna 1.

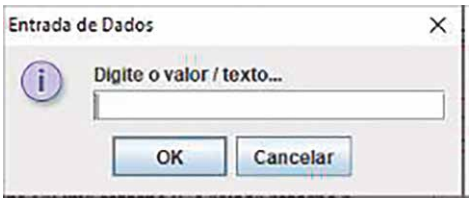



```
int n = JOptionPane.showConfirmDialog(
     frame,
     "Alguma outra pergunta a fazer?",
     "Importante!",
     JOptionPane.YES_NO_CANCEL_OPTION);
```

Figura 29 – Tela de mensagem

Observação

Clicando em Sim retorna 0, clicando em Não retorna 1, clicando em Cancelar retorna 2.




```
String s = JOptionPane.showInputDialog(
     frame,
     "Digite o valor / texto...",
     "Entrada de Dados",
     JOptionPane.INFORMATION_MESSAGE);
```

Figura 30 – Tela de mensagem

Nesse último exemplo, a string s receberá o texto digitado (independentemente do valor, se numérico ou alfanumérico). Se nada for digitado, ou apertar-se Cancelar, então s receberá nulo (null).

6.8 Wrapper classes

A palavra **wrapper** vem do inglês **wrap**, que significa **envolver**, e tem como principal função "envolver elementos" adicionando funcionalidades a eles.

O Java possui diversos wrappers que adicionam funcionalidades a outras classes, ou tipos primitivos. Há vários tipos, seja para tratar o fluxo de conexões, (ObjectInputStream), para fluxos de áudio (AudioInputStream), baseados em tipos primitivos (DataInputStream) ou para adicionar buffers (BufferedInputStream).

Com os wrappers de tipos primitivos consegue-se executar métodos como:

- parseTipo (utilizado na transformação de valores de strings);
- valueOf.

Quadro 7 – Relação entre os tipos primitivos e suas respectivas wrappers classes

Tipo primitivo	Wrapper class
boolean	Boolean
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double

6.9 Transformando textos em números: utilizando as wrapper classes

É possível transformar textos (valores do tipo string) em números. Neste caso, transforma-se o valor de uma string em um tipo primitivo numérico a partir dos métodos da sua respectiva wrapper class, desde que os valores das strings representem efetivamente um valor do tipo numérico que se quer recuperar, de forma que caso não seja possível, uma exceção será gerada.

Exemplo:

```
String s1, s2, s3;
s1 = "27";
int i = Integer.parseInt(s1);
long l = Long.parseLong(s1);
```

```
s2 = “27.98”;
float f = Float.parseFloat(s2);
double d = Double.parseDouble(s2);
//int i = Integer.parseInt(s2);
// este último geraria uma exceção pois o valor não é inteiro
s3 = “3956”;
char c = s3.charAt(2); // pega o valor 5
int val = Character.getNumericValue(c);
```

Observação

É importante ficar atento quando se vai realizar a transformação de informações do tipo string para tipos numéricos, já que caso o valor da informação do tipo string não represente um valor numérico, o compilador acusará um erro que poderá interromper a execução do programa, parando-o imediatamente se não for tratado (o tratamento de exceções é um tópico explicado mais a frente neste livro-texto).

Também devemos ficar atentos ao fato de que, se tentarmos transformar valores numéricos do tipo real (double ou float) para variáveis do tipo inteiro (int, long etc.), também será gerado um erro que interrompe a execução do programa.

Para finalizar esta unidade, os exemplos de aplicação a seguir. As resoluções são apresentadas na sequência.

Exemplos de aplicação

1) Criar a classe Pessoa (encapsulada) com as seguintes características:

Deve possuir os atributos:

- nome (String)
- endereco (String)

Deve possuir o método abstrato: cadastrarPessoa()

público e sem retorno

2) Criar as classes PessoaFisica e PessoaJuridica (encapsuladas) com as seguintes características:

- devem herdar a classe Pessoa;

- devem ser classes finais.

A classe PessoaFisica deve possuir os atributos:

- cpf (String);
- idade (int).

A classe PessoaJuridica deve possuir os atributos:

- cnpj (String);
- nomeFantasia (String).

O método de cadastro de cada uma delas deverá permitir a inclusão de um valor em cada um de seus atributos (via JOptionPane – com input box).

3) Criar uma classe abstrata de funções especiais com strings cujo nome deverá ser "StrEspecial".

Essa função deverá ter três métodos estáticos:

trimGeral(...)

Esta função deverá receber uma string e retornar a mesma string, porém tendo sido retirados os espaços iniciais e finais a mais (se houver) e todo e qualquer espaço a mais (extras) internamente à string.

Por exemplo, se receber a frase:

" Olá Mundo ! "

Deverá retornar:

"Olá Mundo !"

Perceba que os espaços iniciais e finais (que não deviam existir) foram totalmente retirados, e os espaços extras na parte interna se transformaram em espaço único.

trimToUpper(..)

Esta função deverá passar a string recebida pelo método anterior (trimGeral(...)) e depois transformar tudo em maiúsculo.

trimToLower(..)

Esta função deverá passar a string recebida pelo método anterior (trimGeral(...)) e depois transformar tudo em minúsculo.

Observação

Para trabalhar com a string, dê uma olhada no item "Trabalhando com strings" deste livro-texto.

Por fim, criar uma classe TesteStr com o método main, que deverá testar o texto a seguir:

" Olá Mundo ! "

E imprimi-lo das quatro seguintes formas:

- formato inicial (como o texto que acabamos de mostrar);
- "passado" pelo método trimGeral(...);
- "passado" pelo método trimToUpper(...);
- "passado" pelo método trimToLower(...).

Resolução

1)

Classe Pessoa:

```
public abstract class Pessoa {
    private String nome;
    private String endereco;
    public abstract void cadastrarPessoa();
    public String getNome() {
       return nome;
    }
    public void setNome(String nome) {
       this.nome = nome;
    }
    public String getEndereco() {
       return endereco;
    }
    public void setEndereco(String endereco) {
       this.endereco = endereco;
    }
}
```

2)

Classe Pessoa Física:

```
import javax.swing.JOptionPane;
public class PessoaFisica extends Pessoa {
     private String cpf;
     private int idade;
     public void cadastrarPessoa() {
        this.setNome(JOptionPane.showInputDialog
     ("Digite o Nome:"));
        this.setEndereco(JOptionPane.showInputDialog
     ("Digite o Endereço:"));
        cpf = JOptionPane.showInputDialog("Digite o CPF:");
        idade = Integer.parseInt(JOptionPane.showInputDialog
     ("Digite a Idade:"));
     }
     public String getCpf() {
        return cpf;
     }
     public void setCpf(String cpf) {
        this.cpf = cpf;
     }
     public int getIdade() {
        return idade;
     }
     public void setIdade(int idade) {
        this.idade = idade;
     }
}
```

Classe Pessoa Jurídica:

```
import javax.swing.JOptionPane;
public class PessoaJuridica extends Pessoa {
     private String cnpj;
     private String nomeFantasia;
     public void cadastrarPessoa() {
        this.setNome(JOptionPane.showInputDialog
     ("Digite o Nome:"));
        this.setEndereco(JOptionPane.showInputDialog
     ("Digite o Endereço:"));
        cnpj = JOptionPane.showInputDialog("Digite o CNPJ:");
        nomeFantasia = JoptionPane.showInputDialog
     ("Digite o Nome Fantasia:");
     }
     public String getCnpj() {
        return cnpj;
     }
     public void setCnpj(String cnpj) {
        this.cnpj = cnpj;
     }
     public String getNomeFantasia() {
        return nomeFantasia;
```

```
    }
    public void setNomeFantasia(String nomeFantasia) {
      this.nomeFantasia = nomeFantasia;
    }
}
```

3)

Classe StrEspecial:

```
public abstract class StrEspecial {
    // Método estático que retira espaços extras
    public static String trimGeral (String txt) {
      // retirando espaços iniciais e finais
      String auxTxt = txt.trim();
      // retirando espaços extras internos a String
      String espExt = "  ";
      String espFin = " ";
      int posic = auxTxt.indexOf(espExt);
      while (posic >= 0) {
        auxTxt = auxTxt.replaceAll(espExt, espFin);
        posic = auxTxt.indexOf(espExt);
      }
      return auxTxt;
    }
    // Método estático que deixa tudo em
    // maiúsculo e retira espaços extras
    public static String trimToUpper (String txt) {
      // retirando espaços gerais
      String auxTxt = trimGeral(txt);
      // deixando todo o texto em MAIÚSCULO
      auxTxt = auxTxt.toUpperCase();
      return auxTxt;
    }
    // Método estático que deixa tudo em
    // minúsculo e retira espaços extras
    public static String trimToLower (String txt) {
      // retirando espaços gerais
      String auxTxt = trimGeral(txt);
      // deixando todo o texto em minúsculo
      auxTxt = auxTxt.toLowerCase();
      return auxTxt;
    }
}
```

Classe TesteStr:

```
public class TesteStr {
    public static void main(String[] args) {
      String txt = "     Olá     Mundo     !     ";
      System.out.println("txt0 = [" + txt + "]");
      txt = StrEspecial.trimGeral(txt);
      System.out.println("txt1 = [" + txt + "]");
```

```
        txt = StrEspecial.trimToLower(txt);
        System.out.println("txt2 = [" + txt + "]");
        txt = StrEspecial.trimToUpper(txt);
        System.out.println("txt3 = [" + txt + "]");
    }
}
```

Resumo

Nesta unidade, vimos inicialmente que polimorfismo é um recurso da orientação a objetos que permite que mais de um elemento possa estar caracterizado de igual forma, mas cada um deles com comportamentos diferentes.

Já tínhamos visto que é possível o polimorfismo de métodos (sobrescrita ou sobrecarga). Nesta unidade, estudamos mais um tipo de polimorfismo, o polimorfismo de variáveis, em que duas variáveis podem coexistir com o mesmo nome, mas comportamentos diferentes (uma com comportamento global e outra com comportamento local), cuja diferença estará na utilização do termo this, de forma que a variável com esse termo será a variável global e a que estiver sem esse termo será a variável local.

```
variavel = valor1;  // esta é uma variável local
this.variavel = valor2;  // esta é uma variável global
```

Na linguagem Java existem alguns modificadores que alteram o comportamento ou a forma com que trabalhamos os elementos. Esses modificadores de comportamento permitem um controle da maneira com que uma equipe de desenvolvimento se utiliza de cada um daqueles elementos. São eles: o modificador final (para classes, métodos e atributos), sendo que uma classe final é uma classe que não pode ser herdada, um método final é um método que não pode ser sobrescrito e um atributo final é uma constante que não pode ter seu valor alterado; o modificador static (para métodos e atributos), o qual faz com que o elemento possa ser acessado sem que a classe a que pertence precise ser instanciada; e o modificador abstract (para classes e métodos), sendo que uma classe abstrata é uma classe que não pode ser instanciada.

Um método abstrato é um método sem implementação cuja declaração termina com ponto-e-vírgula que exige que a classe a que pertence seja também abstrata e que exige a sua implementação nas subclasses da classe a que pertence.

Uma interface é um elemento que, assim como as classes, pode possuir atributos e métodos, porém todos os seus métodos são abstratos e todos os seus atributos são finais e estáticos (constantes). Os métodos de uma interface não levam o modificador abstract (apesar de serem abstratos) e os seus atributos não levam o modificador final nem static (apesar de serem finais e estáticos).

Uma classe implementa uma interface utilizando-se do termo implements, e ela pode implementar múltiplas interfaces, utilizando-se da seguinte sintaxe:

```
public class ClasA implements InterfA, InterfB, InterfC {
     ...
}
```

A classe que implementa uma ou mais interfaces deve implementar todos os métodos de todas as interfaces que ela está implementando.

Por sua vez, a classe JOptionPane permite que mostremos janelas de pop-up de mensagens onde aparecem os botões OK, Cancelar, Sim e Não alternadamente, além de ícones padrões de mensagens de tela. Essa classe possui os métodos showMessageDialog(), showConfirmDialog() e showInputDialog(), que nos permite variar entre pop-ups de simples mensagens, pop-ups que retornam um número inteiro equivalente ao botão apertado e pop-ups que retornam o texto digitado em seu campo de inclusão.

Existem algumas wrapper classes que representam tipos primitivos, como as classes: Integer (que representa o tipo int), Short, Long, Double, Float e Character (que representa o tipo char). Uma das utilidades dessas classes é a realização do parse, ou seja, a transformação de um valor do tipo string para o tipo primitivo da linguagem Java que representa.

Exercícios

Questão 1. (FCC 2019) Considere as linhas de um pseudocódigo abaixo, levando em conta que a variável consumo_mes tenha sido declarada anteriormente para receber valores inteiros:

```
[...]
Início
   Escreva ("Entre com o valor consumido:")
   Leia (consumo_mes)
      [...]
Fim
```

Usando Java SE em uma aplicação com as bibliotecas necessárias devidamente importadas, essas linhas podem ser substituídas por:

A) consumo_mes = Integet.toInt(Dialog.showDialogBox("Entre com o valor consumido:"));

B) consumo_mes = Integer.parseInt(JOptionPane.showInputDialog(null, "Entre com o valor consumido:"));

C) consumo_mes = Integer.toInteger(Dialog.input("Entre com o valor consumido:"));

D) consumo_mes = Int.parseInt(Console.input("Entre com o valor consumido:"));

E) consumo_mes = Int.parse(Console.write("Entre com o valor consumido:"));

Resposta correta: alternativa B.

Análise da questão

A classe JOptionPane permite que mostremos ao usuário janelas do tipo pop-up. Dentre os métodos dessa classe, temos o showInputDialog(), que solicita um valor ao usuário e retorna o texto digitado como um valor do tipo string. Porém, de acordo com o enunciado, a variável consumo_mes deve receber valores inteiros. É possível transformar textos (valores do tipo string) em valores de tipos numéricos utilizando wrapper classes. No caso, o comando Integer.parseInt() realiza essa modificação, utilizando o método parseInt() da classe Integer. Dessa forma, por meio do trecho

```
consumo_mes = Integer.parseInt(JOptionPane.showInputDialog(null, "Entre com o
valor consumido:"));
```

a variável consumo_mes receberá um valor do tipo inteiro, correspondente ao texto digitado pelo usuário na janela pop-up.

Questão 2. (Idecan 2020, adaptada) A orientação a objetos (OO) é um paradigma de programação para o qual "tudo é um objeto", sendo Java uma das principais linguagens que implementam esse paradigma.

Em relação à linguagem Java e à OO, avalie as afirmativas:

I – Uma classe Java pode implementar mais de uma interface Java.

II – Uma classe Java abstrata, obrigatoriamente, deve possuir um ou mais métodos abstratos.

III – Uma classe Java declarada como final não pode ser herdada (não pode ter subclasses Java).

É correto o que se afirma em:

A) I, apenas.

B) II, apenas.

C) I e III, apenas.

D) II e III, apenas.

E) I, II e III.

Resposta correta: alternativa C.

Análise das afirmativas

I – Afirmativa correta.

Justificativa: em Java, uma interface é um recurso utilizado para "obrigar" as classes que a implementam a implementarem, também, os seus métodos. Interfaces podem servir como guias na implementação das regras de negócio de empresas, por exemplo. Uma classe pode, sim, implementar mais de uma interface. Nesse caso, a classe em questão deverá implementar todos os métodos de todas as interfaces.

II – Afirmativa incorreta.

Justificativa: uma classe abstrata é uma classe que não pode ser instanciada. Ela pode ser herdada por subclasses, mas não pode, diretamente, instanciar objetos. Um método abstrato só pode existir em uma classe abstrata, mas uma classe abstrata não precisa, necessariamente, ter métodos abstratos.

III – Afirmativa correta.

Justificativa: uma classe final é uma classe que não pode ser herdada. Isso quer dizer que, em uma cadeia de heranças, uma classe final será a última classe da cadeia. Dessa forma, ela não pode ter subclasses.

Unidade IV

7 EXCEÇÕES

Ao longo do desenvolvimento de sistemas, estamos sujeitos a situações que podem gerar erros de execução. Um erro de execução não é detectado na compilação do sistema, já que o código está correto, e pode acontecer a qualquer momento durante a utilização do sistema pelo cliente.

A prática de programação permite que o desenvolvedor preveja muitos dos possíveis problemas que podem ocorrer quando o sistema estiver rodando em produção. O problema está no fato de que caso ocorram alguns desses imprevistos, um erro pode encerrar todas as tarefas executadas no servidor, de modo a termos que reiniciá-lo.

Seguem alguns exemplos de erros de execução de sistema que podem ocorrer, apesar de toda a previsão e experiência do desenvolvedor:

- tentativa de acesso a banco de dados quando ele está fora do ar;
- um cálculo impossível (como uma divisão por zero) executado pelo cliente;
- uma falha de leitura de um determinado hardware ou periférico;
- uma falha física de acesso à memória.

A ocorrência desses erros gera o que chamamos de erros ou exceções no sistema (que vamos chamar a partir de agora apenas de exceções).

O compilador Java está preparado para identificar boa parte dessas exceções, gerando um aviso de exceção na tela da console.

Por exemplo, o programa a seguir não gera interrupção, pois não há exceções ocorrendo durante a sua execução:

```
int a = 12;
int b = 4;
int c = a / b;
System.out.println(c);
// --------------------
System.out.println("FIM DO PROGRAMA");
// ## Imprimirá na tela:
//   3
//   FIM DO PROGRAMA
```

No entanto, se fizermos com que a variável b tenha valor 0, como no exemplo a seguir, o programa sofrerá uma interrupção:

```
int a = 12;
int b = 0;
int c = a / b;
System.out.println(c);
// --------------------
System.out.println("FIM DO PROGRAMA");
```

Neste caso, como não há tratamento desse tipo de ocorrência, essa interrupção provocará uma parada geral de todas as outras execuções de programa do compilador. No caso, se esse sistema estiver sendo rodado em um servidor de aplicação, ele sofrerá uma parada e interromperá todas as outras execuções de programas, necessitando ser reiniciado.

Ao executar este último programa, veremos na console do servidor uma mensagem (geralmente em vermelho):

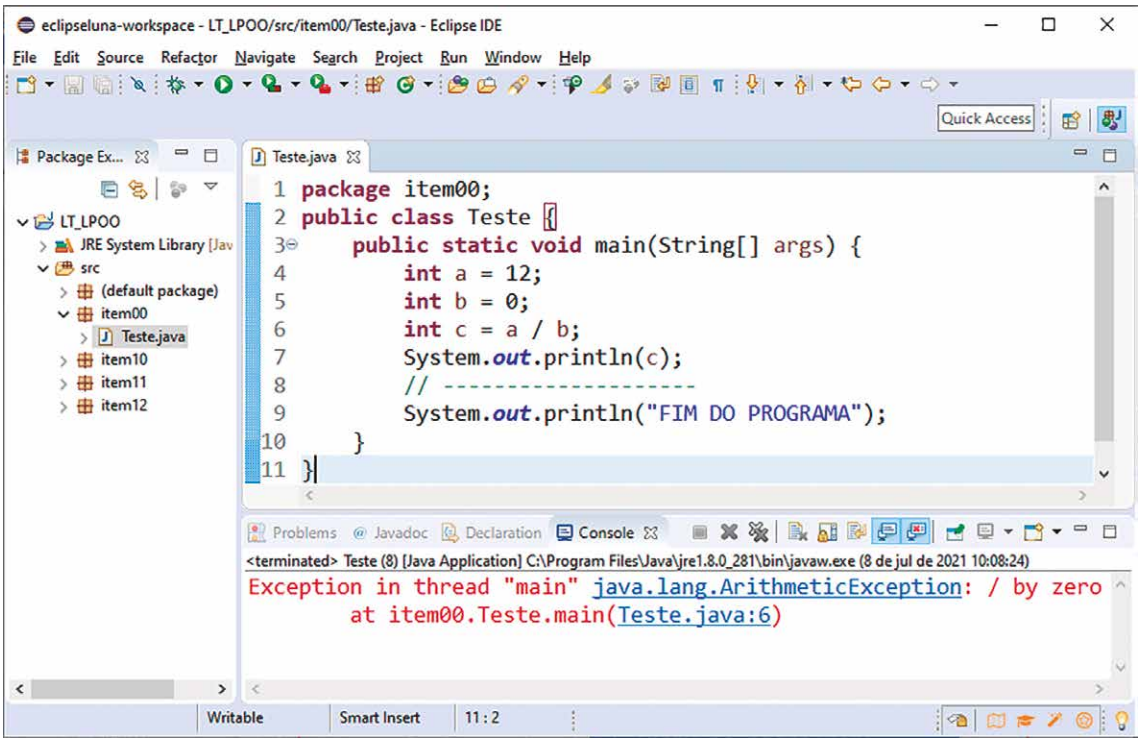


Figura 31 – Tela do Eclipse mostrando a localização do arquivo, o programa e a console com o resultado da execução

A mensagem da imagem mostra em que ponto (e de qual programa) ocorreu a exceção. Nela, temos o número da linha de código (6), de qual método e classe (método main da classe Teste do pacote item00), além da identificação da exceção ocorrida (java.lang.ArithmeticException) e uma explicação textual do erro (/ by zero, ou divisão por zero).

Observação

Na aritmética com números inteiros, não é possível realizarmos uma divisão de um número por 0.

Percebe-se nessa tela da console que o programa nem imprimiu o texto "FIM DO PROGRAMA", já que foi interrompido na linha 6 de execução e, consequentemente, não executou nenhuma linha de comando abaixo dela.

Essa mensagem, deixada pelo compilador na tela da console, procura dar uma pista aos administradores de rede e, também, à equipe de desenvolvedores Java, para identificar qual sistema gerou a exceção, para que ele possa ser acertado pela equipe de desenvolvimento, como um meio de facilitar a sua manutenção, e não gerar mais problemas ao servidor.

7.1 Hierarquia das exceções

Sabemos que na linguagem Java a base de construção de sistemas é a criação de classes (com seus atributos e métodos). Dessa forma, as exceções são classes da biblioteca do Java e, portanto, ao se manifestarem (ao serem instanciadas) se transformam em objetos (em memória) que também possuem atributos, métodos e construtores, como qualquer outra classe que vimos até agora.

As exceções se utilizam dos conceitos de orientação a objetos e compartilham de suas possibilidades e facilidades, como herança, polimorfismo, encapsulamento etc. Seguindo essa ideia, existe uma árvore hierárquica das exceções composta de todas as classes da biblioteca do Java, que podem ser acionadas assim que o problema é identificado. Para cada tipo de problema existe uma classe (uma exceção) específica a ser acionada.

A hierarquia das exceções está relacionada a superclasses e subclasses que compõem todas as possíveis classes que podem ser acionadas, dependendo do problema ocorrido na execução dos programas.

Lembrete

Uma superclasse (classe mãe) é uma classe que foi herdada por outra classe, enquanto a subclasse (classe filha) é a classe que herdou a superclasse.

A figura 32 mostra uma pequena parte dessa hierarquia, já que existem muitas outras classes na biblioteca além daquelas que podem ser criadas por qualquer desenvolvedor, e que são específicas para determinados sistemas.

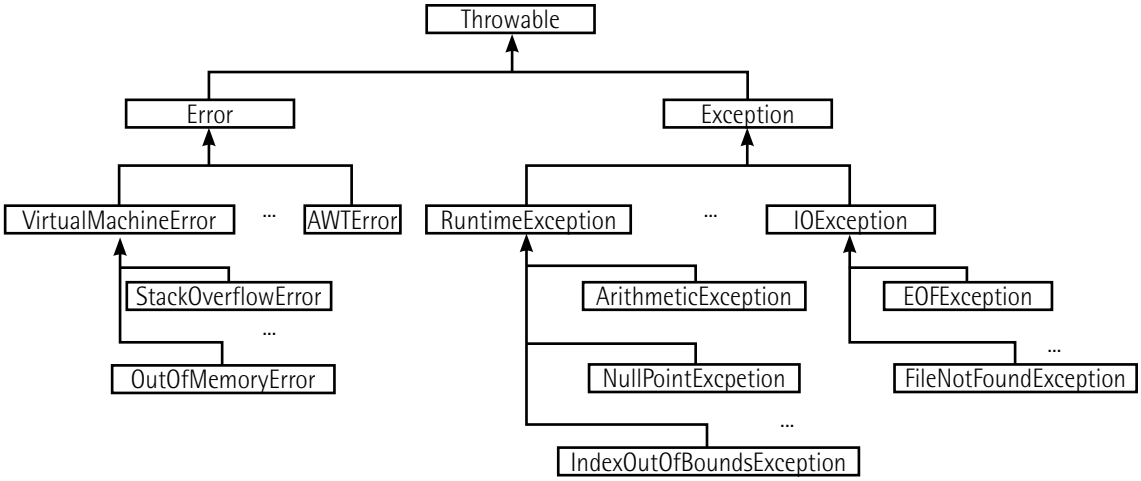


Figura 32 – Estrutura da classe java.lang.Throwable, raiz (mãe) de todas as classes que provocam interrupções

Na imagem, pode-se perceber que a classe Throwable é a classe mãe de todo tipo de interrupção que pode ocorrer em um sistema.

A hierarquia que tem como base a classe Error, uma das subclasses da classe Throwable, equivale mais a problemas que podem ocorrer na parte física de processamento, como na memória, por exemplo, além de outras situações. Esses são problemas que não costumam ocorrer normalmente. Já a hierarquia que tem como base a classe Exception, outra subclasse da classe Throwable, equivale a problemas de funcionalidade que podem ser previstas, porém nunca se sabe quando efetivamente poderão ocorrer.

Para facilitar o trabalho dos desenvolvedores, existe na linguagem Java um recurso que obriga e auxilia a "previsão" desses problemas, de forma a exigir o seu tratamento (que veremos no item a seguir).

Algumas classes de exceções obrigam sempre o seu tratamento, enquanto outras classes não o fazem. A utilização das classes de exceções derivadas da classe Exception – **exceto as derivadas de RuntimeException** – obrigam o tratamento e representam condições previsíveis, ou seja, que podem acontecer ao longo da execução de um método.

Observação

Como observado no parágrafo anterior, a classe RuntimeException e suas subclasses não obrigam o seu tratamento. No entanto, se elas ocorrerem na execução de um programa em que não foi feito o seu tratamento, uma interrupção será acionada, e o servidor em que o sistema estiver rodando sofrerá uma parada de funcionamento, precisando ser acionado (ou "subido") manualmente.

7.2 Tratamento de exceções

Uma exceção é um objeto gerado para indicar a ocorrência de algum tipo de condição excepcional durante a execução de um método. Uma vez detectada essa condição anormal, é necessário ativar mecanismos para corrigi-la e permitir o prosseguimento da execução.

Esse tipo de procedimento é chamado de tratamento (ou manipulação) de exceções, o que, no código, se faz por meio da utilização de uma estrutura própria, um bloco de tratamento de exceções (exception handler). O programador deverá prever os pontos do programa suscetíveis a algum tipo de exceção e definir, antecipadamente, blocos de tratamento de exceções para cada um deles.

O tratamento de exceções consiste em duas etapas:

1) "Tentativa" da execução do código.

2) Captura do tipo da exceção.

Quando ocorre uma exceção, o sistema (ou a máquina virtual do Java) está preparado para verificar se há um tratamento previsto (codificado no programa) para aquela exceção e o executa. No entanto, se isso não foi previsto, e o programador não programou uma "saída" para aquele problema específico, o sistema gerará uma falha e bloqueará a execução dos códigos subsequentes, provocando uma parada (uma interrupção) em todas as eventuais execuções da máquina virtual.

Percebe-se que a importância de se capturar e tratar uma exceção é impedir que haja uma parada nas execuções da máquina virtual, impedindo com isso que haja um travamento das execuções do servidor de aplicações do Java.

A estrutura de captura e tratamento de exceções (try-catch-finally) apresenta a seguinte sintaxe:

```
try {
   // códigos que podem gerar uma exceção (tenta-se executar)
} catch (Excecao01 e) {
   // bloco de tratamento realizado a partir da
   // identificação de uma exceção do tipo Excecao01
} catch (Excecao02 e) {
   // bloco de tratamento realizado a partir da
   // identificação de uma exceção do tipo Excecao02
...
} catch (ExcecaoN e) {
   // bloco de tratamento para exceção mais genérica
   // (geralmente Exception ou Throwable)
} finally {
   // bloco opcional mas que sempre será executado
   // independente da ocorrência ou não de uma exceção
}
```

Uma estrutura de captura e tratamento de exceções (try-catch-finally) deve ter:

- apenas um bloco try (no início da estrutura);
- um ou mais blocos catch (cada bloco capturando uma exceção específica).

Opcionalmente, pode ter apenas um bloco finally (ao final da estrutura).

Observação

O bloco finally, quando existente, será sempre executado, tendo ocorrido uma exceção ou não. De um modo geral, esse bloco inclui comandos de liberação de recursos que possam ter sido alocados no processamento do bloco try e que precisam ser liberados tendo sua execução sido concluída com sucesso ou não.

De uma forma geral, uma exceção, quando ocorre, gera a seguinte mensagem na console:

```
Exception in thread "main" java.lang.ExceptionName: general description
    at Class.method01(Class.java:line)
    at Class.method02(Class.java:line)
    ...
    at Class.main(Class.java:line)
```

Dessa forma, é possível rastrear uma exceção, já que na mensagem aparece o nome do método e de sua classe, além do número da linha de comando que gerou a exceção.

Por exemplo, a linha de comando a seguir gerará uma exceção de cálculo:

```
int a = 3, b = 0;
int c = a / b;
System.out.println("FIM DO PROGRAMA");
```

A segunda linha desse código (na verdade, a linha 9 do programa do qual esse código faz parte) gerará a seguinte mensagem de exceção:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero

    at TesteA.main(TesteA.java:9)
```

Assim, neste caso, para se criar uma estrutura de captura e tratamento de exceções, basta **envolver** a linha que gerou a exceção com um bloco try-catch, copiar o nome da exceção e colocá-lo como parâmetro de captura:

```
int a = 3, b = 0;
try {
   int c = a / b;
} catch (ArithmeticException e) {
   // lógica de tratamento
}
System.out.println("FIM DO PROGRAMA");
```

Executando este último programa-exemplo, veremos que não estamos impedindo de gerar o erro na sua execução, no entanto, perceberemos que dessa forma o sistema como um todo não será interrompido, já que existe um tratamento para o erro ocorrido.

7.2.1 Tratando várias exceções

Muitas vezes, num mesmo bloco de código, podemos ter várias situações de erros diversos, podendo haver a necessidade de tratarmos de forma diferente cada uma delas.

Podemos citar como exemplo uma situação em que estamos lidando com dados em banco de dados (BD), na qual, ao tentarmos acessar determinada informação nele guardada, podemos nos deparar com os seguintes problemas:

- o BD está temporariamente fora do ar;
- o BD está no ar, mas excedeu o número máximo de sua capacidade de receber solicitações de acesso;
- foi possível acessar o BD, mas o dado solicitado não foi encontrado.

Perceba que num conjunto de ações sequenciais, vários problemas podem necessitar de tratamentos diferentes. Por exemplo:

- com o BD fora do ar, pode-se tentar acessá-lo mais aproximadamente quatro vezes, automaticamente via sistema; caso o erro persista, deve-se gerar um aviso ao usuário para que ele se comunique com o help-desk ou, ainda, gerar um e-mail à equipe de help-desk avisando-a do problema;
- para o problema de excesso de solicitações de acesso, pode-se mostrar uma mensagem ao usuário para que ele tente novamente mais tarde (pois aquele pode ser um horário de pico);
- para o problema da informação não encontrada, deve-se simplesmente avisar sobre o ocorrido e sugerir uma nova busca.

Tratar cada uma das exceções significa justamente dar um encaminhamento diferente para cada tipo de problema que ocorrer (se ocorrer). Dessa forma, quando diversos tipos de exceções podem ser gerados na execução de um mesmo bloco de código, então é importante que se identifique a maioria das exceções que podem ocorrer, gerando um bloco de tratamento específico para alguns deles, ou todos eles, ou genérico para boa parte deles. Isso é muito importante pois caso ocorra alguma exceção não prevista, e ela não for devidamente capturada, essa exceção poderá interromper o curso de execuções de serviços do servidor, finalizando-o.

Como exemplo, podemos analisar o programa apresentado na imagem a seguir:

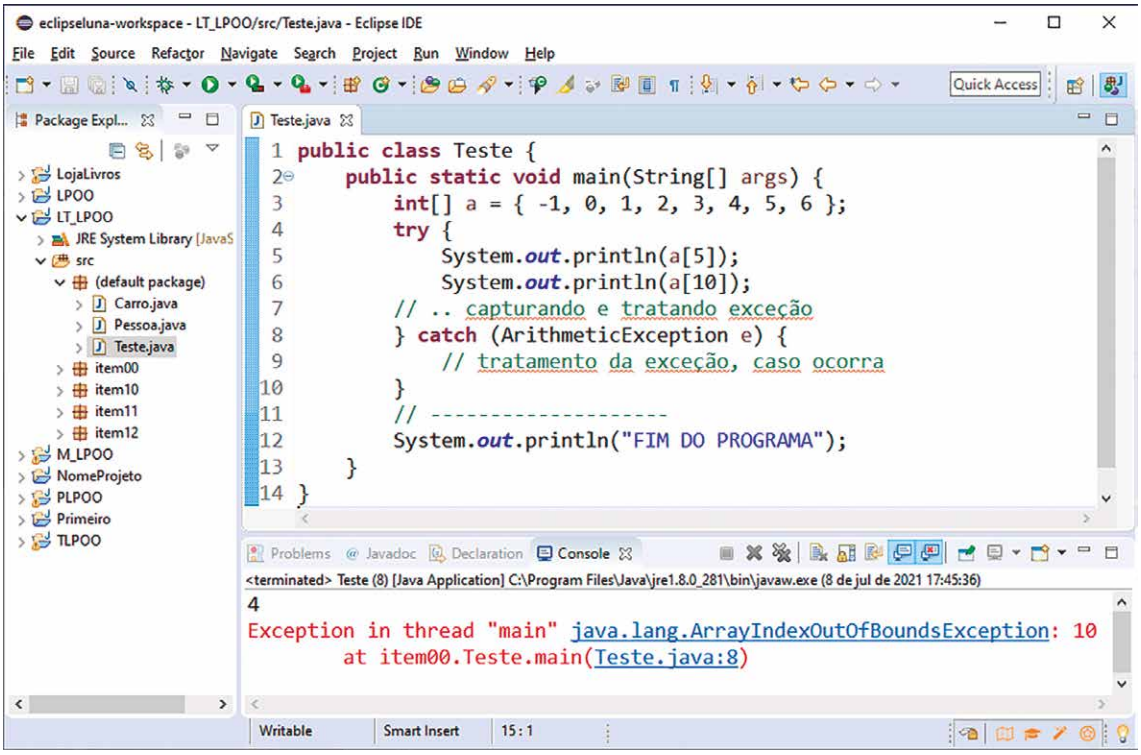


Figura 33 – Programa em que o tratamento da exceção não condiz com a exceção que ocorrerá efetivamente

Percebam que a primeira impressão na tela da console ocorreu sem problema (mostrando o valor da 5ª posição do vetor ocupada pelo número 4). No entanto, o sistema foi interrompido antes de chegar à última impressão (a impressão "FIM DO PROGRAMA" não foi executada como deveria).

Segundo a mensagem de exceção (em vermelho na imagem), o problema aconteceu na linha 8 e a exceção ocorrida foi:

```
ArrayIndexOutOfBoundsException
```

Decerto, se verificarmos o código da linha 8, veremos que ele tenta ler o valor da posição 10 do vetor (array) a, posição esta que não existe, já que ele contém apenas 8 valores. Essa tentativa incorreta de leitura gerou a exceção da mensagem.

Lembrete

Um array é uma variável que pode conter mais de um valor de um determinado tipo. Seus índices (posição de cada valor) iniciam em 0 e vão até o valor de seu tamanho (quantidade de valores) "menos um".

Contudo, por que o programa travou a continuidade da sua execução se havia um tratamento de erro previsto na sua codificação?

Se prestarmos atenção, a captura e o tratamento de exceção estavam focados sobre a exceção ArithmeticException. Porém, a exceção gerada foi outra, relacionada à leitura de posição inexistente em um vetor, e por isso o sistema entendeu que, como essa exceção não foi encontrada entre as opções de captura, não conseguiu seguir em frente, finalizando todas as execuções em andamento. Podemos, contudo, realizar várias capturas de exceções numa mesma estrutura try-catch, gerando um tratamento específico para cada uma delas:

Desse modo, a partir do exemplo anterior, podemos fazer:

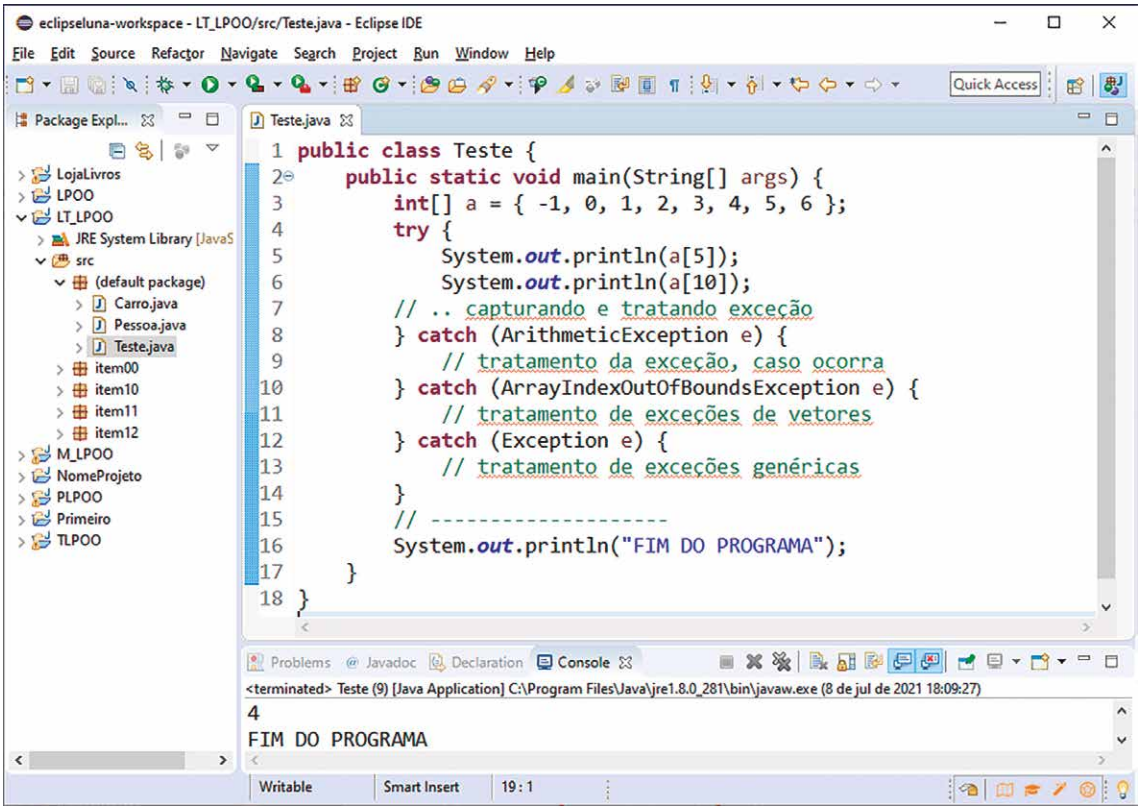


Figura 34 – Programa em que observamos múltiplos tratamentos de exceções a fim de garantir a integridade do sistema e do servidor

No exemplo que acabamos de apresentar, a exceção que ocorre na execução do código da linha 6 está devidamente capturada e tratada pelo sistema (nas linhas 10 e 11), de forma que, ao ser acionado, o programa executa o primeiro print (da linha 5), porém sofre uma interrupção no print da linha 6, passando a executar o tratamento que foi programado para rodar na ocorrência de sua captura.

Percebe-se que a última captura (linhas 12 e 13) é sobre um tipo de exceção mais genérico. Caso ao longo do bloco try se execute algum comando que provoque uma exceção não especificamente capturada, então cairá sobre uma captura genérica, e o tratamento programado a partir da linha 13 será acionado sem que o servidor seja finalizado, dando continuidade inclusive à execução das linhas que existem após o final do bloco try-catch.

Isso acontece porque, como dito no exemplo hipotético do parágrafo anterior, caso ocorra algum outro tipo de exceção, assim como explicado no item sobre polimorfismo de classes, o objeto e, parâmetro da captura da linha 13, por ser um objeto do tipo Exception (classe mãe das exceções), receberá e se comportará como um objeto relativo à exceção ocorrida (que certamente é uma classe filha da classe Exception) e, portanto, tem um tratamento definido nesse programa.

Observação

Sempre que se criar uma estrutura de captura e tratamento de exceções, caso venha a ser possível o sistema gerar uma exceção não prevista, deve-se inserir como última captura uma exceção genérica (Exception ou Throwable), garantindo assim a integridade de funcionalidade do seu sistema.

Essa captura deverá ser sempre a última, pois na ocorrência de uma exceção, o sistema sempre irá testar as exceções na ordem sequencial em que foram codificadas, de forma que caso a captura da Exception seja a primeira, sempre cairá nela, pois todas as exceções são subclasses de Exception.

7.3 Lançamento de exceções

Ao trabalharmos em um sistema em que muitas pessoas programam simultaneamente, ou ainda quando implementamos um sistema já existente no cliente, devemos sempre observar as regras de negócio desse cliente, gerando um sistema que esteja de acordo com elas. Neste caso, em vez de utilizarmos apenas as estruturas condicionais para definir e controlar essas regras, utilizamos também o lançamento de exceções nesse contexto, de modo a garantirmos que o programador se preocupe com as regras sempre que for utilizar aqueles métodos condicionados às exceções.

Lançar uma exceção é provocar uma interrupção em um ponto determinado do programa, que foi definido segundo as regras de negócio daquele sistema. Para fazer o lançamento da exceção e provocarmos essa interrupção em determinadas circunstâncias, utilizamos a palavra-chave throw, que gera uma exceção na JVM bloqueando a execução dos códigos existentes a partir do lançamento, caso ele efetivamente ocorra.

A sintaxe de um lançamento de exceção é:

```
throw new <tipo de exceção> (parâmetro);
```

O <tipo de exceção> é uma classe de exceção existente (na biblioteca Java ou alguma que tenha sido criada para o sistema), e o parâmetro (que pode existir ou não), que deve ser uma string, representará a mensagem de erro que será mostrada na console quando o erro (a exceção) ocorrer. Podemos usar como exemplo uma classe Conta (de um banco) em que existe um método de nome sacar(...), que recebe como parâmetro o valor que será acrescido na conta corrente.

Neste caso, sempre que algum programador venha a realizar alguma manutenção ao sistema e porventura se utilizar do método sacar(...), ele será forçado a envolver o método em um bloco try-catch para evitar que o servidor trave e, com isso, garantir que ele trabalhe na lógica de tratamento dessa ação em seu sistema.

Vejamos como fica a codificação do sistema desse exemplo:

```
public class Conta {
    private double saldo;
    public void sacar(double valor) {
       if (valor < 0) {
          String msg = "Este valor não pode ser negativo!!";
          System.out.println(msg);
          //.. e eventualmente alguma outra lógica necessária
          //.. para não deixar seguir com a transação
          //.. caso isto ocorra.
       } else {
          saldo -= valor;
       }
    }
}
```

Com o programa codificado que acabamos de apresentar, o criador do método sacar(...) será obrigado a definir toda a lógica de tratamento do problema quando for sacado um valor negativo.

```
public class TesteConta {
    public static void main(String[] args) {
       Conta c1 = new Conta();
       c1.sacar(-40);
    }
}
// A execução deste programa resultará na seguinte mensagem:
// Este valor não pode ser negativo!!
// ..de forma que este valor não será "retirado" no saldo.
```

Porém, esse código pode ser necessário ao funcionamento de vários outros sistemas criados muito tempo depois, não sendo possível se pensar em todos os problemas que poderão ocorrer em sistemas futuros com a utilização errada desse método.

Para isso, em vez de a lógica de tratamento estar codificada no método sacar(...), fazemos com que sempre que algum desenvolvedor (futuro) for se utilizar desse método, chamando o método sacar(...), será ele quem deve se preocupar em tratar o problema. Assim, o programador pode "lançar uma exceção" sempre que o problema ocorrer, para que a obrigação do tratamento do problema (o que fazer caso ele ocorra) fique a cargo do autor do outro programa que está chamando aquele método, sendo que para cada tipo de situação, o tratamento pode ser diferente.

No exemplo, vamos alterar apenas a 6ª linha de programação, lançando a exceção IllegalArgumentException(...).

```
public class Conta {
    private double saldo;
    public void sacar(double valor) {
      if (valor < 0) {
        String msg = "Este valor não pode ser negativo!!";
        throw new IllegalArgumentException(msg);
      } else {
        saldo -= valor;
      }
    }
}
```

No entanto, ao se chamar esse método, como a Classe IllegalArgumentException é uma subclasse da RuntimeException, ela não obriga o tratamento com a estrutura try-catch e, dessa forma, o programa a seguir, ao ser acionado, irá travar o servidor:

```
public class TesteConta {
    public static void main(String[] args) {
      Conta c1 = new Conta();
      c1.sacar(-40);
    }
}
```

Ele mostrará na console a seguinte mensagem:

```
Problems  @ Javadoc  Declaration  Console
<terminated> TesteConta (1) [Java Application] C:\Program Files\Java\jre1.8.0_281\bin\javaw.exe (9 de jul de 2021 18:18:38)
Exception in thread "main" java.lang.IllegalArgumentException: Este valor não pode ser negativo!!
        at Conta.depositar(Conta.java:7)
        at TesteConta.main(TesteConta.java:5)
```

Figura 35 – Mensagem de exceção mostrada na console do Eclipse

No entanto, se tratamos a exceção ao chamarmos o método sacar(...), assim como no exemplo da classe TesteConta a seguir, evitaremos o travamento da JVM (do servidor) e, caso o problema efetivamente ocorra, garantimos a continuidade da execução do sistema e de todos os outros sistemas existentes em execução naquele servidor.

```
public class TesteConta {
    public static void main(String[] args) {
      Conta c1 = new Conta();
      try {
        c1.sacar(-40);
      } catch (Exception e) {
        // Lógica de Tratamento da Exceção
      }
    }
}
```

Mas a pergunta é: como o desenvolvedor iria saber que o método sacar(...) deveria ser envolto em uma estrutura try-catch para tratamento da exceção?

Na verdade, da forma com que o sistema está construído, não tem como o desenvolvedor saber que deveria tratar o método sacar() com uma estrutura try-catch, e a não ser que ele esteja a par do lançamento da exceção existente, o sistema estaria em risco de travar o servidor caso essa situação aconteça, e ele só ficaria sabendo da necessidade do tratamento da exceção após ocorrer o problema.

Porém, existe uma forma de obrigarmos o desenvolvedor a envolver a chamada do método sacar(...) numa estrutura try-catch. Para que isso seja possível, utilizamos a palavra reservada throws na linha de declaração do método, arremessando uma exceção que obriga o tratamento (uma exceção que **não** seja subclasse da RuntimeException, podendo ser, por exemplo, a própria classe Exception).

```
public void sacar(double valor) throws Exception {
    ...
}
```

Assim, a linha de código que chamar esse método sacar(...) será obrigada a ser tratada com uma estrutura de tratamento de exceção, envolvendo-a com o bloco try-catch.

Dessa forma, vimos que para o lançamento de uma exceção usamos duas palavras reservadas:

- **Throw**: palavra-chave utilizada dentro do código de um método, com a finalidade de interromper a execução de uma sequência de códigos por meio do arremesso de uma exceção:

```
throw new TipoDeException(...);
```

- **Throws**: palavra-chave utilizada na declaração de um método e que vai arremessar aquele tipo de exceção, obrigando o tratamento a quem se utilizar daquele método.

```
public tipoRetorno metodo(param) throws TipoDaException { ... }
```

7.4 Criação de exceções

Além das exceções disponibilizadas na biblioteca do Java (em java.lang.Exception e java.lang.Error), é possível criar novas exceções de acordo com a necessidade do programador.

À medida que queremos impor as regras de negócio da empresa cliente no sistema que se está desenvolvendo, é importante que a equipe esteja ciente dessas regras. Com isso, podemos gerar interrupções personalizadas com exceções criadas a partir dessas regras. Essas exceções personalizadas podem ser úteis para se associar determinadas falhas de execução ou regras de negócio da empresa que exigem determinadas situações que definem como ela deve ser utilizada e que obrigarão ao desenvolvedor o tratamento do código à medida que se utilizem dos métodos controlados.

Para criarmos exceções novas, deve-se criar uma classe que representa essa regra (sempre, de acordo com a convenção de programação Java, com a palavra Exception ao final), e fazemos com que ela herde a classe Exception. Essa nova classe terá métodos construtores que devem chamar a classe superior (com a palavra-chave super), inserindo a mensagem a ser mostrada na tela da console, caso o problema aconteça.

Vamos imaginar que para o exemplo anterior, em que estamos gerando um sistema de acesso à conta bancária, o cliente queira controlar a situação da conta corrente, e o programador não pode permitir que o cliente do banco saque um valor acima do valor de seu saldo. Para isso, podemos criar uma classe de exceção com esse objetivo, fazendo com que todo desenvolvedor de sistemas que venha a se utilizar do método de saque de valor fique a par da necessidade desse controle e preveja o seu tratamento.

Com isso, no exemplo pode-se ser criada a seguinte classe:

```
public class SemSaldoException extends Exception {
   public SemSaldoException () {
      super("Não há Saldo para esta Transação!!");
   }
   public SemSaldoException (String msg) {
      super(msg);
   }
}
```

No exemplo, criou-se a classe SemSaldoException, que herda Exception (e por definição é uma Exception). Essa classe, como normalmente é o padrão das exceções, possui dois métodos construtores: um sem parâmetro, com uma mensagem padrão; e outro que recebe a mensagem como parâmetro. Além disso, o fato de herdar a classe Exception, e não a classe RuntimeException, torna-o um método que quando for chamado obrigará o seu tratamento no método que o chamar (utilizar).

Dessa forma, a classe Conta pode se utilizar dessa exceção para o método sacar(...), lançando a exceção e ficando da seguinte forma:

```
public class Conta {
    private double saldo;
    public void depositar(double valor) throws Exception {
      if (valor < 0)
        throw new IllegalArgumentException("Este valor não
pode ser negativo!!");
      else
        saldo += valor;
    }
    public void sacar(double valor) throws Exception {
      if (valor < 0)
        throw new IllegalArgumentException("Este valor não pode ser
negativo!!");
      else if (valor > saldo)
        throw new SemSaldoException("Saque acima do Saldo...");
      else
        saldo -= valor;
    }
    public void mostrarSaldo() {
      System.out.println("Saldo atual: R$ " + String.format("%.2f", saldo));
    }
}
```

Assim, ao gerarmos a classe que vai se utilizar dos métodos depositar e sacar, se simplesmente chamarmos esses métodos sem tratá-los, observaremos o seguinte erro:

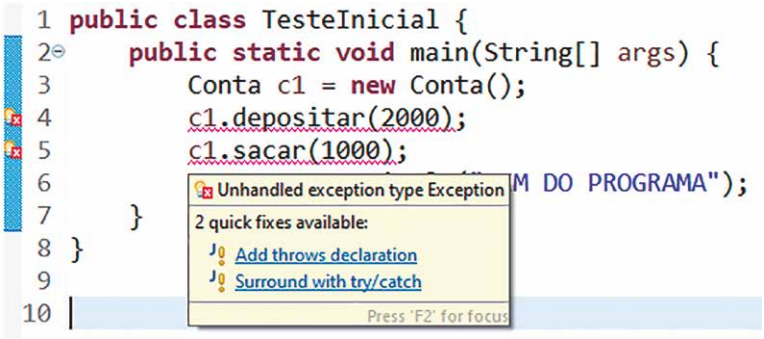


Figura 36 – Mensagem do erro de compilação do Eclipse sobre a chamada dos métodos depositar e sacar

Observa-se que duas linhas da classe TesteInicial (as linhas de código 4 e 5) estão acusando erro, pois os métodos depositar e sacar agora exigem o tratamento, o qual podemos ver no seguinte exemplo:

```
public class TesteInicial {
    public static void main(String[] args) {
      Conta c1 = new Conta();
      try {
```

```
            c1.depositar(2000);
            c1.mostrarSaldo();
            c1.sacar(1500);
            c1.mostrarSaldo();
            c1.sacar(2000);
        } catch (Exception e) {
            e.printStackTrace();
        }
        c1.mostrarSaldo();
        System.out.println("FIM DO PROGRAMA");
    }
}
```

Rodando o exemplo, teremos a seguinte saída na tela da console:

```
<terminated> TesteInicial [Java Application] C:\Program Files\Java\jre1.8.0_281\bin\javaw.exe
Saldo atual: R$ 2000,00
Saldo atual: R$ 500,00
SemSaldoException: Saque acima do Saldo...
        at Conta.sacar(Conta.java:14)
        at TesteInicial.main(TesteInicial.java:9)
Saldo atual: R$ 500,00
FIM DO PROGRAMA
```

Figura 37 – Impressão da tela da console ao rodarmos a classe TesteInicial

Neste exemplo, ao tentar sacar R$ 2.000,00 após sacar R$ 1.500,00, o sistema acusou uma exceção do tipo SemSaldoException, que foi a exceção criada para tratar essa regra de negócio da empresa.

Saiba mais

Saiba mais sobre exceções no capítulo 11, itens 11.4 e 11.5, de:

HORSTMANN, C. *Conceitos de computação com Java*. 5. ed. Porto Alegre: Bookman, 2009.

Exemplo de aplicação

Com relação ao conceito das exceções, podemos afirmar que:

I – Quando um bloco try pode gerar vários tipos de exceções diferentes, podemos criar várias formas de tratamento, capturando individualmente cada uma delas, mas sempre lembrando de deixar a exceção mais genérica por último.

II – Quando uma exceção acontece, o sistema irá sempre acusar sua ocorrência para que se possa ter um controle dela a partir de mensagens na console e continuar a executar as linhas de códigos subsequentes à que gerou aquela exceção.

III – Para que uma exceção exija o seu tratamento, devemos lançar a exceção na declaração do método com a instrução throws, desde que não seja lançada utilizando-se a exceção RuntimeException (ou suas subclasses).

IV – Num mesmo bloco try, é possível que aconteçam duas exceções uma após a outra em uma mesma execução daquele bloco, e isso pode exigir um tratamento individual para cada uma delas.

De acordo com as afirmativas, podemos dizer que estão corretas:

A) Apenas as afirmativas II e III.

B) Apenas as afirmativas II e IV.

C) Apenas as afirmativas I e II.

D) Apenas as afirmativas I e III.

E) Apenas as afirmativas III e IV.

Resposta correta: alternativa D.

Resolução

I – Afirmativa correta.

Justificativa: um bloco try pode vir a gerar, em ocasiões diferentes, mais de um tipo de exceção, de modo que o tratamento pode ser individualizado em cada uma das ocasiões. O tratamento dá-se no bloco de captura (catch), o qual poderá existir um para cada tipo de exceção. Devemos sempre lembrar de incluir uma captura de exceção genérica (por exemplo a Exception ou a Throwable), pois erros inesperados podem ocorrer, e se não estiverem tratados nas capturas individuais, poderão travar a execução do servidor. No entanto, essa captura de erro genérico deve ser a última, pois o tratamento se dá no bloco da primeira exceção da qual aquele erro se encaixa e pela característica de polimorfismo, uma exceção genérica, pode representar qualquer uma de suas especificidades (subclasses).

II – Afirmativa incorreta.

Justificativa: ao acontecer uma exceção, o sistema verifica se há tratamento para ela ou para uma generalização dela (superclasse); se não houver, o programa trava sua execução, encerrando-se. E mesmo que haja o tratamento, a execução não continua a partir do ponto da exceção, mas sim nos códigos subsequentes ao bloco try-catch de captura e tratamento.

III – Afirmativa correta.

Justificativa: um método em que na sua declaração é lançada uma exceção com a instrução throws (por exemplo, public void nomeMetodo() throws Excepton {...}) exigirá que o desenvolvedor envolva a chamada ao método em um bloco try-catch. É importante saber que a exceção RuntimeException, ou qualquer uma de suas subclasses, não exige o envolvimento com o bloco try-catch na invocação do método que porventura a lançar.

IV – Afirmativa incorreta.

Justificativa: quando ocorre uma exceção, o sistema interrompe a execução do bloco try e pula para a captura da exceção a partir dos eventuais blocos catch que existirem, e não mais volta a execução daquele bloco try, seguindo a execução dos códigos que existirem após a estrutura de tratamento (try-catch). Isso quer dizer que a partir da execução de um mesmo bloco try não é possível ocorrer mais uma exceção na mesma execução.

8 THREADS

A maioria dos programas que utilizamos em nosso micro trabalha no modo multitarefa (ou multitask), ou seja, funciona acionando vários recursos ao mesmo tempo. Nosso sistema operacional é um sistema multitarefa, já que conseguimos acionar vários programas ao mesmo tempo.

Enquanto estou escrevendo este livro-texto em meu computador, posso escutar uma playlist em meu fone de ouvido conectado ao micro acionado a partir de um streaming de músicas e, numa outra tela, tenho meu browser acessando meu e-mail, com o micro conectado à internet. Isso só é possível porque, assim como todo notebook existente, esse meu micro permite que vários programas rodem ao mesmo tempo, caracterizando um processo multitarefado – ou, como alguns livros descrevem, um processamento de sistemas em paralelo.

O recurso de programação que permite esse multiprocessamento de sistemas em paralelo é chamado de thread. A utilização de vários programas distintos normalmente já é gerenciado pelo próprio sistema operacional.

Como um exemplo mais simples, imagine que estejamos construindo um programa que gera um relatório em PDF. Como às vezes esse processo é um pouco demorado, para dar alguma satisfação ao usuário, é interessante que ele veja na tela uma barra de progresso, que pode dar uma ideia do tempo restante de processamento. Só isso já caracteriza um processamento paralelo.

Neste caso, quando temos um programa rodando e gerando vários processamentos, normalmente para isso utilizam-se threads, que permitem múltiplas atividades dentro de um único processo.

Existem várias razões para se utilizar threads:

- maior desempenho em ambientes multiprocessados;
- responsividade em interfaces gráficas;
- simplificação na modelagem de algumas aplicações.

8.1 Criando threads

Existem duas maneiras de se trabalhar com uma thread em Java:

- **utilizando herança**: criamos uma classe que herda (estende a ideia) a classe Thread;
- **usando implementação de interface**: criamos uma classe que implementa a interface Runnable.

Por boas práticas, geralmente implementamos a interface Runnable em vez de herdar a classe Thread. Isso ocorre porque a linguagem Java permite a herança direta de apenas uma classe, e como a herança é um recurso muito comum (é uma das características fundamentais da orientação a objetos), podemos fazer com que a classe herde uma outra classe e implemente a interface Runnable.

De qualquer forma, seja herdando a classe Thread, seja implementando a interface Runnable, o método que rodará em paralelo com qualquer outro processamento do sistema, inclusive com outras threads, será o método run(), a partir do acionamento do método start(), e é esse método run() que deve acionar todos os outros métodos necessários para o processo.

A seguir, veremos como é essa implementação com ambas as formas.

8.1.1 Pela herança da classe thread

Utilizando herança, a classe que terá um ou mais processamentos em paralelo deve herdar a classe Thread. Para que funcione, deve-se sobrescrever o método

```
public void run()
```

Exemplo:

```
public class Exemplo1 extends Thread {
    public void run() {
       // roda alguma lógica a ser
       // executada em paralelo com outros processos
    }
}
```

8.1.2 Usando a interface runnable

Implementando a interface Runnable, seremos obrigados a implementar o método

```
public void run() {…}
```

Exemplo:

```
public class Exemplo2 implements Runnable {
    public void run() {
       // roda alguma lógica a ser
       // executada em paralelo com outros processos
    }
}
```

8.1.3 Comparando

As duas formas são utilizadas praticamente da mesma forma. A única diferença é que uma herda a classe Thread e a outra implementa a interface Runnable. Como foi dito anteriormente, a vantagem de se implementar a interface Runnable é que a classe que se comportará como uma thread fica livre para herdar outra classe qualquer.

8.2 Rodando threads

Para acionarmos uma ou várias threads é necessário acionar o start em cada uma delas, ou seja, rodar o método start() que existe (por herança) em cada classe que implemente essa funcionalidade.

Quando acionamos o start, estamos iniciando um processamento em paralelo e liberando o programa para executar qualquer outra thread. O acionamento desse start aciona o método run() criado na classe que implementou ou herdou a thread.

Do exemplo anterior (a classe Exemplo), seja ela herdando a classe Thread ou implementando a interface Runnable, para que seu método run() rode em paralelo com outros processamentos, basta que acionemos o método start() que, automaticamente, seu método run() será acionado em paralelo.

A única diferença é que no segundo caso, ao se implementar a interface Runnable, é necessário que se crie uma instância da classe Thread em que esta recebe como parâmetro de seu método construtor a classe que implementou a inteface Runnable. Veja o exemplo a seguir:

```
Exemplo1 ex1 = new Exemplo1(); // que herda Thread

ex1.start();

Exemplo2 ex2 = new Exemplo2(); // que implementa Runnable

Thread t2 = new Thread(ex2);
```

```
// ou…: Thread t2 = new Thread(ex2, "Nome da Thread");

t2.start();
```

Neste caso, tanto o método run() da classe Exemplo1 quanto o da classe Exemplo2 rodarão em paralelo.

Observação

O programador não tem nenhum controle sobre o escalonador do processador. Isso significa que sendo os processos executados paralelamente, não há como saber qual a ordem de execução das linhas de código de cada um desses métodos acionados pela thread.

Um bom exemplo de acionamento de threads é o conhecido e clássico jogo do Pac-Man. Se esse jogo fosse desenvolvido na linguagem Java, poderia ser construído de várias maneiras, e para cada personagem teríamos uma thread controlando seus movimentos. Uma thread controlaria os movimentos do personagem principal a partir do acionamento das setas do teclado (acionamento realizado pelo usuário-jogador), enquanto outra controlaria cada um dos movimentos dos fantasmas que perseguirão o personagem principal do jogo, com estratégias controladas por algum programa de inteligência artificial que seria acionado pelo método run() de cada uma das classes que os representam.

Construindo um programa que utiliza Thread

Neste exemplo, construiremos uma classe chamada Contagem com um método run() que, ao rodar, imprimirá os números de 1 a 10 na tela da console, sendo cada número numa linha, e precedidos de uma identificação (uma string) recebida a partir do método construtor daquela classe. Essa classe implementará a interface Runnable, e com isso será possível executá-la por meio de uma thread.

```
public class Contagem implements Runnable {
    private String id = "";
    public void run() {
      System.out.println("Iniciando para " + id + ":");
      for (int x = 1; x <= 10; x++) {
        System.out.println(id + ": " + x);
        // Dando um tempo de espera de 0,1 segundos
        try {
          Thread.sleep(100);
        } catch (InterruptedException e) {
        }
      }
    }
    public Contagem(String s) {
      id = s;
    }
}
```

A partir dessa classe, construiremos uma classe de teste que irá instanciar a classe Contagem duas vezes, cada uma com uma identificação diferente, de modo que iremos rodar individualmente o método run() de cada uma delas e, em seguida, criaremos duas threads individuais, em que acionaremos esses mesmos métodos a partir do método start(). Com isso, veremos a diferença do acionamento individual do método run() e do seu acionamento via thread, por meio do método start().

```
public class TesteCont {
    public static void main(String[] args) {
       Contagem c1 = new Contagem("C1");
       Contagem c2 = new Contagem("C2");
       // Primeiro: rodaremos individualmente os
       // métodos de ambos os Objetos (c1 e c2)
       c1.run();
       c2.run();
       // Segundo: acionaremos os mesmos métodos
       // a partir de Threads
       Thread t1 = new Thread(c1);
       Thread t2 = new Thread(c2);
       t1.start();
       t2.start();
    }
}
```

Ao rodarmos o programa, veremos que o sistema primeiro rodará **todo** o método run() para c1, mostrando os números de 1 a 10 precedidos do termo C1, em seguida rodará **todo** o método run() para c2, mostrando também os números de 1 a 10, no caso, precedidos do termo C2. Isso acontece porque estamos acionando individualmente, e diretamente, o método run() de cada um dos objetos. Em seguida a isso, veremos que para as duas classes, os métodos run() serão acionados quase ao mesmo tempo (um logo após o outro, pois o start é dado individualmente para cada um deles) e que rodarão praticamente juntos, tanto para c1 quanto para c2 (às vezes, inclusive, alternando a impressão entre um e outro).

Saiba mais

Saiba mais sobre threads no capítulo 11, itens 11.4 e 11.5, de:

WINDER, R. *Desenvolvendo software em Java*. Rio de Janeiro: LTC, 2009.

Exemplo de aplicação

Vimos no item anterior que a linguagem Java é uma linguagem multithread. Isso quer dizer que:

A) Uma classe pode herdar múltiplas classes.

B) Podemos executar diversos eventos simultaneamente a partir de um mesmo programa Java.

C) Um mesmo método pode ser criado múltiplas vezes em uma mesma classe.

D) Um mesmo erro pode ser capturado de múltiplas formas.

E) Um programa Java pode ser desenvolvido por uma equipe com múltiplas pessoas.

Resposta correta: alternativa B.

Resolução

Nesta unidade, estudamos o que é thread e qual a sua utilidade. Vimos que ele pode ser acionado a partir do acionamento do método start() da classe Thread, executando uma série de eventos definidos e programados no método run() de uma classe que herda a classe Thread ou implementa a interface Runnable.

A) Alternativa incorreta.

Justificativa: a herança de classes independe de thread. Na linguagem Java, não existe herança múltipla, mas sim por encadeamento.

B) Alternativa correta.

Justificativa: o objetivo do Thread é justamente possibilitar a execução simultânea de diversos eventos (métodos) sempre que necessário, como acontece em games, ou ainda quando, por exemplo, queremos mostrar para o usuário uma barra de progresso mostrando o andamento da execução de uma transação, quando ela pode ser demorada.

C) Alternativa incorreta.

Justificativa: o thread não está relacionado ao desenvolvimento de métodos com características semelhantes. No entanto, sabemos que não pode haver mais de um método com mesma assinatura em uma mesma classe.

D) Alternativa incorreta.

Justificativa: a captura de erros ou exceções pode permitir que situações diferentes possam ser tratadas de formas diferentes. Mas essa característica não está relacionada a threads, mas sim às múltiplas utilidades do tratamento de exceções.

E) Alternativa incorreta.

Justificativa: o fato de trabalharmos individualmente ou em equipe não está relacionado à utilização de threads em um sistema. Normalmente trabalha-se em equipe na construção de sistemas em Java, e essa característica apenas exige um bom gerenciamento e acompanhamento de projetos.

Para finalizar esta unidade, observe os exemplos de aplicação a seguir. As resoluções são apresentadas na sequência.

Exemplos de aplicação

1) Criar uma classe abstrata para guardar funções diversas (pode ser a classe FuncoesDiversas). Criar nessa classe uma função estática de nome ehNumero(...) que deve receber um valor do tipo string e retornar true ou false dependendo se o valor recebido for numérico (real, ou seja, double) ou não. Procure fazer uma verificação em que o sistema tente transformar o valor string recebido em um valor numérico do tipo double, utilizando-se da estrutura try-catch.

2) Criar uma classe de teste com o método main, de modo que uma variável deve receber o valor a partir de uma input box (utilizar JOptionPane), e em seguida deve-se testar o valor utilizando-se do método criado no exercício 1 e somente transformar o valor em número se ele for efetivamente numérico.

3) Criar uma classe abstrata chamada Calculadora com um método estático de nome dividir(...) que deve receber dois parâmetros do tipo double, de modo que o método deverá retornar o resultado da divisão do valor do primeiro parâmetro pelo valor do segundo. Faça com que esse método lance uma exceção caso o segundo parâmetro tenha valor zero (com a mensagem "Valor inapropriado – Zero!"), de modo que sempre que esse método for acionado (chamado) por qualquer ação de uma classe, ele obrigue que essa chamada seja feita sobre uma estrutura de tratamento de exceção (try-catch).

Criar então numa classe de teste que tenha em seu método main uma chamada para este método com quaisquer valores numéricos (zero ou não).

Observação importante: no Java, uma divisão de um valor numérico do tipo double por zero não gera uma exceção no sistema, retornando como resultado o valor Infinity (infinito).

4) Criar uma thread equivalente à do exemplo descrito antes nesta unidade (exemplo da classe Contagem), mas que deverá imprimir os números de 1 a 100, de forma que cada número impresso tenha um tempo aleatório de espera entre uma impressão e outra.

Observação: o exemplo a seguir é o de um código que gera um número aleatório entre 1 e 100:

Sabendo que o número entre parênteses do método sleep(...) é o tempo de espera em milissegundos (milésimos de segundos), faça com que o número aleatório seja um valor inteiro entre 0 e 10 décimos de segundos.

Lembre-se que:

100 milésimos de segundo = 1 décimo de segundo

200 milésimos de segundo = 2 décimos de segundo

300 milésimos de segundo = 3 décimos de segundo

...

Exemplo de código que retorna um número inteiro aleatório entre 1 e 100:

```
Random rndm = new Random();

// Neste caso deve-se importar a Classe java.util.Random
...
int num = rndm.nextInt(100) + 1;
// Desta forma a variável "num" obterá um número entre 1 e 100
// Obs,: A função nextInt(valor) da Classe Random retorna
// um número entre 0(zero) e valor-1.
```

5) Pesquise e descreva quando devemos utilizar a exceção do tipo IllegalArgumentException, que é uma subclasse da classe RuntimeException.

Resolução

Existem várias formas de se resolver cada um dos exercícios propostos. Nossa intenção é apresentar aqui uma dessas possíveis soluções, tentando dar uma lógica simples e prática.

1)

```
public abstract class FuncoesDiversas {
    public static boolean ehNumero(String str) {
      boolean res = false;
      try {
        double num = Double.parseDouble(str);
        res = true;
      } catch (Exception e){
      }
      return res;
    }
}
```

A ideia dessa solução é tentar (try) transformar em double o valor recebido como string, de forma que a variável res, que inicia com o valor false, somente se tornará true após a transformação do valor em número caso isso seja possível (lembrando que quando ocorre uma exceção no bloco try, que no caso acontecerá se o valor não for numérico, o sistema para de executá-lo, passando a executar o bloco de captura catch), de forma que depois, ao sair do bloco try-catch, continua a executar a função, retornando o valor de res tendo este sido alterado ou não.

2)

```
import javax.swing.JOptionPane;
public class TesteFD {
    public static void main(String[] args) {
      String str = JOptionPane.showInputDialog(null,
          "Digite o valor!", "Entrada de Dados",
          JOptionPane.INFORMATION_MESSAGE);
```

```
        double val = -1;
        if (FuncoesDiversas.ehNumero(str)) {
          val = Double.parseDouble(str);
        }
        System.out.println("Valor = " + val);
      }
}
```

3)

```
public abstract class Calculadora {
    public static double dividir(double a, double b)
        throws Exception {
      double res = -1;
      if (b == 0)
        throw new Exception("Valor inapropriado - Zero!");
      else res = a/b;
      return res;
    }
}
public class TesteCalc {
    public static void main(String[] args) {
      double a = (double)5/(double)0;
      System.out.println(a);
      try {
        double x = Calculadora.dividir(6, 0);
        System.out.println(x);
      } catch (Exception e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
      }
      System.out.println("FIM DO PROGRAMA.");
    }
}
```

Esse teste terá como resultado (na tela da console):

```
Infinity
java.lang.Exception: Valor inapropriado - Zero!
FIM DO PROGRAMA.
    at Calculadora.dividir(Calculadora.java:6)
    at TesteCalc.main(TesteCalc.java:6)
```

Neste caso, inicialmente foi forçada uma conta de divisão por zero com valores do tipo double (para isso, fizemos um casting para double do valor zero – o que poderia ter sido feito simplesmente com o termo double a = 5.0 / 0;). Podemos observar que a execução não gerou uma exceção, e o resultado final foi o termo Infinity. Vê-se nesse programa que a chamada ao método dividir(...) está "obrigatoriamente" envolta numa estrutura try-catch; se não estivesse, o sistema não iria compilar o programa e consequentemente não iria rodá-lo.

4)

```
import java.util.Random;
public class Contagem implements Runnable {
    private String id = "";
    public void run() {
      Random rndm = new Random();
      System.out.println("Iniciando para " + id + ":");
      for (int x = 1; x <= 100; x++) {
        System.out.println(id + ": " + x);
        // Dando um tempo de espera de valores diferentes
        // aleatoriamente variando entre 1 e 10
        // décimos de segundo
        try {
          int num = rndm.nextInt(10) + 1;
          Thread.sleep(num * 100);
        } catch (InterruptedException e) {
        }
      }
    }
    public Contagem(String s) {
      id = s;
    }
}
```

Veja o comando a seguir:

```
int num = rndm.nextInt(10) + 1;
Thread.sleep(num * 100);
```

Nele, inicialmente gera-se um número aleatório entre 1 e 10; depois, no método sleep, multiplica-se o valor por 100 para que o tempo de espera seja um número inteiro em décimos de segundos.

A exceção IllegalArgumentException é uma exceção bastante utilizada sempre que precisamos validar algum valor, a fim de que esteja dentro de determinados parâmetros aceitáveis, seja para o programa, seja para alguma regra de negócios da empresa em relação àquele sistema.

A única atenção a essa exceção é que, como ela é uma subclasse da RuntimeException, se lançarmos um método com essa exceção, o sistema não obrigará o tratamento com a estrutura try-catch quando aquele método for acionado (chamado). Nesse caso, o ideal é lançar a exceção IllegalArgumentException com o processo throw, mas lançar para o método a exceção Exception juntamente com o comando throws.

Resumo

Nesta unidade, vimos que algumas ações realizadas por métodos podem resultar em erros (ou exceções) que podem barrar o funcionamento do compilador, podendo inclusive interromper a execução de todas as aplicações de um servidor de aplicação Java. Para evitar que essa interrupção aconteça, nos utilizamos de uma estrutura de tratamento de exceções que, apesar de não evitar que a exceção aconteça, consegue evitar que haja a interrupção do sistema que está sendo rodado e consequentemente evita a interrupção do funcionamento do servidor.

Para isso, utilizamos uma estrutura de tentativa de execução e de captura de exceção, que é a estrutura try-catch. A sintaxe dessa estrutura é:

```
try {
   // códigos que podem gerar uma exceção (tenta executar)
} catch (Excecao01 e) {
   // bloco de tratamento realizado a partir da
   // identificação de uma exceção do tipo Excecao01
} catch (Excecao02 e) {
   // bloco de tratamento realizado a partir da
   // identificação de uma exceção do tipo Excecao02
...
} catch (ExcecaoN e) {
   // bloco de tratamento para exceção mais genérica
   // (geralmente Exception ou Throwable)
} finally {
   // bloco opcional mas que sempre será executado
   // independente da ocorrência ou não de uma exceção
}
```

Aqui, pode-se capturar e tratar diversas exceções, desde que estejam dispostas em ordem hierárquica (de herança), sendo a mais genérica por último, e nas quais o termo finally contém o bloco que será executado independentemente se a exceção acontecer ou não. O tratamento da exceção é a lógica de ação realizada (no bloco de código) caso a exceção aconteça.

Podemos forçar, para um método, o uso da estrutura de tratamento de exceções, utilizando os termos throw e throws (em conjunto), obrigando assim a criação de uma estrutura específica (obrigar uma atenção especial) sempre que o desenvolvedor for se utilizar daquele método. De uma forma geral, a sintaxe do método que lança uma exceção é:

```
modificador tipoRetorno nomeMetodo(parametros) throws Excecao {
      ...
      throw new ExcecaoEspecifica (mensagem);
      ...
}
```

Podemos também criar exceções novas que especifiquem eventuais regras de negócios da empresa para a qual estamos gerando um sistema em Java. Para isso, basta criar uma classe, que por convenção tenha seu nome terminado com a palavra Exception e que herde a classe Exception. Esta classe criada deve ter um método construtor que acione o método construtor de sua classe mãe por meio do método super(), enviando como parâmetro uma string com a mensagem de exceção.

A thread é um recurso (uma classe Java) que permite que nosso sistema tenha multiprocessamento. Quando um programa exige a existência de multitarefa, como a maioria dos games, por exemplo, fazemos com que cada uma dessas tarefas seja rodada a partir de uma thread.

Uma thread (uma classe) pode ser criada por herança da classe Thread, ou implementando a interface Runnable (sendo esta última a forma mais utilizada), de modo que a lógica com as linhas de comando, que devem rodar em paralelo com outras, deve ser codificada no método run() dessa classe (método público e sem retorno – void).

A diferença entre uma forma ou outra está na hora de acionar o processo de rodagem em paralelo (acionar o método start()) das classes Thread criadas. Esse acionamento faz rodar o método run() das classes Thread. Desse modo, dependendo de como se criou a thread (se herdando a classe Thread ou implementando a interface Runnable), temos a seguinte sintraxe de acionamento:

```
Exemplo ex = new Exemplo();

// quando Exemplo herda Thread

ex.start();
```

Ou:

```
Exemplo ex = new Exemplo();

// quando Exemplo implementa Runnable

Thread t = new Thread(ex);

// ou…: Thread t = new Thread(ex, "Nome da Thread");

t.start();
```

Exercícios

Questão 1. (FGV 2018, adaptada) Avalie o código Java mostrado a seguir:

```
public class X
{
    public static void main(String [] args)
    {
        try
        {
            falha();
            System.out.print("A1");
        }
        catch (RuntimeException ex)
        {
            System.out.print("A2");
        }
        catch (Exception ex1)
        {
            System.out.print("A3");
        }
        finally
        {
            System.out.print("A4");
        }
        System.out.print("A5");
    }
    public static void falha()
    {
        throw new RuntimeException();
    }
}
```

Ao ser executado, o programa produz:

A) A1A2A3

B) A1A2A3A4

C) A2A4

D) A2A4A5

E) A2A3A4A5

Resposta correta: alternativa D.

Análise da questão

Para analisar o código, deve-se lembrar como funciona uma estrutura try-catch-finally, responsável pela captura e pelo tratamento de exceções em Java. A estrutura é iniciada com um bloco try, que tenta executar comandos que podem gerar exceções. Depois, há um ou mais blocos catch, responsáveis por tratar exceções específicas. Ao fim da estrutura, pode haver um bloco finally, que sempre é executado (caso exista). Vamos, agora, à análise do código da questão.

Ao executar o bloco try, é acionado o método falha(), que é responsável pelo lançamento da exceção RuntimeException. Depois, a execução avança para o bloco catch responsável por tratar essa exceção, imprimindo "A2". O bloco finally, que sempre será executado, é responsável por imprimir "A4". Após esse bloco, já fora da estrutura try-catch-finally, a execução do código continua e é feita a impressão "A5". Portanto, o programa produz "A2A4A5".

Questão 2. (Copese UFPI 2017, adaptada). As unidades concorrentes em Java são métodos denominados run, cujo código pode estar em execução simultânea com outros métodos (de outros objetos) e com o método main. Uma das maneiras de definir uma classe com um método run é definir uma subclasse da classe Thread predefinida e substituir o método run.

Sobre a classe Thread da linguagem Java, avalie as afirmativas a seguir.

I – A classe provê diversos métodos para controle de execução de threads, sendo o método start responsável por iniciar a execução do thread.

II – O método sleep é utilizado para forçar um método a atrasar sua execução até que o método run de outro thread tenha completado sua execução.

III – O método yield, que não possui parâmetros, é um pedido do thread em execução para entregar o processador voluntariamente.

É correto o que se afirma em:

A) I, apenas.

B) II, apenas.

C) I e III, apenas.

D) II e III, apenas.

E) I, II e III.

Resposta correta: alternativa C.

Análise das afirmativas

I – Afirmativa correta.

Justificativa: uma das maneiras de se trabalhar com threads em Java é criar uma classe que herda a classe Thread. Para acionarmos uma ou várias threads, é necessário rodar o método start() que existe, por herança, em cada classe que implemente essa funcionalidade. Quando acionamos o start, estamos iniciando um processamento em paralelo.

II – Afirmativa incorreta.

Justificativa: o método sleep() da classe Thread faz com que o thread em questão fique suspenso, ou "adormecido", por um intervalo de tempo pré-determinado. Portanto, sua função não é atrasar a execução de um método até que outro thread tenha completado sua execução.

III – Afirmativa correta.

Justificativa: o método yield() da classe Thread é capaz de parar a execução do thread atual, dando uma "oportunidade" de execução para outros threads, de mesma prioridade, que estão aguardando. Mesmo com essa capacidade, o método yield() nem sempre interrompe uma execução. Caso não haja outros threads aguardando, ou eles sejam de prioridade mais baixa, o thread atual continuará a ser executado.

REFERÊNCIAS

BARNES, D. J.; KÖLLING, M. *Programação orientada a objetos com Java*: uma introdução prática usando o BlueJ. São Paulo: Pearson Prentice Hall, 2004.

CLARO, D. B.; SOBRAL, J. B. M. *Programação em Java*. Florianópolis: Copyleft Pearson Education, 2008. Disponível em: https://cutt.ly/ITCBvIK. Acesso em: 3 dez. 2025.

DEITEL, P.; DEITEL, H. *Java*: como programar. São Paulo: Pearson Education do Brasil, 2017.

FURGERI, S. *Java 7*: ensino didático. 2. ed. São Paulo: Érica, 2012.

FURGERI, S. *Java 8*: ensino didático: desenvolvimento e implementação de aplicações. São Paulo: Érica, 2015.

HORSTMANN, C. *Conceitos de computação com Java*. 5. ed. Porto Alegre: Bookman, 2009.

HORSTMANN, C. S.; CORNELL, G. *Core Java, volume I*: fundamentos. São Paulo: Pearson Prentice Hall, 2010.

THE JAVA™ tutorials. *Oracle*, [s.d.]a. Disponível em: https://cutt.ly/YTDYEfo. Acesso em: 3 dez. 2025.

THE JAVA™ tutorials: formatting numeric print output. *Oracle*, [s.d.]b. Disponível em: https://cutt.ly/2TDTFDr. Acesso em: 3 dez. 2025.

THE JAVA™ tutorials: using JAR files: the basics. *Oracle*, [s.d.]c. Disponível em: https://cutt.ly/YTfIwV2. Acesso em: 3 dez. 2025.

OLIVEIRA, A. P.; MACIEL, V. V. *Java na prática*: volume I. Viçosa: Departamento de Informática da Universidade Federal de Viçosa, 2002.

SCHILDT, H. *Java para iniciantes*: crie, compile e execute programas Java rapidamente. 6. ed. Porto Alegre: Bookman, 2015.

TIMELINE of key Java milestones. *Oracle*, 2020. Disponível em: https://cutt.ly/LTDT8zY. Acesso em: 29 ago. 2021.

TIOBE Index for November 2025. *Tiobe*, [s.d.]. Disponível em: https://cutt.ly/wTa0X8E. Acesso em: 3 dez. 2025.

WINDER, R. *Desenvolvendo software em Java*. Rio de Janeiro: LTC, 2009.

UNIP

UNIVERSIDADE PAULISTA

Informações:
www.sepi.unip.br ou 0800 010 9000